

Module M13.C01 — Modéliser : entités, attributs, relations, cardinalités

Analyse de données appliquée & Business Intelligence
9 partie(s) assemblée(s) · document de production

Table des matières

MODULE M13.C01 — MODÉLISER : ENTITÉS, ATTRIBUTS, RELATIONS, CARDINALITÉS	8
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le plan et les matériaux	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Trois niveaux de lecture : MCD, MLD, MPD	10
5.2 Entité, attribut, occurrence, identifiant	11
5.3 La cardinalité et ses trois lectures	11
5.4 Les cinq entités du fil rouge, mesurées	11
5.5 Du MCD au MLD : où se pose la clé étrangère	12
5.6 Clé technique, clé métier : deux clés, deux rôles	12
5.7 Le MPD : exécuter le modèle et vérifier le total	13
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — la fiche client recopiée	13
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — compter les occurrences avant de dessiner	14
7.2 Étape 2 — prouver l'identifiant	14
7.3 Étape 3 — mesurer le coût d'une modification	14
7.4 Étape 4 — lire la relation au bon niveau	15
7.5 Étape 5 — écrire le MLD : les douze clés étrangères	15
7.6 Étape 6 — exécuter le MPD et contrôler le total	16
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	18
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	19
MODULE M13.C02 — NORMALISER : 1FN, 2FN, 3FN ET LES QUATRE ANOMALIES	21
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le vestiaire et les étiquettes	22
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 La 1FN — une valeur par champ	23
5.2 La 2FN — dépendre de la clé entière	23

5.3 La 3FN — dépendre de la clé, et d'elle seule	24
5.4 Les quatre formes et la quatrième, citée	24
5.5 Les quatre anomalies, mesurées sur le fil rouge	24
5.6 Quand ne pas normaliser	25
5.7 Une table de correspondance n'est pas une correction	25
6. Exemple concret — le classement qui change de tête	25
7. Démonstration pas à pas — quatre étapes de normalisation	26
7.1 Étape 1 — écrire les dépendances fonctionnelles	26
7.2 Étape 2 — chercher les dépendances qui ne partent pas de la clé	26
7.3 Étape 3 — mesurer ce que la correction rapporte	27
7.4 Étape 4 — corriger la donnée, pas la requête	27
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	18
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	19
MODULE M13.C03 — GRAIN, ADDITIVITÉ ET TABLES DE FAITS	32
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — l'eau, la photo et la température	33
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Ce que porte une ligne : le grain se dit en une phrase	35
5.2 Sept grains, un facteur 1 101	35
5.3 Le grain se prouve, il ne se déclare pas	35
5.4 Trois régimes d'additivité, et la règle qui va avec	36
5.5 Le théorème du grain : ne jamais mélanger les grains	36
5.6 Les trois façons de fabriquer un faux total	37
5.7 Le contrôle négatif : la jointure n'est pas toujours la coupable	37
5.8 La fiche de grain : une page qui évite les trois fautes	38
6. Exemple concret — six lignes qui expliquent tout le chapitre	39
7. Démonstration pas à pas — quatre gestes, dans cet ordre	39
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	18
14. Résumé du chapitre	19
15. À retenir	19

16. Évaluation formative	19
MODULE M13.C04 — LE SCHÉMA EN ÉTOILE ET LES CHANGEMENTS LENTS	46
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — l'étoile, la fiche et la photographie	47
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 L'étoile, et pourquoi pas autre chose	48
5.2 Les dimensions conformes, mesurées	49
5.3 Attributs et hiérarchies : ce qu'une dimension porte	49
5.4 Les quatre types de changement lent	49
5.5 La requête « au moment du fait »	50
5.6 Les deux trous, et pourquoi il faut les boucher exprès	50
5.7 Ce que la lecture historisée coûte, et ce qu'elle rapporte	51
5.8 La clé de version, et la ligne qu'on n'oublie pas	52
6. Exemple concret — deux fiches qui bougent	52
7. Démonstration pas à pas — quatre gestes	52
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	18
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	19
MODULE M13.C05 — LE SCHÉMA EN FLOCON ET SES VARIANTES	59
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — l'entrepôt, la fiche d'expédition et la caisse à objets perdus	60
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 L'étoile et le flocon : deux façons de ranger la même information	61
5.2 La table de pont : le N-M et son facteur de répétition	62
5.3 La dimension déchet : trois drapeaux, une table	63
5.4 Les faits multiples : quatre appareils, quatre unités	63
5.5 Dénormaliser exprès : la seule question qui compte	64
6. Exemple concret — le ticket qui vaut quatre fois son montant	64
7. Démonstration pas à pas — quatre gestes	52
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17

12. Correction détaillée	18
13. Mini-projet de chapitre	18
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	19
MODULE M13.C06 — LE TEMPS ET LE CALENDRIER	71
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le calendrier du mur et le cahier de caisse	72
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Pourquoi une dimension plutôt qu'une colonne de date	73
5.2 Ce que le calendrier du socle contient	74
5.3 Le branchement : fermer le trou déclaré au chapitre 4	74
5.4 Les attributs qu'aucun fait ne sait dire	75
5.5 Les deux calendriers du socle	75
5.6 L'exercice fiscal, et un nom qui promet plus que la mesure	76
5.7 Le mois partiel : deux vérités contradictoires sur la même table	76
5.8 Le temps dans les six autres faits	77
6. Exemple concret — les 46 commandes que la jointure interne effaçait	77
7. Démonstration pas à pas — cinq gestes	78
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	18
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	19
MODULE M13.C07 — LA QUALITÉ DU MODÈLE, SA DOCUMENTATION ET SA REVUE	85
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le contrôle technique et la fiche de visite	86
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Les quatre contrôles, et ce qu'ils ont en commun	87
5.2 Ce qu'aucun contrôle automatique ne voit	87
5.3 La documentation : ce que le script dit de lui-même	88
5.4 Le dictionnaire de données, colonne par colonne	88
5.5 La grille de revue en 15 points	89
5.6 Les cinq défauts du modèle fautif, et ce que chacun coûte	89

5.7 La revue coûte une heure, l'erreur coûte un trimestre	90
6. Exemple concret — cinq défauts, cinq totaux faux	90
7. Démonstration pas à pas — les quatre contrôles en SQL	91
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	18
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	19
PROJET M13.P — « LE MODÈLE DE SAHEL DISTRIBUTION »	97
1. Énoncé	97
Le matériel fourni	97
Les huit totaux de contrôle	97
2. Les quatre livrables	98
P1 — Le modèle conceptuel et le modèle logique (6 points)	98
P2 — Le script de création exécutable (6 points)	98
P3 — La revue du modèle en 15 points (5 points)	98
P4 — La note de choix (3 pages, 3 points)	99
3. Le barème	99
4. Ce qui fait rejeter un livrable	99
5. Déroulé conseillé (4 séances, environ 10 h)	100
6. Comment vous corriger vous-même	100
ÉVALUATION M13 — « MODÉLISATION DES DONNÉES »	101
A · Questions de récupération (non notées)	101
B · Quiz (15 questions · 20 points)	101
Bloc 1 — Définir (Q1 à Q5)	101
Bloc 2 — Mesurer (Q6 à Q10)	101
Bloc 3 — Modéliser (Q11 à Q15, 2 points chacune)	102
C · Deux exercices de normalisation (20 points)	102
N1 — Normaliser une table plate en 3FN (10 points)	102
N2 — Dénormaliser volontairement, et le chiffrer (10 points)	103
D · Étude de cas — « trois tableaux reliés donnent un chiffre d'affaires faux de 40 % » (30 points · seuil 18)	103
La commande	103
Le résultat attendu (à retrouver, pas à recopier)	104
Barème de l'étude de cas	104
E · Auto-test de contrôle du modèle (5 points)	104
Correction détaillée	104
A · Réponses de récupération	104
B · Corrigé du quiz	105
C · Corrigé des exercices de normalisation	105

D · Corrigé de l'étude de cas	106
E · Corrigé de l'auto-test	106

Module M13.C01 — Modéliser : entités, attributs, relations, cardinalités

Outils : DuckDB 1.5.5 (exécuté) ; pandas (contrôle croisé, exécuté) ; le script 03_exercices/dossier_M13/socle_m13.sql (exécuté à chaque ouverture du socle). PostgreSQL et Power BI (cités, non exécutés — règle §1.5). Durée indicative : 5 h. Niveau : N3 → N4. Pré-requis : M06 (tables, clés, relations), M11 (SQL avancé), M12 (indicateurs et architecture).

L'idée du chapitre. Un modèle n'est pas un rangement : c'est un **plan**. Le module M12 a appris à choisir les indicateurs ; celui-ci apprend à construire la structure qui les rend calculables. Le point de départ du fil rouge tient en un fichier plat de **18 lignes** — et le défaut y est déjà visible : le nom du client y est recopié **278** caractères pour **92** caractères utiles, et le client 41 y porte déjà **2** villes différentes. À l'échelle du socle, le même défaut recopie **2 826 394** caractères de noms de clients là où **239 599** suffisent. Ce chapitre remplace la recopie par un plan : **5** entités, **7** associations, **12** clés étrangères, **12** tables.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **distinguer** les trois niveaux de lecture d'un modèle — **conceptuel** (ce que le métier décrit), **logique** (tables, clés, colonnes) et **physique** (le code exécuté par un moteur) — et dire lequel vous êtes en train d'écrire quand vous dessinez ou quand vous tapez du SQL ;
2. **nommer** les objets d'un modèle : entité, attribut, occurrence, identifiant, relation, cardinalité, clé primaire, clé étrangère, clé technique, clé métier ;
3. **lire une cardinalité de trois façons** — la phrase métier, la notation $0,N / 1,N$, et la traduction en clés — et vérifier chacune par une mesure sur le socle, plutôt que par une intuition ;
4. **choisir entre clé technique et clé métier**, et **justifier** ce choix par un cas du fil rouge (la ligne de vente a les deux : `id_vente` et `id_ticket`) ;
5. **passer d'un fichier plat au modèle** et **mesurer** ce que le passage rapporte : lignes à modifier, caractères recopiés, et surtout le **faux total** qu'un mauvais niveau de relation fabrique — les ventes jointes aux commandes sur le client valent **x 0,31**, soit **4 804 233 057** FCFA au lieu de **15 595 154 955** FCFA.

2. Pourquoi cette notion est importante

Vous avez déjà vu ce qu'un modèle mal conçu produit comme dégâts, séparément : un chiffre d'affaires multiplié par **15,8** pour une clé oubliée (M12.C03), une rotation divisée par douze pour une somme de stocks (M12.C01), une vente attribuée au mauvais magasin parce que la clé qui décide ce qu'est « une ligne » n'était pas dans la jointure (M11.C01). Ce chapitre remonte d'un cran : il ne corrige plus la requête, il construit la **structure** qui rend la faute visible.

La scène est toujours la même. Un commercial demande « la fiche du client 41 ». On lui répond en ouvrant le fichier de ventes : 4 lignes, deux villes — Ouagadougou pour les ventes de 2023, Bobo-Dioulasso ensuite. Aucune des deux n'est fautive : c'est le **modèle** qui est ambigu. La ville n'est pas un attribut de la vente, c'est un attribut du **client**, et un attribut du client rangé dans la table des ventes n'a plus de valeur unique dès que le client déménage.

Définition. Un **modèle de données** est la description, écrite et vérifiable, des objets dont parle une entreprise (les entités), de leurs caractéristiques (les attributs) et des liens qui les unissent (les relations) — y compris le nombre d’occurrences que chaque lien autorise (les cardinalités). « Vérifiable » est le mot qui compte : un modèle dont on ne peut pas prouver les affirmations n’est pas un modèle, c’est un schéma d’intention.

Le coût d’un modèle absent ne se paie pas en élégance, il se paie en heures et en chiffres faux. Sur le socle du fil rouge, la même information — le nom du client — coûte **2 826 394** caractères quand elle est recopiée sur chaque ligne de vente, contre **239 599** quand elle est rangée une fois par client : **× 11,8**. Et le jour où un client change de ville, il faut modifier ses **20** lignes dans le fichier plat, contre **1** ligne dans la dimension — en espérant ne pas en oublier une, ce qui est exactement le défaut que ce chapitre supprime.

Attention. Un fichier plat ne « marche pas » par hasard : il marche **jusqu’au premier changement**. Une vente ne change pas ; un client change. Tant que les clients sont stables, le fichier plat rend les mêmes totaux que le modèle — c’est pour cela que le défaut survit des années. Le modèle ne se justifie pas par les totaux d’aujourd’hui, mais par les totaux du jour où quelque chose bouge.

3. Explication simple — le plan et les matériaux

Pour construire une maison, on distingue trois choses : ce que le client demande (« trois chambres, une cuisine qui donne sur la cour »), le plan de l’architecte (murs, ouvertures, cotes), et les matériaux posés (parpaings, ciment, tôles). Personne ne confond les trois — et personne ne construit une maison en empilant des parpaings là où ça tombe bien.

Vous demandez	Plan	Matériaux
« 240 000 lignes de ventes, 23 912 clients »	le modèle : 5 entités, 7 associations, 12 clés étrangères	socle_m13.sql exécuté sur DuckDB
Entités : client, produit, magasin, vendeur, temps	la table des cardinalités	les lignes de dim_client
Le sujet d’une ligne	le grain écrit sous chaque table	COUNT(*) = COUNT(DISTINCT clé)

Un modèle se dessine **deux fois** : une fois avec les mots du métier, une fois avec des clés. La première version, tout le monde la comprend et personne ne peut l’exécuter. La seconde version, personne ne la comprend sans effort et la machine l’exécute sans discuter. Le travail de modélisation consiste à passer de l’une à l’autre **sans perdre d’information** — et à écrire, à chaque étape, ce qu’on a perdu volontairement.

4. Vocabulaire essentiel

Terme	Définition	Sur le fil rouge
Entité	Ce dont on garde la trace, au singulier, indépendamment des événements	un client, un produit, un magasin, un vendeur, un jour
Occurrence	Un exemplaire d’une entité — une ligne d’une dimension	le client 41, le magasin 6 (le dépôt)

Terme	Définition	Sur le fil rouge
Attribut	Une caractéristique de l'entité, qui dépend d'elle seule	la ville d'un client, la désignation d'un produit
Identifiant	L'attribut (ou la combinaison) qui désigne une occurrence et une seule	<code>id_client</code> , et pour un stock : produit × mois
Relation	Un lien entre deux entités, décrit par une phrase du métier	« un client passe plusieurs commandes »
Cardinalité	Le nombre d'occurrences autorisées de part et d'autre d'une relation	un magasin → 0,N ventes ; une vente → 1,1 magasin
MCD	Modèle conceptuel : les entités et les relations, sans clés ni types	le dessin du §5.4
MLD	Modèle logique : les tables, les colonnes, les clés	les 12 tables du socle
MPD	Modèle physique : le code exécuté par un moteur, avec ses types	<code>socle_m13.sql</code> sous DuckDB
Clé primaire	L'identifiant déclaré d'une table (PRIMARY KEY)	<code>dim_client(id_client)</code>
Clé étrangère	Une colonne qui pointe vers la clé primaire d'une autre table (FOREIGN KEY)	<code>fait_ventes.id_produit</code> → <code>dim_produit.id_produit</code>
Clé technique	Un identifiant fabriqué par le système, sans sens métier (surrogate key)	<code>id_vente</code> : 1 à 240 000
Clé métier	Un identifiant venu du métier, stable pour l'utilisateur	<code>id_ticket</code> : T01-230101-000000

Définition. Une **clé technique** (surrogate key) est un nombre créé par le système, sans signification pour le métier, qui sert à désigner une ligne de façon stable. Une **clé métier** est un identifiant qui vient du métier — un numéro de ticket, un code produit — et qui peut changer, être reformaté, ou se répéter d'un millésime à l'autre. Un modèle sérieux porte **les deux** : la technique pour les jointures, la métier pour parler aux utilisateurs.

5. Cours approfondi

5.1 Trois niveaux de lecture : MCD, MLD, MPD

Le **modèle conceptuel** (MCD) décrit le métier : des entités, des relations, des cardinalités. Il ne parle ni de type de donnée, ni de clé, ni de moteur. Sa qualité se juge à une question : un gestionnaire de Sahel Distribution reconnaît-il sa propre entreprise dans ce dessin ?

Le **modèle logique** (MLD) descend d'un cran : chaque entité devient une table, chaque relation devient une clé étrangère placée du côté « plusieurs », et chaque table reçoit un grain écrit. C'est le document de travail des équipes : c'est lui qu'on relit en revue.

Le **modèle physique** (MPD) est le code : les types, les index, les contraintes, le moteur. C'est le seul des trois qui s'exécute, et donc le seul dont la vérité se **mesure**.

Définition. Une **cardinalité** décrit combien d'occurrences d'une entité peuvent être reliées à une occurrence de l'autre. On l'écrit par deux nombres : le minimum (0 ou 1) et le maximum (1 ou N). 0,N se lit « de zéro à plusieurs » : l'entité n'est pas obligée d'apparaître. 1,1 se lit « exactement une » : le lien est obligatoire et unique.

5.2 Entité, attribut, occurrence, identifiant

Le test qui sépare une entité d'un attribut tient en une phrase : **si l'information peut changer sans qu'aucun autre objet ne change, elle appartient à son propre objet**. Une ville de client change sans que la vente change : la ville est un attribut du client, pas de la vente. Une désignation de produit change sans que la vente change : même conclusion.

Le test qui sépare deux entités d'une seule : **peut-il exister sans l'autre ?** Un vendeur existe même les jours sans vente ; un magasin existe avant d'ouvrir. En revanche, la ligne de vente n'existe pas sans le client, le produit, le magasin, le vendeur et le jour : ce n'est pas une entité, c'est une **association porteuse** — ce que la suite du module appellera une **table de faits**.

Définition. Une **association porteuse** (ou **table de faits**) est une relation entre plusieurs entités qui porte elle-même des mesures et des attributs de l'événement — la quantité vendue, le montant, l'heure. Sa ligne est un **événement**, pas un objet : on n'y range jamais un attribut d'entité, sinon le modèle recopie et fabrique le défaut du §6.

5.3 La cardinalité et ses trois lectures

Une même relation se lit de trois façons, et il faut savoir faire les trois :

Lecture	Exemple sur le fil rouge	Ce qu'on en fait
La phrase métier	« un client passe zéro, une ou plusieurs commandes »	elle se discute avec le métier, pas avec un informaticien
La notation	client 1,N commande — ou 0,N si un client peut ne jamais commander	elle se dessine sur le MCD
La traduction	la clé <code>id_client</code> se pose dans la table <code>fait_commandes</code> , jamais dans <code>dim_client</code>	elle se code dans le MLD

La traduction obéit à une règle unique : **la clé étrangère se pose du côté « plusieurs »**. Un magasin a plusieurs ventes : `id_magasin` va dans les faits. Une vente a un seul magasin : on n'écrit jamais une colonne `id_vente` dans `dim_magasin` — ce serait admettre qu'un magasin n'a qu'une vente, ou alors impossible à remplir.

Définition. L'**optionalité** est la moitié gauche d'une cardinalité : elle dit si le lien est obligatoire. 0,N (facultatif côté entité pointée) et 1,N (obligatoire) se ressemblent dans un schéma et ne se ressemblent pas du tout dans un contrôle : une cardinalité 1,N fausse produit des **orphelins**, une cardinalité 0,N fausse produit des **lignes inutiles**. Sur le socle, `fait_ventes.id_vendeur` est obligatoire, `fait_ventes.id_client` ne l'est pas — le client 0 le prouve.

5.4 Les cinq entités du fil rouge, mesurées

Le socle de Sahel Distribution fournit ses propres réponses. Aucune cardinalité de ce tableau n'est déduite : la dernière colonne est la mesure qui l'établit, et le §7 la rejoue.

Entité	Dimension	Occurrences	Cardinalité avec les ventes	La mesure qui l'établit
Client	<code>dim_client</code>	23 912	0,N	23 496 clients achètent, 416 n'ont jamais acheté
Produit	<code>dim_produit</code>	154	1,N	de 873 à 2 672 lignes par produit, aucun produit à zéro
Magasin	<code>dim_magasin</code>	6	0,N	5 magasins vendent, le dépôt central ne vend pas

Entité	Dimension	Occurrences	Cardinalité avec les ventes	La mesure qui l'établit
Vendeur	dim_vendeur	22	1,N	les 22 vendeurs ont des ventes
Temps	dim_date	1 339	1,N	les 1 339 jours portent au moins une vente

Deux lignes de ce tableau méritent la révision : le magasin à zéro vente et le client à zéro achat. Dans les deux cas, la bonne réaction n'est pas de supprimer la ligne — c'est de **déclarer l'optionnalité**. Un dépôt qui ne vend pas reçoit des colis : il est indispensable à la table des livraisons. Un client qui n'a pas encore acheté est un prospect : il est indispensable au suivi commercial. Un modèle qui les supprime rend deux indicateurs impossibles à calculer.

Dans les faits. L'instrument du module rejoue ces cinq cardinalités à chaque exécution (`tools/modele_M13.py`) et publie le verdict des quatre contrôles de recette : **14** tables sur **14** ont une clé unique, les **7** tables de faits portent exactement les lignes de leur source, **0** orphelin sur les **12** clés étrangères vérifiées, et la recette du modèle égale celle de la source : **15 595 154 955** FCFA.

5.5 Du MCD au MLD : où se pose la clé étrangère

Le passage du conceptuel au logique suit quatre gestes, toujours les mêmes :

1. **une entité** → **une table** : `dim_client`, `dim_produit`, `dim_magasin`, `dim_vendeur`, `dim_date` ;
2. **une association porteuse** → **une table de faits** : `fait_ventes`, `fait_commandes`, `fait_encaissements`, `fait_stock_mensuel`, `fait ruptures`, `fait_logistique`, `fait_objectifs` ;
3. **une relation** → **une clé étrangère** du côté « plusieurs » : `id_produit` dans `fait_ventes`, et jamais `id_vente` dans `dim_produit` ;
4. **un attribut reste là où il dépend de son identifiant** : la ville dans `dim_client`, la quantité dans `fait_ventes`.

Le résultat sur le fil rouge : **5** dimensions, **7** tables de faits, **12** tables au total, et **12** clés étrangères. Ce dernier nombre n'est pas un détail de comptage : c'est la liste des contrôles d'intégrité que le §7.5 exécute.

5.6 Clé technique, clé métier : deux clés, deux rôles

La ligne de vente du fil rouge porte les deux, et c'est volontaire :

- `id_vente`, entier de **1** à **240 000** : la **clé technique**. Elle sert aux jointures, elle est courte, stable, et personne au métier ne la connaît ;
- `id_ticket`, texte du type `T01-230101-000000` : la **clé métier**. Elle porte du sens — le magasin (T01), la date (230101), un compteur — et c'est celle que le service client cite au téléphone.

Attention. La clé métier n'est pas une clé primaire fiable pour autant. `T01-230101-000000` contient la **date** : une facture antidatée, un ticket corrigé, et le numéro ne correspond plus à rien. Pire, un magasin qui change de code rend les anciens numéros ambigus. La règle du module : **la clé métier peut être unique, elle n'est jamais la clé de jointure** ; les **240 000** lignes de vente se joignent par `id_vente`, et les **146 161** tickets se regroupent par `id_ticket`.

Un ticket n'est pas une ligne : **146 161** tickets portent **240 000** lignes, soit **1,64** ligne par ticket (de **1** à **4**). Confondre les deux donne un panier moyen faux d'un facteur 1,64 — le genre d'erreur que M12 avait déjà chiffrée sur les paniers par jour.

5.7 Le MPD : exécuter le modèle et vérifier le total

Le modèle physique du module est écrit dans `03_exercices/dossier_M13/socle_m13.sql` : **12** tables, **5** dimensions, **7** tables de faits, plus **2** dimensions historisées que le chapitre C04 construira. Le fichier s'exécute à chaque ouverture du socle, et il finit par le contrôle qui compte le plus :

```
-- la recette : le chiffre d'affaires du modèle doit être IDENTIQUE à celui de la source
SELECT ROUND(SUM(montant_ttc)) AS ca_modele FROM fait_ventes WHERE
est_retour = 0;
SELECT ROUND(SUM(TRY_CAST(montant_ttc AS DOUBLE))) AS ca_source FROM ventes WHERE est_retour
= 0;
```

Ici, les deux nombres sont égaux : **15 595 154 955 FCFA**. Ce n'est pas une coquetterie : un modèle qui ne rend pas le même total que sa source a déplacé ou perdu des lignes, et tout ce qui suit (indicateurs, tableaux de bord, décisions) hérite de l'erreur.

Conseil professionnel. Ne présentez jamais un modèle sans son contrôle de recette. Une revue de modèle qui commence par « combien de tables ? » et non par « combien de chiffre d'affaires ? » finit toujours par discuter de style. Le total, lui, ne discute pas.

5.8 Ce que ce chapitre ne couvre pas

La normalisation des référentiels est le sujet du chapitre C02 ; le grain et l'additivité sont celui de C03 ; le schéma en étoile et les changements lents, celui de C04 ; le flocon et les variantes, C05 ; le temps et le calendrier, C06 ; la qualité, la documentation et la revue, C07. La traduction du modèle en relations d'un outil de restitution (Power BI) appartient au module M14 : elle est **citée** ici une fois, jamais exécutée — règle §1.5.

6. Exemple concret — la fiche client recopiée

Le dossier du module contient un extrait volontairement plat : `03_exercices/dossier_M13/table_plate.csv`. Ce n'est pas un mauvais fichier, c'est un fichier **d'avant le modèle** : une ligne par vente, tous les attributs recopiés.

Ce qu'on y trouve	Mesure
Lignes	18
Clients distincts	6
Libellés de catégorie distincts	11 — pour 7 familles réelles (M12)
Caractères de noms de clients recopiés	278 pour 92 caractères utiles
Nom le plus répété	« SARL Faso Batiment », 4 fois sur 18 lignes
Villes portées par le client 41	2 (Ouagadougou, puis Bobo-Dioulasso)

Deux défauts cohabitent dans ce petit fichier, et ils sont de nature différente :

1. **la redondance** — le nom du client est recopié **4** fois pour « SARL Faso Batiment ». Ce n'est pas faux, c'est coûteux : sur le socle complet, la même recopie pèse **2 826 394** caractères contre **239 599** rangés une fois (**× 11,8**) ;
2. **l'ambiguïté** — le client 41 a **2** villes. Ici, la question n'est plus le coût mais le **sens** : « la ville du client 41 » n'a pas de réponse unique dans ce fichier. Une question simple devient indécidable.

Le modèle répond aux deux par un seul geste : une table `dim_client` de **8** colonnes, une ligne par client, et une clé `id_client` posée dans les **240 000** lignes de ventes. La redondance disparaît par construction ; l'ambiguïté disparaît parce que la ville n'appartient plus qu'à un seul endroit — et si elle doit changer dans le temps, c'est le chapitre C04 qui s'en charge, avec une période de validité.

À retenir. Un attribut rangé au mauvais endroit ne provoque pas une erreur : il provoque une **question sans réponse**. C'est pour cela qu'un modèle se juge sur les questions qu'il rend possibles, pas sur les totaux qu'il rend justes.

7. Démonstration pas à pas — six étapes sur le socle

Toutes les requêtes de cette section s'exécutent sur le socle du module, ouvert ainsi :

```
import sys
CHEMIN = '03_exercices/dossier_M13'
sys.path.insert(0, CHEMIN)
from connexion import ouvrir
con = ouvrir()          # rejoue M11, puis M12, puis le modèle M13
```

7.1 Étape 1 — compter les occurrences avant de dessiner

Une cardinalité se mesure, jamais ne se devine. On commence donc par compter les occurrences des cinq entités et la façon dont elles se relient aux ventes :

```
SELECT (SELECT COUNT(*) FROM dim_client) AS clients,
       (SELECT COUNT(*) FROM dim_produit) AS produits,
       (SELECT COUNT(*) FROM dim_magasin) AS magasins,
       (SELECT COUNT(*) FROM dim_vendeur) AS vendeurs,
       (SELECT COUNT(*) FROM dim_date) AS jours;
```

Résultat : **23 913, 154, 6, 22, 1 339**. Le premier nombre vaut **23 912** clients du référentiel **plus** la ligne « client non identifié » — une ligne qui n'existe pas dans le référentiel et que le modèle ajoute exprès (voir l'étape 5).

7.2 Étape 2 — prouver l'identifiant

Un identifiant n'est un identifiant que s'il est **unique**. La preuve tient en une ligne, et elle doit être exécutée sur chaque table :

```
-- le grain de chaque table est PROUVÉ, pas supposé
SELECT COUNT(*) AS lignes, COUNT(DISTINCT id_vente) AS cles FROM fait_ventes;
```

240 000 et **240 000**. Le même contrôle passe sur les **14** tables du modèle : **14** clés uniques, y compris les deux tables de faits dont le grain est une **combinaison** — le stock (produit × mois) et les ruptures (produit × magasin × mois).

7.3 Étape 3 — mesurer le coût d'une modification

Combien de lignes faut-il modifier pour changer la ville d'un client ? Dans le fichier plat : toutes ses ventes. Dans le modèle : une seule ligne.

```
-- dans le fichier plat : 4 lignes pour un client de l'extrait
SELECT COUNT(*) FROM read_csv_auto('03_exercices/dossier_M13/table_plate.csv',
                                   header = true, delim = ',') WHERE id_client = 41;

-- dans le modèle : une ligne, et une seule
SELECT COUNT(*) FROM dim_client WHERE id_client = 41;
```

4 contre **1** sur l'extrait ; **20** contre **1** pour le client le plus actif du socle. Le rapport n'est pas le sujet : ce qui compte est qu'à **240 000** lignes, la version plate demande une mise à jour par ligne, et que l'oubli d'une seule ligne recrée le défaut du §6.

7.4 Étape 4 — lire la relation au bon niveau

Voici l'erreur que ce chapitre existe pour empêcher. Deux tables parlent du client : `fait_ventes` (240 000 lignes) et `fait_commandes` (9 000 lignes). Les joindre par le client semble naturel :

```
-- FAUX : deux tables jointes sur le client, donc au niveau « plusieurs à plusieurs »
SELECT ROUND(SUM(v.montant_ttc)) FROM fait_ventes v
JOIN fait_commandes c ON c.id_client = v.id_client
WHERE v.est_retour = 0;      -- 4 804 233 057 FCFA
```

Le chiffre d'affaires tombe à **4 804 233 057** FCFA, soit **× 0,31**. La requête s'exécute, ne lève aucune erreur, et le total obtenu a l'air crédible : c'est le pire des cas. La même faute avec les encaissements donne **4 797 608 252** FCFA, également **× 0,31**.

La cause est une question de modèle, pas de SQL : les deux tables n'ont pas le même sujet de ligne. Réunir deux grains différents dans une seule jointure oblige le moteur à choisir un appariement — et le seul appariement disponible (le client) multiplie ou filtre. La correction n'est pas dans la requête, elle est dans le modèle : **on ne joint pas deux faits ; on les agrège séparément, puis on rapproche leurs résultats par la dimension commune**, ce que le module M12 faisait déjà pour comparer ventes et objectifs.

Attention. Un total faux par mauvais niveau de relation est **plus dangereux** qu'un total faux par erreur de saisie : il est stable, reproductible, et il se défend en réunion. **4 804 233 057** FCFA ne ressemble pas à une faute de frappe — il ressemble à un chiffre.

7.5 Étape 5 — écrire le MLD : les douze clés étrangères

Le modèle logique se vérifie en énumérant ses clés étrangères, puis en les testant. Les **12** du socle, avec le contrôle d'orphelins qui va avec :

Table de faits	Clé étrangère	Pointe vers	Orphelins
<code>fait_ventes</code>	<code>id_produit</code> , <code>id_vendeur</code> , <code>id_client</code> , <code>id_magasin</code>	produit, vendeur, client, magasin	0
<code>fait_commandes</code>	<code>id_client</code> , <code>id_magasin</code>	client, magasin	0
<code>fait_encaissements</code>	<code>id_client</code>	client	0
<code>fait_stock_mensuel</code>	<code>id_produit</code>	produit	0
<code>fait_ruptures</code>	<code>id_produit</code> , <code>id_magasin</code>	produit, magasin	0
<code>fait_logistique</code>	<code>id_magasin</code>	magasin	0
<code>fait_objectifs</code>	<code>id_magasin</code>	magasin	0

Une seule ligne du référentiel ne suffisait pas : le client **0** apparaît dans **43 161** lignes de vente — **2 852 612 447** FCFA — et n'existe pas dans `dim_client`. Deux solutions sont défendables, et le module en a choisi une :

- **refuser le chargement** : la clé étrangère proteste, l'ingénieur corrige le référentiel ;
- **créer la ligne « inconnue »** : le modèle accepte le fait et le **déclare** — c'est le choix retenu, parce que supprimer **43 161** ventes réelles fausserait tous les totaux de **2 852 612 447** FCFA.

7.6 Étape 6 — exécuter le MPD et contrôler le total

Dernier geste : exécuter le modèle et comparer à la source. `tools/modele_M13.py` le fait en quatre contrôles, dans l'ordre où un modèle se vérifie — **unicité** (le grain), **grain** (les tables de faits portent les lignes de leur source), **orphelins** (les clés étrangères), **totaux** (la recette) :

```
1. unicite   : 14 tables sur 14 ont une cle unique
2. grain    : 7 tables de faits sur 7 portent exactement les lignes de leur source
3. orphelins : 12 cles etrangeres verifiees, 0 orphelin
4. totaux   : 15 595 154 955 FCFA par le modele, 15 595 154 955 FCFA par la source — identiques
```

Un modèle qui passe ces quatre contrôles n'est pas pour autant un bon modèle : il est **juste**. La qualité — lisibilité, documentation, usage des utilisateurs — se juge au chapitre C07.

8. Erreurs fréquentes

Erreur	Ce qu'elle produit	La parade
Ranger un attribut d'entité dans la table de faits	2 villes pour un client, une question sans réponse	un attribut n'existe qu'à un seul endroit
Poser la clé étrangère du côté « un »	une colonne impossible à remplir, ou un 1,1 faux	la clé se pose du côté « plusieurs »
Joindre deux tables de faits au niveau d'une dimension	x 0,31 de chiffre d'affaires (4 804 233 057 FCFA)	agréger séparément, rapprocher ensuite
Croire qu'une clé métier est une clé primaire	des doublons le jour d'une correction ou d'un antitadage	clé technique pour joindre, clé métier pour parler
Confondre ticket et ligne de vente	un panier moyen faux de x 1,64	écrire le grain sous chaque table
Supprimer une dimension à zéro occurrence	un dépôt sans vente, 416 clients sans achat	déclarer l'optionalité (0,N)

9. Bonnes pratiques professionnelles

- **Écrivez le grain sous chaque table**, dans le fichier lui-même, au-dessus du `CREATE` : un grain écrit se relit, un grain supposé se discute.
- **Vérifiez l'unicité par une requête**, pas par confiance : `COUNT(*) = COUNT(DISTINCT clé)` sur chaque table, à chaque chargement.
- **Déclarez la ligne inconnue** quand le référentiel est incomplet : une clé étrangère qui ne pointe nulle part est une décision, et une décision non écrite est un défaut.
- **Comptez les clés étrangères** de votre modèle et testez chacune : **12** ici, **0** orphelin — le jour où l'une d'elles casse, le contrôle le dit avant l'utilisateur.
- **Terminez par la recette** : le total du modèle contre le total de la source, à l'unité près.

Conseil professionnel. En revue de modèle, la première question n'est pas « combien de tables ? » mais « **qu'est-ce qu'une ligne ?** », posée table par table. Un modèle dont les grains sont écrits se relit en vingt minutes ; un modèle sans grains produit trois réunions.

À retenir. Un modèle de données est une **décision écrite**. Chaque clé étrangère est une décision (« ce fait appartient forcément à ce magasin »), chaque ligne inconnue en est une autre (« nous acceptons des faits non identifiés »). Ce qui n'est pas écrit sera décidé par accident — et l'accident, ici, porte un nom : **4 804 233 057 FCFA**.

10. Exercice guidé

Sujet. Le service commercial veut une table des clients livrés par magasin : combien de clients distincts chaque magasin a servis, et combien de lignes de vente il a enregistrées. Objectif : écrire la requête **au bon niveau**, puis montrer ce que produirait la version fautive.

Étape 1 — compter les lignes et les clients séparément, par magasin.

```
SELECT id_magasin,
       COUNT(*)           AS lignes_de_vente,
       COUNT(DISTINCT id_client) AS clients_servis
FROM fait_ventes
GROUP BY id_magasin
ORDER BY lignes_de_vente DESC;
```

Deux mesures de nature différente cohabitent dans le même `SELECT`, et c'est légitime : `lignes_de_vente` s'additionne d'un magasin à l'autre, `clients_servis` ne s'additionne pas (un client peut acheter dans deux magasins). Le magasin 1 sert **81 769** lignes, le magasin 5 en sert **26 496**.

Étape 2 — vérifier l'additivité avant de publier un total. Additionnez les `clients_servis` des cinq magasins : **90 679**, contre **23 496** clients réels. Le total dépasse le réel de près de quatre fois, parce qu'un client achète dans plusieurs magasins. C'est la démonstration, vue en M12.C03, que ce total ne se publie pas : il se **recalcule** par un `COUNT(DISTINCT)` général.

Étape 3 — écrire la version fautive et l'expliquer. Joindre les ventes aux clients (`dim_client`) puis aux commandes multiplie les lignes : le `COUNT(*)` n'a plus aucun sens, et un `SUM(montant_ttc)` tombé à **× 0,31** non plus. La règle à écrire dans le rapport : **une requête qui change le nombre de lignes doit être annoncée et justifiée**.

11. Exercices autonomes

Exercice 1.1. Écrivez la phrase métier des 5 cardinalités du §5.4 en français, sans symbole, et vérifiez chacune par une requête sur le socle. Une des cinq comporte un 0 qui n'est pas une erreur : laquelle, et pourquoi faut-il la garder ?

Exercice 1.2. Le client 41 de `03_exercices/dossier_M13/table_plate.csv` porte 2 villes. Écrivez la requête qui montre l'ambiguïté, puis celle qui, sur le modèle, répond sans ambiguïté. Que faudrait-il de plus pour répondre à la question « quelle était sa ville le 14 février 2023 » ?

Exercice 1.3. Choisissez trois tables du modèle et écrivez, pour chacune, la requête qui prouve son grain. Deux d'entre elles ont un grain composé : nommez-les et dites ce qui se passerait si l'on omettait une des deux colonnes de la clé.

Exercice 1.4. Un stagiaire propose de supprimer `dim_magasin` et d'écrire directement le nom du magasin dans `fait_ventes` : « cela évite une jointure ». Rédigez une réponse de dix lignes qui

chiffre le coût de cette décision (lignes à modifier, cas du dépôt, cas du magasin qui change de responsable) et concluez.

12. Correction détaillée

Exercice 1.1. 1) « un client peut passer zéro, une ou plusieurs ventes » — **23 496** clients achètent, **416** n'achètent pas : 0,N. 2) « un produit du catalogue peut être vendu une ou plusieurs fois » — de **873** à **2 672** lignes, aucun produit à zéro : 1,N (la cardinalité est obligatoire **parce que** le catalogue ne contient que des produits vivants ; si le catalogue admettait un article jamais vendu, elle deviendrait 0,N). 3) « un magasin peut vendre zéro, une ou plusieurs fois » — le dépôt central ne vend pas : 0,N. 4) « un vendeur vend une ou plusieurs fois » — les **22** vendeurs ont des ventes : 1,N. 5) « un jour du calendrier porte une ou plusieurs ventes » — les **1 339** jours en portent au moins une : 1,N. Le 0 à conserver est celui du **magasin** : le dépôt reçoit des colis, donc il appartient au modèle — le supprimer rendrait la table logistique incomplète.

Exercice 1.2. L'ambiguïté se montre en comptant les villes distinctes par client :

```
SELECT id_client, COUNT(DISTINCT ville_client) AS villes
FROM read_csv_auto('03_exercices/dossier_M13/table_plate.csv', header = true, delim = ',')
GROUP BY id_client HAVING COUNT(DISTINCT ville_client) > 1;
```

Le modèle répond sans ambiguïté parce que la ville n'est écrite qu'une fois : `SELECT ville FROM dim_client WHERE id_client = 41` donne **une** ligne. Pour répondre « au 14 février 2023 », il faut plus que le modèle de ce chapitre : il faut une **période de validité** sur l'attribut — c'est exactement la dimension historisée de type 2 du chapitre C04 (**24 892** versions client).

Exercice 1.3. `dim_client : COUNT(*) = COUNT(DISTINCT id_client)` — **23 913** lignes, **23 913** clés. `dim_date` : la clé est la date elle-même, **1 339** = **1 339**. Les deux grains composés sont `fait_stock_mensuel` (**produit × mois**) et `fait ruptures` (**produit × magasin × mois**). Omettre `mois` dans le premier ferait de la clé un produit : **154** clés pour **6 776** lignes, donc **6 776** lignes pour un même produit — le grain serait faux et la somme des stocks par produit multiplierait le résultat.

Exercice 1.4. Réponse attendue : écrire le nom du magasin dans les faits recopie **1** valeur sur **240 000** lignes ; corriger un nom demande **240 000** modifications, une par ligne ; le dépôt central (**1** magasin sur **6**) n'a aucune vente, il disparaîtrait donc du modèle — et avec lui la table logistique de M12 (**264** lignes) ; et un magasin qui change de responsable changerait d'identité ou garderait l'ancienne selon les lignes. Conclusion : la jointure économisée coûte **240 000** recopies pour économiser une jointure sur une table de **6** lignes. La bonne réponse n'est pas « c'est plus lent », c'est « c'est indécidable ».

13. Mini-projet de chapitre

« **Le MCD des cinq entités, mesuré** » (2 h). Produisez, pour le fil rouge : (1) le **MCD** des **5** entités et de leurs associations, avec les cardinalités écrites en clair ; (2) le **MLD** en tableaux, chaque table portant son grain écrit au-dessus ; (3) la **liste des 12 clés étrangères** avec, pour chacune, la requête d'orphelins correspondante ; (4) la **phrase métier** de chaque cardinalité, relue à voix haute. Livrable : deux pages et les requêtes exécutées. Ce travail est directement la matière de P1 du projet de module.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Trois niveaux	MCD (le métier), MLD (les clés), MPD (le code exécuté)
Entités du fil rouge	client 23 912 , produit 154 , magasin 6 , vendeur 22 , temps 1 339
Cardinalités	0,N client (416 sans achat), 0,N magasin (le dépôt), 1,N produit et vendeur
Associations	7 tables de faits, 5 dimensions, 12 clés étrangères
Clés	technique pour joindre (<code>id_vente</code>), métier pour parler (<code>id_ticket</code>)
Ticket contre ligne	146 161 tickets, 240 000 lignes, 1,64 ligne par ticket
Redondance	2 826 394 caractères recopiés contre 239 599 rangés une fois (× 11,8)
Mauvais niveau	ventes × commandes sur le client : × 0,31 , soit 4 804 233 057 FCFA
Recette	15 595 154 955 FCFA, modèle = source

15. À retenir

À retenir. Modéliser, c'est décider **ce qu'est une ligne** — et l'écrire. Les trois niveaux (conceptuel, logique, physique) ne sont pas trois documents à produire : ce sont trois façons de relire le même objet, pour trois publics différents.

- **Une cardinalité se mesure** : **416** clients sans achat, **1** dépôt sans vente, **22** vendeurs actifs — ces trois nombres sont écrits dans le modèle, pas observés après coup.
- **La clé étrangère se pose du côté « plusieurs »** ; l'inverse produit des colonnes impossibles à remplir.
- **Deux tables de faits ne se joignent pas** : elles s'agrègent séparément — × **0,31** et **4 804 233 057** FCFA sont le prix de l'oubli.
- **La ligne inconnue est une décision** : **43 161** ventes non identifiées (**2 852 612 447** FCFA) restent dans les totaux parce que le modèle a choisi de les déclarer.
- **Tout modèle se termine par une recette** : **15 595 154 955** FCFA des deux côtés, ou le modèle n'est pas prêt.

16. Évaluation formative

1. Donnez la définition d'une **entité**, d'un **attribut** et d'une **association porteuse**, avec un exemple du fil rouge pour chacune.
2. Quelle est la différence entre le MCD, le MLD et le MPD ? Lequel s'exécute ?
3. Écrivez la cardinalité client → ventes en notation, puis en phrase métier, puis en clés.
4. Pourquoi la clé étrangère ne se pose-t-elle jamais du côté « un » ?
5. Combien de clients du socle n'ont jamais acheté, et que fait-on de leur ligne ?
6. Quelle est la différence entre `id_vente` et `id_ticket` ? Laquelle utilise-t-on pour joindre, et pourquoi ?
7. **146 161** tickets pour **240 000** lignes : qu'est-ce que cela change pour un panier moyen ?
8. Joindre ventes et commandes sur le client donne **4 804 233 057** FCFA. Expliquez la cause et proposez la correction.
9. Pourquoi le nom de client est-il recopié **11,8** fois dans le fichier plat, et que coûte cette copie ?
10. Quels sont les quatre contrôles de recette du modèle, et que vérifie chacun ?

Corrigé : 1. Entité : le client ; attribut : sa ville ; association porteuse : la ligne de vente, qui porte la quantité et le montant. 2. Le conceptuel décrit le métier, le logique les tables et les clés, le physique le code ; seul le **physique** s'exécute. 3. **0,N** ; « un client peut passer zéro, une ou plusieurs ventes » ; `id_client` se pose dans `fait_ventes`. 4. Parce qu'elle obligerait à écrire une ligne unique dans la table « un » pour chaque ligne de la table « plusieurs » : impossible à remplir. 5. **416** ; on garde leur ligne, sinon la relation devient obligatoire et les prospects disparaissent du suivi. 6. `id_vente` est technique (**1** à **240 000**), `id_ticket` est métier (**T01-230101-000000**) ; on joint par la technique, parce que la métier porte une date et peut devenir ambiguë. 7. Il ne faut pas compter les tickets comme des lignes : **1,64** ligne par ticket, donc un panier calculé sur les lignes est faux d'un facteur **1,64**. 8. Les deux tables n'ont pas le même sujet de ligne ; la jointure se fait au niveau du client et multiplie ou filtre : on agrège séparément, puis on rapproche par la dimension commune. 9. Parce que la ville du client n'est pas un attribut de la vente ; la recopie coûte **2 826 394** caractères contre **239 599**, et le jour du déménagement elle coûte une mise à jour par ligne. 10. Unicité (le grain), grain (les lignes de la source), orphelins (les **12** clés étrangères) et totaux (la recette de **15 595 154 955** FCFA).

Module M13.C02 — Normaliser : 1FN, 2FN, 3FN et les quatre anomalies

Outils : DuckDB 1.5.5 (exécuté) ; pandas (contrôle croisé, exécuté). PostgreSQL et Power BI (cités, non exécutés — règle §1.5). Durée indicative : 5 h. Niveau : N4. Prérequis : M13.C01 (entités, clés, cardinalités), M04 (qualité des données), M12.C03 (grain et additivité).

L'idée du chapitre. Normaliser, c'est retirer d'une table ce qui ne dépend pas de sa clé — et le mesurer. Le chapitre ne raisonne pas sur des exemples de manuel : il compare la **même** information rangée de deux façons sur le fil rouge. Dans la table plate, le référentiel écrit la même famille de produits de **4** façons, si bien que le classement par étiquette brute place la famille la plus lourde du réseau en **5e** position — **3 492 592 638** FCFA cachés derrière quatre écritures. Dans le modèle, la même requête prend **3 ms** au lieu de **6** : normaliser, ici, ne coûte pas une milliseconde de lecture. Ce qu'elle coûte, c'est l'écriture et la discipline — et ce chapitre les chiffre.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **énoncer** les trois premières formes normales sans les réciter : la 1FN interdit les valeurs multiples dans un champ, la 2FN interdit qu'un attribut ne dépende que d'une **partie** d'une clé composée, la 3FN interdit qu'un attribut dépende d'un autre attribut non clé ;
2. **détecter** une dépendance fonctionnelle dans une table donnée, et l'écrire sous la forme **déterminant** → **dépendant** — c'est le seul outil dont vous avez besoin pour normaliser ;
3. **reconnaître et nommer les quatre anomalies** qu'une table plate fait naître : redondance, mise à jour, insertion, suppression — et **mesurer** chacune sur le fil rouge ;
4. **corriger** une table non normalisée sur le socle, en créant la table de correspondance quand la donnée source est sale (**16** étiquettes pour **7** familles) plutôt qu'en corrigeant la requête ;
5. **décider quand ne pas normaliser**, et l'assumer par écrit : la dénormalisation volontaire d'une dimension de restitution est un choix, la redondance subie est un défaut.

2. Pourquoi cette notion est importante

Le chapitre C01 a montré qu'un attribut rangé au mauvais endroit rend une question indécidable. Celui-ci montre ce qui se passe **à l'échelle** : la même faute, répétée sur des milliers de lignes, ne rend plus seulement les questions indécidables — elle rend les **classements** faux.

Voici ce que le socle du fil rouge produit, mesuré aujourd'hui. Une requête sur la table plate — la table qui porte les étiquettes de catégorie telles qu'elles sont écrites dans le référentiel — classe les catégories dans cet ordre : Plomberie **3 418 137 911** FCFA, Electricité **2 469 701 472** FCFA, Bois & panneaux **1 727 101 681** FCFA. Trois lignes, un classement net, un tableau de bord prêt à publier. Et ce classement est **faux** : la première famille du réseau, matériaux, n'y figure pas, parce que son montant est éclaté entre quatre étiquettes — **matériaux**, **Matériaux** (avec une espace finale), **Materiaux**, **MATÉRIAUX** — soit **3 492 592 638** FCFA au total.

Définition. Une **dépendance fonctionnelle** $A \rightarrow B$ se lit : « si je connais A, je connais B sans regarder le reste de la ligne ». $\text{id_client} \rightarrow \text{nom_client}$ signifie qu'un identifiant de client détermine un nom, quel que soit le reste. Toute la normalisation tient dans une phrase : **un attribut ne doit dépendre que de la clé** — ni d'une partie de la clé (2FN), ni d'un autre attribut non clé (3FN).

Une table non normalisée ne produit pas d'erreur : elle produit des totaux plausibles et faux. C'est pourquoi la normalisation n'est pas un exercice d'école, mais le premier contrôle de qualité d'un système de restitution. Un chiffre éclaté en quatre, additionné sans faire exprès, passe tous les contrôles techniques — et se fait démolir en réunion par la première personne qui additionne à la main.

Attention. La normalisation n'a jamais pour but d'« économiser de la place ». Sur le fil rouge, la table plate recopie les noms de clients pour **2 826 394** caractères là où la dimension en range **239 599** — mais l'argument sérieux n'est pas là. L'argument est que la version plate rend le classement du paragraphe précédent faux. **La place est un symptôme ; la justesse est le sujet.**

3. Explication simple — le vestiaire et les étiquettes

Imaginez un vestiaire où chaque veste porte sa propre étiquette : le nom du propriétaire, sa taille, sa ville. Deux vestes du même propriétaire, et le nom est écrit deux fois. Le manteau change de propriétaire : il faut découdre l'étiquette et la recoudre — et si vous en oubliez une, la même personne devient deux personnes.

Le modèle fait l'inverse : une fiche par personne (**la dimension**), un numéro cousu sur chaque veste (**la clé étrangère**). Changer de propriétaire demande de modifier **une** fiche ; connaître le propriétaire d'une veste demande une recherche par numéro — la jointure.

Question	Vestiaire à étiquettes (table plate)	Vestiaire à fiches (modèle)
Où est la ville du client ?	sur chaque veste	sur la fiche, une fois
Mettre à jour une ville	autant de décousues que de vestes	1 fiche
Ajouter un client sans veste	impossible	une fiche, et c'est tout
Retirer la dernière veste	le client disparaît	la fiche reste

Les quatre lignes du bas sont exactement les **quatre anomalies** du §5.5. Elles ne sont pas théoriques : chacune se mesure sur le socle.

4. Vocabulaire essentiel

Terme	Définition	Sur le fil rouge
Dépendance fonctionnelle	$A \rightarrow B$: connaître A suffit à connaître B	$\text{id_produit} \rightarrow \text{designation}$
Dépendance partielle	$A \rightarrow B$ où A n'est qu'une partie de la clé	$\text{id_produit} \rightarrow \text{designation}$ quand la clé est (produit × mois)
Dépendance transitive	$A \rightarrow B \rightarrow C$, avec B non clé	$\text{id_produit} \rightarrow \text{libellé} \rightarrow \text{famille}$
1FN	Chaque champ porte une valeur atomique	une étiquette de catégorie, jamais une liste

Terme	Définition	Sur le fil rouge
2FN	1FN et aucun attribut ne dépend d'une partie de la clé	<code>fait_stock_mensuel</code> (produit × mois)
3FN	2FN et aucun attribut ne dépend d'un attribut non clé	la famille ne se range pas dans les faits
Anomalie redondance	de la même information écrite plusieurs fois	2 826 394 caractères recopiés
Anomalie de mise à jour	une modification à faire en plusieurs endroits	4 lignes pour un client de l'extrait
Anomalie d'insertion	impossible d'enregistrer une occurrence seule	416 clients sans vente
Anomalie suppression	de supprimer une ligne efface une information unique	le client 84 de l'extrait
Table correspondance	de table qui porte une règle de traduction du référentiel	16 étiquettes → 7 familles
Dénormalisation	redondance choisie et documentée	la famille recopiée dans <code>dim_produit</code>

Définition. Une **forme normale** est une propriété d'une table, pas d'une base : une table est en 1FN, en 2FN ou en 3FN — et elle peut l'être sans que ses voisines le soient. On dit d'un modèle qu'il est « en 3FN » quand **chacune** de ses tables l'est, ce qui n'est presque jamais le cas des modèles en étoile : leurs dimensions de restitution sont volontairement dénormalisées.

5. Cours approfondi

5.1 La 1FN — une valeur par champ

La première forme normale demande une chose simple : **un champ, une valeur**. Pas de liste dans une cellule, pas de « et », pas de virgule qui sépare deux valeurs. Ce n'est pas une règle de style : c'est la condition pour qu'une requête puisse filtrer, grouper et compter.

Définition. Une table est en **première forme normale** (1FN) quand chaque champ contient une valeur **atomique** — indivisible du point de vue du modèle — et quand il n'existe ni colonne répétitive (« produit 1 », « produit 2 », « produit 3 ») ni valeur composite rangée dans un seul champ (« Ciment, Tôle »). Une clé, elle, peut être composée : c'est la **valeur** qui doit être atomique, pas la clé.

Le module a rencontré cette faute dans son **propre** fichier, et c'est instructif. L'extrait `03_exercices/dossier_M13/table_plate.csv` contient deux lignes dont la désignation produit est `Cable électrique 2,5 mm`. La virgule, écrite sans guillemets, compte pour un séparateur de colonne : ces deux lignes portent **13** champs au lieu de **12**, et le fichier ne se lit plus — le moteur refuse de deviner. Le défaut était invisible à l'œil nu, aucun total ne s'en plaignait, et il a fallu **essayer d'ouvrir le fichier** pour le trouver.

5.2 La 2FN — dépendre de la clé entière

La deuxième forme normale ne concerne que les tables dont la clé est **composée**. Elle demande qu'aucun attribut ne dépende d'une partie seulement de cette clé.

Le fil rouge fournit l'exemple parfait : le relevé de stock mensuel a pour clé (produit × mois) et compte **6 776** lignes pour **154** produits. Si l'on recopiait la désignation du produit dans chaque

ligne du relevé, cette désignation dépendrait du **produit** seul — une partie de la clé. Recopier la désignation coûterait **196 328** caractères contre **4 462** rangés une fois, soit **× 44,0**.

Définition. Une table est en **deuxième forme normale** (2FN) quand elle est en 1FN et qu’aucun attribut non clé ne dépend d’une **partie** d’une clé composée. Corollaire pratique : une table dont la clé est simple est automatiquement en 2FN — la question ne se pose que pour les clés composées, c’est-à-dire pour les tables de faits dont le grain est une combinaison.

5.3 La 3FN — dépendre de la clé, et d’elle seule

La troisième forme normale interdit les dépendances **transitives** : $A \rightarrow B \rightarrow C$, avec B non clé.

Le référentiel produit du fil rouge porte **16** étiquettes de catégorie pour **7** familles réelles. L’étiquette détermine la famille (libellé $\rightarrow$ famille) et la famille ne détermine pas l’étiquette : c’est une dépendance transitive, donc une violation de la 3FN si l’on range la famille dans la même table que la vente. Le modèle a fait un choix précis : `dim_produit` porte **à la fois** l’étiquette d’origine (libelle_source, **16** valeurs) et la famille corrigée (**7** valeurs) — c’est une **dénormalisation assumée**, justifiée par le fait qu’une dimension de restitution se lit, alors qu’une table de faits s’additionne. Ce choix est discuté au §5.6, et il n’est pas gratuit.

Définition. Une table est en **troisième forme normale** (3FN) quand elle est en 2FN et qu’aucun attribut non clé n’en détermine un autre. On dit parfois qu’un attribut doit dépendre « de la clé, de toute la clé, et de rien d’autre que la clé » — c’est la formule à retenir, à condition de se souvenir qu’elle parle de **chaque** table séparément.

5.4 Les quatre formes et la quatrième, citée

Au-delà de la 3FN existent la **forme normale de Boyce-Codd** (BCNF, une variante plus stricte de la 3FN pour les clés multiples) et les formes 4 et 5, qui traitent des dépendances multivaluées. Le module les **cite** sans les démontrer : les modèles en étoile du programme vivent en 3FN, et la seule question qui se pose en pratique est de savoir **quand** une dénormalisation est un choix. Un cours qui s’arrête à la 3FN n’est pas un cours incomplet ; c’est un cours qui sait où il s’arrête.

5.5 Les quatre anomalies, mesurées sur le fil rouge

Anomalie	Ce qu’elle est	La mesure du fil rouge
Redondance	la même information écrite plusieurs fois	2 826 394 caractères de noms recopiés contre 239 599 rangés une fois (× 11,8) ; 278 contre 92 sur l’extrait (× 3,0)
Mise à jour	une modification à répéter partout	4 lignes pour changer la ville du client 41 dans l’extrait ; 20 lignes pour le client le plus actif du socle
Insertion	impossible de créer une occurrence seule	416 clients du référentiel n’ont aucune vente : le fichier plat ne peut pas les porter
Suppression	effacer une ligne efface une information unique	le client 84 (« SARL Nabonswendé ») tient sur 2 lignes : supprimer ses deux ventes le fait disparaître du fichier

Les deux dernières anomalies sont les plus coûteuses en pratique, parce qu’elles sont **silencieuses** : un client qui n’existe plus ne se voit pas dans un total de chiffre d’affaires. Sur le fil rouge, les **416** clients sans achat représentent l’avenir commercial de l’entreprise : ce sont des prospects, et un modèle qui ne peut pas les ranger ne peut pas les suivre.

Dans les faits. Le socle du module **peut** ranger ces quatre cas, et il le prouve : les **416** clients sans vente existent dans `dim_client` (anomalie d'insertion abolie) ; une vente peut être supprimée sans faire disparaître son client (anomalie de suppression abolie) ; modifier une ville touche **1** ligne au lieu de **20** (anomalie de mise à jour abolie) ; et les noms tiennent en **239 599** caractères au lieu de **2 826 394** (anomalie de redondance abolie).

5.6 Quand ne pas normaliser

Un modèle en étoile n'est **pas** en 3FN, et il a raison de ne pas l'être. Trois cas justifient une dénormalisation, à condition de l'écrire :

1. **la dimension de restitution** : `dim_produit` porte l'étiquette d'origine **et** la famille corrigée, parce qu'un rapport par famille doit pouvoir se comparer au référentiel tel qu'il est écrit — **16** étiquettes affichées, **7** lignes publiées ;
2. **la table d'export** : un fichier destiné à un tableur doit être plat, sinon il n'est pas utilisable (le socle du module en fournit un : `table_plate.csv`, **18** lignes) ;
3. **le cache de calcul** : une table agrégée par mois et par magasin recopie ce qui a déjà été calculé — c'est une redondance de **performance**, pas de modèle, et elle se régénère.

Attention. Une dénormalisation non écrite devient une redondance subie : personne ne sait plus laquelle des deux colonnes fait foi, et un jour elles se contrediront. La règle du module : **une redondance se déclare, avec sa raison et sa requête de régénération**. Sinon, elle se paiera en anomalies — les quatre du §5.5.

5.7 Une table de correspondance n'est pas une correction

Les **16** étiquettes pour **7** familles posent une question de gouvernance : où corrige-t-on ? Trois réponses possibles, et une seule est bonne :

- **dans la requête** : chaque analyste écrit son CASE, et deux analystes produisent deux classements — c'est ce que fait la table plate, et c'est pour cela que le classement du §2 est faux ;
- **dans la source** : on corrige le fichier du référentiel — la bonne cible, mais elle n'est pas toujours accessible, et l'historique reste sale ;
- **dans une table de correspondance du modèle** : la règle est écrite **une fois**, versionnée, et applicable à l'historique. C'est le choix du module.

Un **piège technique** accompagne la deuxième solution : deux étiquettes du référentiel portent une **espace finale** (Matériaux et Quincaillerie). Sans le TRIM avant la correspondance, la traduction laisse passer **9** familles au lieu de **7** — la table de correspondance a l'air juste, et le modèle compte deux familles fantômes. Le module a rencontré ce défaut, l'a mesuré et l'a corrigé.

6. Exemple concret — le classement qui change de tête

Le chapitre C01 a présenté l'extrait `table_plate.csv` (**18** lignes, **6** clients). Voici la même question posée de deux façons sur le socle complet : « quelle est la famille de produits qui pèse le plus ? »

Réponse par étiquette brute (table plate, **16** groupes) : Plomberie **3 418 137 911** FCFA, Electricité **2 469 701 472** FCFA, Bois & panneaux **1 727 101 681** FCFA.

Réponse par famille corrigée (7 groupes) : Materiaux **3 492 592 638** FCFA, Plomberie **3 418 137 911** FCFA, Electricite **2 469 701 472** FCFA, Quincaillerie **1 891 970 547** FCFA, Bois et

panneaux **1 727 101 681** FCFA, Consommables **1 375 225 530** FCFA, Peinture **1 045 255 378** FCFA.

La différence n'est pas dans la requête — les deux comptent des ventes — mais dans la **structure**. Dans la version brute, la famille matériaux est coupée en quatre morceaux :

Étiquette brute	Chiffre d'affaires
materiaux	1 351 022 329 FCFA
Matériaux (espace finale)	929 913 501 FCFA
Materiaux	923 055 165 FCFA
MATÉRIAUX	288 601 643 FCFA
Total de la famille	3 492 592 638 FCFA

Le plus gros fragment (**1 351 022 329** FCFA) n'apparaît qu'en **5e** position d'un classement par étiquette. Un analyste qui publie le « top 3 » de ce classement annonce trois familles et en oublie une — la première. C'est exactement la faute que le module M11 avait chiffrée sur les classements (publier un classement sans vérifier le libellé), et la parade est la même : **le libellé publié est celui de la mesure triée**, jamais celui du fichier source.

À retenir. Un mauvais modèle ne se contente pas de ralentir : il **classe faux**. Et un classement faux est bien plus difficile à contester qu'un total faux, parce qu'il a l'air d'une opinion — « matériaux est un gros poste » — alors que c'est une erreur de structure.

7. Démonstration pas à pas — quatre étapes de normalisation

7.1 Étape 1 — écrire les dépendances fonctionnelles

Avant de toucher aux tables, on écrit les dépendances sur le papier. Sur le fil rouge, la liste complète des tables de faits tient en quelques lignes :

```
id_client    → nom, ville, segment, conditions_paiement
id_produit   → designation, libelle_source, famille, sous_categorie, unite
id_magasin   → nom, ville, type_magasin, responsable
id_vendeur   → nom_complet, id_magasin, statut
id_vente     → date_vente, id_produit, id_client, id_magasin, id_vendeur, quantite, montant_ttc
(id_produit, mois)      → quantite_en_stock
(id_produit, id_magasin, mois) → est_en_rupture
```

Chaque flèche est une affirmation vérifiable : `id_produit → designation` se contrôle en comptant les désignations distinctes par produit — **154** produits, **154** désignations.

7.2 Étape 2 — chercher les dépendances qui ne partent pas de la clé

On relit la liste en cherchant deux motifs : une flèche qui part d'une **partie** de clé composée (2FN), et une flèche qui part d'un attribut non clé (3FN). Sur le fil rouge, la table plate en porte deux :

```
-- 3FN : la famille dépend de l'étiquette, pas de la clé de la table
SELECT COUNT(DISTINCT libelle_source) AS etiquettes,
       COUNT(DISTINCT famille)       AS familles
FROM dim_produit;                    -- 16 et 7
```

```
-- 2FN : la désignation ne dépend que du produit, pas du couple (produit, mois)
SELECT COUNT(*) AS lignes FROM fait_stock_mensuel;      -- 6 776
SELECT COUNT(DISTINCT id_produit) FROM fait_stock_mensuel; -- 154
```

7.3 Étape 3 — mesurer ce que la correction rapporte

Une normalisation se justifie par un écart. Trois écarts mesurés sur le socle :

Ce qu'on mesure	Table plate	Modèle
Lignes / colonnes de la table la plus large	240 000 lignes, 11 colonnes	240 000 lignes, 17 colonnes dans les faits, 50 colonnes dans les cinq dimensions
Caractères de noms de clients	2 826 394	239 599
Caractères de désignations (relevé de stock)	196 328	4 462
Groupes obtenus par catégorie	16	7
Durée de la même requête de groupe	6 ms	3 ms

La dernière ligne mérite qu'on s'y arrête : sur DuckDB, un moteur **en colonnes**, la version normalisée est **plus rapide** que la version plate, parce qu'elle ne lit que les colonnes utiles et que la dimension tient en **154** lignes. Le coût de la normalisation n'est donc pas là : il est dans l'écriture (deux tables à charger au lieu d'une), dans la discipline (une clé à respecter) et dans la documentation (une correspondance à écrire).

Attention. Les deux durées ci-dessus sont mesurées dans l'atelier du module, sur ce socle et ce moteur. Elles disent une **propriété du moteur** (un moteur en colonnes ne paie pas la jointure), pas une loi générale : sur un système ligne à ligne et sans index, la même requête normalisée peut coûter cher. La règle qui survit au changement de moteur : **mesurez chez vous, et ne citez pas la mesure d'un autre.**

7.4 Étape 4 — corriger la donnée, pas la requête

La famille est sale à la source (**16** étiquettes). Le module ne corrige pas le fichier : il écrit la correspondance **dans le modèle**, une fois, et il compte le résultat.

```
SELECT famille, COUNT(*) AS produits, ROUND(SUM(f.montant_ttc)) AS ca
FROM fait_ventes f JOIN dim_produit p ON p.id_produit = f.id_produit
GROUP BY 1 ORDER BY ca DESC;
```

Sept lignes en sortie, contre seize avec l'étiquette brute. Le TRIM de l'étape précédente n'est pas un détail de propreté : sans lui, cette requête rend **9** lignes et le modèle compte deux familles de plus que la réalité.

8. Erreurs fréquentes

Erreur	Ce qu'elle produit	La parade
Corriger la donnée dans chaque requête	deux analystes, deux classements (16 contre 7 groupes)	une table de correspondance, écrite une fois
Oublier le TRIM avant une correspondance	9 familles au lieu de 7	trimmer, puis mapper, puis compter
Normaliser sans écrire les dépendances	des tables coupées « au feeling », des jointures oubliées	écrire A → B avant de couper

Erreur	Ce qu'elle produit	La parade
Croire qu'une table à clé simple est en 3FN	une clé simple n'exempte que de la 2FN	vérifier les dépendances transitives
Supprimer la dernière ligne d'un client	le client disparaît, le prospect aussi	une dimension, une ligne par client
Dénormaliser sans le dire	deux colonnes qui finiront par se contredire	déclarer, justifier, régénérer

9. Bonnes pratiques professionnelles

- **Écrivez les dépendances fonctionnelles** de chaque table, en clair, avant tout `CREATE` : c'est le document de travail de la normalisation.
- **Cherchez les flèches qui ne partent pas de la clé** : partie de clé → 2FN, attribut non clé → 3FN.
- **Ne corrigez jamais une étiquette dans une requête** : une règle de traduction s'écrit une fois.
- **Contrôlez le nombre de groupes** après chaque correspondance : **16** étiquettes devenues **7** familles se vérifient en une ligne ; **9**, c'est un défaut.
- **Mesurez le coût de vos choix** : une dénormalisation se justifie par un chiffre, pas par une intuition (ici, **6** ms contre **3** ms).

Conseil professionnel. Quand un référentiel est sale, la tentation est de le nettoyer « plus tard ». Le module propose l'inverse : **consignez la saleté dans une table de correspondance**, publiez avec la correspondance, et nettoyez la source quand vous en aurez les moyens. Un modèle qui attend un référentiel propre ne sort jamais.

À retenir. Normaliser n'est pas ranger, c'est **décider de qui dépend de quoi**. Une table plate ne ment pas sur les totaux : elle ment sur les **classements** — Plomberie en tête au lieu de matériaux, et **3 492 592 638** FCFA invisibles dans le tableau.

10. Exercice guidé

Sujet. La table ci-dessous vient d'un service commercial. Elle n'est pas en 3FN. Trouvez les dépendances, nommez les violations, proposez le découpage, et chiffrez le gain.

id_vente	id_produit	designation	categorie	famille	id_client	ville_client	quantite
1	12	Ciment CPJ45 50 kg	Materiaux	Materiaux	41	Ouagadougou	40
2	12	Ciment CPJ45 50 kg	MATÉRIAUX	Materiaux	41	Ouagadougou	25
3	7	Peinture acrylique 20 l	PEINTURE	Peinture	41	Bobo-Dioulasso	6

Étape 1 — écrire les dépendances visibles.

```
id_vente → id_produit, id_client, quantite
id_produit → designation, categorie, famille
id_client → ville_client
categorie → famille
```

Étape 2 — nommer les violations. `id_produit → designation` viole la 3FN si ces colonnes sont rangées dans une table dont la clé est `id_vente` : la désignation dépend d'un attribut non clé. `categorie → famille` est une dépendance transitive. Et les deux lignes du produit 12 montrent la

conséquence : 2 étiquettes différentes pour la même famille, donc un classement par étiquette qui créera deux groupes.

Étape 3 — proposer le découpage. Trois tables : `dim_produit(id_produit, designation, categorie, famille)`, `dim_client(id_client, ville_client)`, `fait_ventes(id_vente, id_produit, id_client, quantite)`. La correspondance `categorie → famille` reste un choix à documenter : dans la dimension (choix du module) ou dans une table dédiée.

Étape 4 — chiffrer. Sur le fil rouge complet, cette même correction fait passer la recopie du nom de client de **2 826 394** à **239 599** caractères, la recopie de la désignation de **196 328** à **4 462**, et le nombre de groupes publiés de **16** à **7**.

11. Exercices autonomes

Exercice 2.1. Écrivez la liste complète des dépendances fonctionnelles de `dim_magasin` (6 lignes, 10 colonnes), puis dites, en justifiant, si la table est en 3FN. Une colonne pose question : `responsable` — dépend-elle du magasin, ou d'autre chose ?

Exercice 2.2. Construisez, à partir du socle, une table plate de 20 lignes environ qui contient la vente, le client, le produit et le magasin, puis appliquez-lui les quatre anomalies du §5.5 en donnant pour chacune la requête qui la démontre.

Exercice 2.3. Le référentiel écrit la famille « matériaux » de 4 façons, pour un total de **3 492 592 638** FCFA. Écrivez la requête qui le prouve, puis celle qui produit le classement correct. Combien de groupes obtient-on avant, combien après ?

Exercice 2.4. Un service vous livre un fichier de **1 200** lignes où la colonne `telephone` contient parfois deux numéros séparés par un point-virgule. Dites quelle forme normale est violée, écrivez la requête qui compte les lignes concernées, et proposez la correction : deux colonnes, ou une table de téléphones ?

12. Correction détaillée

Exercice 2.1. Dépendances : `id_magasin → nom, ville, quartier, region, type_magasin, surface_m2, ouverture` et `id_magasin → responsable`. La table est en 3FN **parce que** tout dépend de la clé et de rien d'autre : `responsable` dépend du magasin (un magasin a un responsable à la fois), et non d'un attribut intermédiaire. La question à se poser est celle du temps : si le responsable change chaque année et qu'on veut l'historique, `responsable` devient un attribut **variable** et appelle une dimension historisée — c'est le sujet du chapitre C04.

Exercice 2.2. La table plate se construit par jointure des faits et des dimensions ; les quatre anomalies se démontrent ainsi : **redondance** — comparer `SUM(LENGTH(nom_client))` sur **240 000** lignes et sur la dimension ; **mise à jour** — compter les lignes du client 41 dans la table plate (4 dans l'extrait, 20 pour le client le plus actif du socle) ; **insertion** — chercher un client du référentiel sans vente (**416** cas) : il n'apparaît dans aucune ligne de la table plate ; **suppression** — supprimer les lignes du client 84 de l'extrait (2 lignes) et constater que le client disparaît entièrement du fichier.

Exercice 2.3. La requête de preuve groupe par étiquette brute en filtrant sur le motif `MAT%` : quatre lignes, dont la plus grosse vaut **1 351 022 329** FCFA ; leur somme vaut **3 492 592 638** FCFA. Le classement correct groupe par `famille` : 7 groupes, avec Matériaux en tête. Avant : 16 groupes. Après : 7.

Exercice 2.4. La 1FN est violée : un champ contient **deux** valeurs. La requête compte les lignes où le séparateur apparaît (`WHERE telephone LIKE '%;%'`) ; la correction dépend de l'usage : **deux**

colonnes si la seconde valeur est toujours de même nature (bureau, mobile), une **table de téléphones** si le nombre de numéros est variable — c’est alors une entité à part entière, reliée au client par une cardinalité 1,N.

13. Mini-projet de chapitre

« **La table de correspondance du référentiel** » (2 h). Produisez, pour le fil rouge : (1) la liste des **16** étiquettes de catégorie avec leur famille cible et le nombre de produits concernés ; (2) la ou les dépendances fonctionnelles qui justifient cette correspondance ; (3) la requête qui prouve que la correspondance donne **7** groupes et non **9** (attention au **TRIM**) ; (4) une phrase de règle : « quand une nouvelle étiquette apparaît, elle doit être ajoutée à la correspondance, sinon elle produit une famille fantôme ». Ce livrable est la matière de P4 (note de choix) du projet.

14. Résumé du chapitre

Notion	Ce qu’il faut retenir
1FN	une valeur par champ — la virgule non protégée du fichier plat l’a démontré
2FN	pas de dépendance à une partie de la clé composée (6 776 lignes de stock)
3FN	pas de dépendance entre attributs non clés (catégorie → famille)
Anomalies	redondance (× 11,8), mise à jour (20 lignes), insertion (416 clients), suppression (2 lignes)
Correspondance	16 étiquettes → 7 familles, avec TRIM (sinon 9)
Classement juste	Matériaux 3 492 592 638 FCFA en tête, pas Plomberie en tête
Coût mesuré	6 ms contre 3 ms : normaliser ne ralentit pas la lecture
Dénormalisation	décidée, écrite et régénérable — jamais subie

15. À retenir

À retenir. Normaliser, c’est écrire **de quoi chaque attribut dépend**. Les trois formes normales ne sont que trois questions posées à chaque table : tes valeurs sont-elles atomiques ? dépendent-elles de toute la clé ? dépendent-elles d’autre chose que la clé ?

- **La 1FN se contrôle par un compte de champs** : deux lignes à **13** champs dans un fichier à **12** colonnes suffisent à rendre un fichier illisible.
- **La 2FN ne concerne que les clés composées** : stock (**6 776** lignes) et ruptures.
- **La 3FN interdit les chaînes** : catégorie → famille doit être traitée une fois, dans une correspondance.
- **Les quatre anomalies se mesurent** : redondance **2 826 394** caractères, mise à jour **20** lignes, insertion **416** clients, suppression **2** lignes d’un client.
- **Une étiquette non corrigée classe faux** : **3 492 592 638** FCFA invisibles derrière **4** écritures de la même famille.

16. Évaluation formative

1. Énoncez la 1FN sans utiliser le mot « normaliser », et donnez un exemple de violation.
2. Qu’est-ce qu’une dépendance fonctionnelle ? Écrivez-en deux sur `dim_client`.

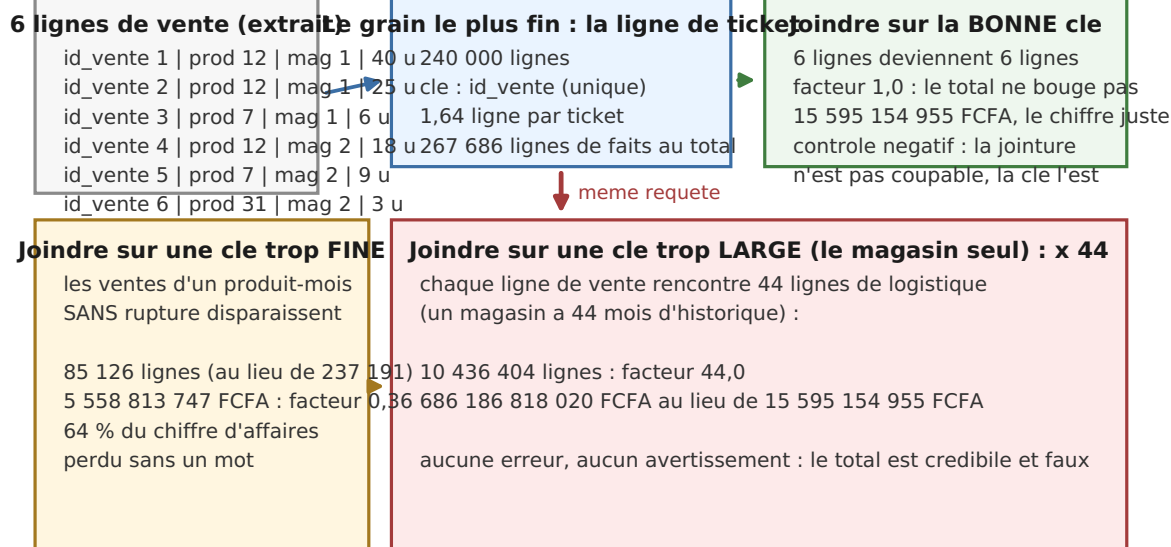
3. Pourquoi la 2FN ne concerne-t-elle que les tables à clé composée ?
4. Donnez un exemple de dépendance transitive sur le fil rouge, et la correction retenue.
5. Nommez les quatre anomalies et la mesure qui les illustre sur le socle.
6. Pourquoi le classement par étiquette brute est-il faux, et quel chiffre est caché ?
7. Combien de groupes obtient-on sans le TRIM de la correspondance, et pourquoi ?
8. Normaliser coûte-t-il du temps de lecture ? Que dit la mesure faite sur ce moteur ?
9. Citez trois cas où la dénormalisation est un bon choix, et la condition à remplir.
10. Une table à clé simple peut-elle violer la 3FN ? Justifiez par un exemple.

Corrigé : 1. Chaque champ contient une valeur indivisible ; une violation : une colonne `produits` contenant `Ciment`, `Tôle`. 2. `id_client` → `nom`, `id_client` → `ville`. 3. Parce qu'une clé simple ne comporte pas de partie : il n'y a rien dont un attribut pourrait dépendre « à moitié ». 4. `catégorie` → `famille` (**16** étiquettes pour **7** familles) ; correction : une table de correspondance appliquée dans la dimension, `TRIM` compris. 5. Redondance (**2 826 394** contre **239 599** caractères), mise à jour (**20** lignes pour le client le plus actif), insertion (**416** clients sans achat), suppression (le client 84, **2** lignes). 6. Parce que la même famille est écrite de **4** façons : `Materiaux` pèse **3 492 592 638** FCFA et n'apparaît pas dans un top trois par étiquette. 7. **9** groupes au lieu de **7**, à cause de deux étiquettes portant une espace finale. 8. Non : **3** ms pour la version normalisée contre **6** ms pour la table plate, sur ce moteur en colonnes. 9. Dimension de restitution, table d'export, cache de calcul — à condition de l'écrire et de pouvoir la régénérer. 10. Oui : une table de ventes à clé `id_vente` qui porterait `catégorie` et `famille` violerait la 3FN, la famille dépendant de la catégorie.

Module M13.C03 — Grain, additivité et tables de faits

Outils : DuckDB 1.5.5 (exécuté) ; pandas (contrôle croisé, exécuté). PostgreSQL et Power BI (cités, non exécutés — règle §1.5).

Le grain : ce que porte UNE ligne, et ce qu'on a le droit d'additionner



1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **dire ce que porte une ligne** d'une table de faits, en une phrase qui ne laisse aucune place au doute — c'est le **grain**, et il s'écrit au-dessus de la table, pas dans la tête de celui qui l'a créée ;
2. **prouver un grain** au lieu de le déclarer : compter les lignes, compter les valeurs distinctes de la clé, et vérifier que les deux nombres coïncident — sur les 7 tables de faits du fil rouge, dont les 7 grains vont de la ligne de ticket à l'objectif mensuel par magasin ;
3. **classer une mesure** dans l'un des trois régimes d'additivité : ce qui s'additionne dans tous les sens (additif), ce qui s'additionne dans l'espace mais jamais dans le temps (semi-additif), et ce qui ne s'additionne pas du tout (non additif) ;
4. **reconnaître les trois façons de fabriquer un faux total** — joindre sur une clé trop large, sur une clé trop fine, ou mélanger deux grains dans la même addition — et **mesurer** chacune sur le socle, jusqu'au facteur **44,0** et au chiffre d'affaires multiplié par **44** ;
5. **écrire le contrôle négatif** qui disculpe la jointure quand c'est la clé qui est en cause : même requête, mauvaise clé contre bonne clé, facteur **44,0** contre facteur **1,0**.

2. Pourquoi cette notion est importante

Le chapitre C02 a nettoyé le tableau de départ : plus de valeurs multiples dans un champ, plus de dépendance à une partie de clé. Le modèle est propre. Il n'est pas encore **juste**.

La différence entre propre et juste tient à une seule question : **quand vous lisez un total, savez-vous ce que chaque ligne comptait ?** Le socle du fil rouge contient **240 000** lignes de ventes, mais aussi **9 000** commandes, **9 000** encaissements, **6 776** lignes de stock mensuel, **2 428** ruptures, **264** bilans logistiques et **218** objectifs. Ces tables vivent ensemble, partagent les mêmes clés, répondent aux mêmes filtres, et **ne se comptent pas de la même façon**.

Toutes les erreurs de ce chapitre ont une propriété commune, et c'est ce qui les rend dangereuses : **elles s'exécutent sans erreur**. Aucune ne lève d'exception, aucune ne renvoie zéro ligne, aucune ne paraît suspecte. La requête fautive rend **10 436 404** lignes et **686 186 818 020** FCFA, quand le chiffre juste est **15 595 154 955** FCFA : deux nombres qui ont la même forme, le même séparateur de milliers, la même unité — et un seul des deux est le chiffre d'affaires de Sahel Distribution.

Dans les faits. Le défaut décrit ici n'est pas un cas d'école. La mesure la plus parlante du socle est le **facteur 44,0** : un magasin possède **44** mois d'historique logistique, et une jointure sur le seul `id_magasin` multiplie donc chaque vente par **44** — les **237 191** ventes nettes deviennent **10 436 404** lignes. Un analyste qui découvre cela en réunion ne découvre pas une erreur de méthode : il découvre que son tableau publié la semaine dernière portait un chiffre d'affaires **44** fois trop gros, et que personne ne l'a vu.

Ce chapitre installe donc la grammaire de tout le reste du module : C04 construira l'étoile et les dimensions conformes **sur** ces grains, C05 ajoutera les variantes quand l'étoile ne suffit plus, C06 traitera le temps du modèle, et C07 vérifiera l'ensemble par une grille de revue. Rien de ce qui suit ne fonctionne si le grain n'est pas écrit, prouvé et respecté.

3. Explication simple — l'eau, la photo et la température

Trois images suffisent pour tenir les trois régimes d'additivité.

L'eau. Versez un litre dans un seau, puis un autre : vous avez deux litres. Le volume s'additionne dans le temps comme dans l'espace, il se recompte à l'identique après avoir été découpé en seaux plus petits, et le total du mois est la somme des totaux des jours. C'est une mesure **additive**, et le chiffre d'affaires en fait partie : **44** totaux mensuels se re-somment exactement au total du modèle, avec un facteur de **1,0**.

La photo. Prenez une photo du seau chaque jour. Additionnez les **44** photos : vous n'obtenez pas **44** seaux, vous obtenez une pile de photos. Une photo de stock est un **instantané** : elle s'additionne dans l'espace (six magasins font bien un stock de réseau) mais jamais dans le temps. Le socle le prouve : les **44** mois d'instantanés se somment à **2 034 733** unités, soit **44** fois le stock d'un mois (**46 244** unités) — un nombre parfaitement crédible, et qui ne veut rien dire. La bonne lecture d'un instantané sur une période est le **dernier instantané** (**45 882** unités sur le dernier mois du socle), ou une moyenne, jamais une somme.

La température. Deux pièces à 20 degrés ne font pas une pièce à 40 degrés. Un taux, une pourcentage, une moyenne, une couverture en mois : ces mesures sont **non additives**, elles ne s'additionnent ni dans l'espace ni dans le temps. Le plan d'objectifs du fil rouge en donne la démonstration : ses **218** taux de marge se somment à **4 248,7 %**, quand leur moyenne vaut **19,5 %** — et ni l'un ni l'autre n'est la marge du réseau, qui se recalcule à partir de deux montants.

4. Vocabulaire essentiel

Terme	Définition	Sur le fil rouge
Grain	ce que porte une ligne d'une table de faits	une ligne de ticket, ou un produit par mois
Table de faits	table qui porte les mesures d'un processus, à un grain unique	fait_ventes, 240 000 lignes
Table de dimensions	table qui décrit les objets du fait (qui, quoi, où)	dim_client, dim_magasin, dim_produit,
Clé du grain	la ou les colonnes qui identifient une ligne de faits	id_vente ; id_produit + mois
Mesure additive	s'additionne dans l'espace et dans le temps	montant_ttc
Mesure additive semi-	s'additionne dans l'espace, pas dans le temps	stock_moyen_unites (instantané)
Mesure additive non	ne s'additionne ni dans l'espace ni dans le temps	marge_objectif_pct, couverture_mois
Explosion en lignes	une jointure qui multiplie les lignes d'une table	237 191 ventes deviennent 10 436 404
Filtre silencieux	une jointure qui fait disparaître des lignes sans le dire	CA à 0,36 du chiffre juste
Contrôle négatif	requête qui doit rendre le même total, pour disculper la jointure	ventes jointes au stock : facteur 1,0
Agréger	réduire des lignes à un total, une moyenne, un compte	GROUP BY sur dim_magasin
Fiche de grain	la ligne de documentation qui déclare le grain et l'additivité	livrable du mini-projet

Définition. Le **grain** d'une table de faits est la description, en langage courant, de ce que représente une ligne : « une ligne de ticket », « une commande », « un produit par magasin et par mois ». Le grain se déclare **avant** l'écriture de la table, s'écrit **dans le modèle**, et se prouve **après** le chargement.

Définition. Une mesure est **additive** quand la somme des parties égale le tout, quel que soit le découpage. Elle est **semi-additive** quand ce n'est vrai que sur certaines dimensions (le stock, additif entre magasins, ne l'est pas entre mois). Elle est **non additive** quand aucune somme n'a de sens — un taux, un ratio, une moyenne.

Définition. L'**explosion en lignes** est la multiplication du nombre de lignes d'une table par une jointure dont la clé est trop large : chaque ligne de gauche rencontre plusieurs lignes de droite, et le total de gauche est répété autant de fois.

Définition. Le **contrôle négatif** est une requête écrite pour **ne rien changer** : on volontairement la même jointure, avec la bonne clé, et l'on vérifie que le total est identique. Il ne prouve pas que la requête est utile ; il prouve que la jointure n'est pas la coupable.

Définition. La **fiche de grain** est le document d'une page qui déclare, pour chaque table de faits : son grain, sa clé, son unité de comptage, son régime d'additivité, les agrégations autorisées et interdites, et la requête qui vérifie l'unicité de la clé.

5. Cours approfondi

5.1 Ce que porte une ligne : le grain se dit en une phrase

Un modèle se lit table par table, et chaque table de faits commence par une phrase. Pas un schéma, pas un diagramme : une phrase, au présent, qui dit ce qu'une ligne représente.

- `fait_ventes` : « une ligne de ticket vendue ».
- `fait_commandes` : « une commande passée ».
- `fait_encaissements` : « une facture encaissée ».
- `fait_stock_mensuel` : « un produit, un mois, un instantané de stock ».
- `fait_ruptures` : « un produit, un magasin, un mois, un nombre de jours de rupture ».
- `fait_logistique` : « un magasin, un mois, un coût de livraison ».
- `fait_objectifs` : « un magasin, un mois, un objectif de chiffre d'affaires ».

Ces sept phrases ne sont pas interchangeables, et c'est tout l'objet du chapitre. Les 7 tables portent **267 686** lignes en tout, mais ce nombre ne veut rien dire en lui-même : il additionne des tickets et des instantanés, des commandes et des objectifs. Une table de faits n'est pas « un tableau de chiffres » ; c'est **un processus**, observé à un grain précis, avec ses clés et ses mesures.

5.2 Sept grains, un facteur 1 101

Le grain le plus fin du modèle est la ligne de ticket : **240 000** lignes, **1,64** ligne par ticket pour **146 161** tickets. Le plus agrégé est l'objectif mensuel par magasin : **218** lignes. Entre les deux, le rapport est de **1 101**.

Ce facteur n'est pas un détail arithmétique, c'est la raison pour laquelle **deux tables de faits ne s'additionnent jamais**. Imaginez qu'un rapport présente « le total du modèle » comme la somme des sept tables : il ajouterait des dinars et des litres. Le résultat serait un nombre à onze chiffres, affiché en gras dans un tableau de bord, et il ne correspondrait à rien.

La bonne lecture de ces sept tables est **par question**, pas par somme :

Question	Table	Grain	Mesure à lire
Combien avons-nous vendu ?	<code>fait_ventes</code>	ligne de ticket	somme du montant
Combien de commandes sont en retard ?	<code>fait_commandes</code>	commande	compte, écart de dates
Combien avons-nous encaissé ?	<code>fait_encaissements</code>	facture	somme encaissée
Où est le stock ?	<code>fait_stock_mensuel</code>	produit et mois	dernier instantané, moyenne
Combien de jours sans vente ?	<code>fait_ruptures</code>	produit, magasin, mois	somme des jours
Combien coûte la livraison ?	<code>fait_logistique</code>	magasin et mois	somme des coûts
Sommes-nous dans l'objectif ?	<code>fait_objectifs</code>	magasin et mois	montant contre montant de ventes

5.3 Le grain se prouve, il ne se déclare pas

Déclarer un grain est une intention ; le prouver est une mesure. La preuve tient en deux comptages : **le nombre de lignes** et **le nombre de valeurs distinctes de la clé**. S'ils coïncident, la clé est unique, donc le grain est celui du chargement — aucune jointure ne l'a multiplié en chemin.

```
-- La preuve du grain de fait_ruptures (grain : produit x magasin x mois)
SELECT COUNT(*) AS lignes,
       COUNT(*) AS cles
FROM (SELECT DISTINCT id_produit, id_magasin, mois FROM fait_ruptures);
```

Le socle passe ce contrôle sur les 7 tables de faits : 7 grains sur 7 sont prouvés. Mais la démonstration la plus utile n'est pas celle qui réussit, c'est celle qui **échoue pour une bonne raison**. Regardez `fait_objectifs` :

Clé testée	Valeurs distinctes	Lignes	Conclusion
<code>id_magasin + mois</code> (numéro de mois)	60	218	« la table a des doublons » — faux
<code>id_magasin + annee_mois</code> (« 2023-01 »)	218	218	le grain est prouvé

La colonne `mois` de cette table porte le **numéro** du mois (1 à 12), pas le mois daté : cinq magasins fois douze numéros de mois donnent **60** valeurs distinctes, et l'on conclurait à tort à des doublons. Le grain n'était pas faux ; la clé de contrôle l'était.

Attention. Un contrôle d'unicité qui échoue ne prouve pas qu'il y a des doublons : il prouve que la clé testée n'est pas celle du grain. Avant de conclure, écrivez la clé en toutes lettres et vérifiez qu'elle correspond bien au grain déclaré. Sur le socle, `id_magasin + mois` rend **60** valeurs pour **218** lignes ; la bonne clé, `id_magasin + annee_mois`, en rend **218**.

5.4 Trois régimes d'additivité, et la règle qui va avec

Le régime d'additivité d'une mesure détermine ce qu'on a le droit d'écrire dans une requête. Le socle a été construit pour que les trois régimes soient mesurables.

Régime	Mesure du socle	Ce qu'on peut faire	Ce qui est interdit
Additif	<code>montant_ttc,</code> <code>jours_rupture</code>	<code>quantite</code> , SUM par magasin, par mois, par client	rien, la somme est vraie partout
Semi-additif	<code>stock_moyen_unites,</code> <code>couverture_mois</code>	<code>colis</code> , SUM entre magasins et produits	SUM sur plusieurs mois
Non additif	<code>marge_objectif_pct,</code> <code>taux_remise</code>	moyenne pondérée, ou recalcul sur les totaux	SUM, AVG des lignes

La démonstration, chiffrée sur le fil rouge :

- **additif** — les **44** totaux mensuels des ventes se re-somment au total du modèle, facteur **1,0** ;
- **semi-additif** — les **44** instantanés de stock se somment à **2 034 733** unités au lieu de **46 244**, soit **44** fois trop ;
- **non additif** — les **218** taux de marge du plan d'objectifs se somment à **4 248,7 %**, leur moyenne vaut **19,5 %**, et la marge du réseau n'est ni l'un ni l'autre.

Conseil professionnel. Quand vous ne savez pas si une mesure s'additionne, faites le test sur deux lignes avant de le faire sur deux millions : additionnez deux mois, et demandez-vous si le résultat a un sens métier. Deux mois de chiffre d'affaires, oui ; deux mois de stock, non ; deux mois de taux de marge, jamais. Ce test prend trente secondes et évite de publier **2 034 733** unités de stock au lieu de **46 244**.

5.5 Le théorème du grain : ne jamais mélanger les grains

Voici la règle qui tient tout le module : **une table de faits porte un grain unique, et deux tables de grains différents ne s'additionnent ni ne se joignent sans précaution.**

Ce n'est pas une préférence de style. Le socle en donne la mesure, et elle est spectaculaire. Sur le schéma fautif livré dans le dossier du module :

Ce que fait la requête	Résultat	Écart
ventes + commandes	22 305 610 960 FCFA	+ 43,0 %
ventes + commandes + encaissements	29 474 916 530 FCFA	+ 89,0 %

Le chiffre d'affaires du modèle est de **15 595 154 955** FCFA. La première addition, celle de deux tables, est déjà fautive de **43** pour cent ; la seconde de **89** pour cent. Et les deux requêtes s'exécutent : rien n'empêche d'additionner deux colonnes qui portent le même nom — `mon_tant_ttc` — dans deux tables qui ne mesurent pas la même chose. C'est précisément le piège : le nom est identique, le grain ne l'est pas.

À retenir. Deux tables de faits ne s'additionnent jamais directement. Si un rapport a besoin de chiffres venus de deux processus, il les présente **côte à côte** (une colonne par processus), ou il les agrège **séparément** à un grain commun avant de les joindre. L'addition de deux grains différents est la faute la plus coûteuse du module : + **43,0 %** sur deux tables, + **89,0 %** sur trois.

5.6 Les trois façons de fabriquer un faux total

Il n'y a pas mille manières de produire un total faux. La mesure sur le socle en isole trois, et les trois rendent un nombre plausible.

1. Joindre sur une clé trop large. C'est l'explosion en lignes. Les ventes jointes à la logistique sur le seul `id_magasin` (au lieu du couple magasin et mois) donnent **10 436 404** lignes et **686 186 818 020** FCFA, soit **44** fois le chiffre juste. Un magasin a **44** mois d'historique : la clé trop large fait rencontrer à chaque vente les **44** lignes du magasin.

2. Joindre sur une clé trop fine, qui filtre sans le dire. Les ventes jointes aux ruptures sur le produit et le mois ne gardent que les ventes des produits **en rupture** : **85 126** lignes au lieu de **237 191**, et **5 558 813 747** FCFA, soit **0,36** du chiffre juste. Ici, aucune ligne n'est multipliée : ce sont **64** pour cent du chiffre d'affaires qui **disparaissent**, et la requête, elle, se porte très bien.

3. Mélanger deux grains dans la même addition. C'est le §5.5 : + **43,0 %** à deux tables, + **89,0 %** à trois.

Attention. Une jointure qui filtre est plus difficile à voir qu'une jointure qui multiplie. Un total trop gros se remarque (il dépasse ce qu'on attend) ; un total trop petit a l'air d'une bonne nouvelle. Le socle perd **64** pour cent de son chiffre d'affaires avec un `JOIN` parfaitement banal entre deux tables auxquelles on fait confiance — les ruptures sont une table **incomplète** : un produit-mois sans rupture n'y figure pas.

5.7 Le contrôle négatif : la jointure n'est pas toujours la coupable

La réaction naturelle, devant un total faux, est d'accuser la jointure. C'est souvent injuste : la plupart du temps, la jointure fait exactement ce qu'on lui demande, et c'est la **clé** qui ne dit pas ce qu'on croit.

Le contrôle négatif tranche la question. On écrit la même requête, sur la même table, avec la clé du grain cette fois :

Jointure	Clé	Lignes	Chiffre d'affaires	Facteur
Ventes et logistique	<code>id_magasin</code> seul	10 436 404	686 186 818 020 FCFA	44,0

Jointure	Clé	Lignes	Chiffre d'affaires	Facteur
Ventes et stock mensuel	id_produit + mois	237 191	15 595 154 955 FCFA	1,0

Même intention, même syntaxe, deux résultats que tout sépare. La deuxième requête ne sert à rien métier — elle ne calcule aucun indicateur utile — et c’est justement pour cela qu’elle est précieuse : elle **isole la cause**. La jointure multiplie quand la clé est trop large, elle ne multiplie pas quand la clé est celle du grain. Le défaut n’était donc pas « la jointure » ; il était la clé choisie.

Ce n’est pas la seule forme de filtre silencieux. Le même effet se produit quand la jointure est pourtant **au bon grain des deux côtés** : joindre deux agrégats mensuels par une jointure interne fait disparaître le magasin qui n’a aucune vente. Sur le fil rouge, le dépôt central porte **44** mois de coûts logistiques, **28 893 543** FCFA, et zéro vente : le total des coûts tombe à **143 816 487** FCFA au lieu de **172 710 030** FCFA. La clé était juste, la jointure aussi, et le filtre s’est produit quand même — parce que les deux tables ne couvrent pas le même ensemble de magasins.

Conseil professionnel. Écrivez toujours le contrôle négatif **en même temps** que la requête métier, pas après l’incident. Deux requêtes voisines dans le même fichier de livrable : celle qui répond à la question, et celle qui prouve que le grain a été respecté. Sur un modèle livré, c’est la seconde qu’un relecteur regarde en premier.

5.8 La fiche de grain : une page qui évite les trois fautes

Une table de faits sans fiche de grain est une table qu’on interrogera mal. La fiche tient en sept lignes, une par table, et chacune répond à six questions :

Colonne	Question à laquelle elle répond
Table	de quelle table parle-t-on ?
Grain	que porte une ligne, en une phrase ?
Clé	par quoi identifie-t-on une ligne ?
Unité de comptage	une ligne compte un quoi ?
Additivité	quelles mesures s’additionnent, et dans quel sens ?
Agrégations interdites	que ne faut-il jamais faire ?
Contrôle d’unicité	quelle requête prouve le grain ?

Pour le fil rouge, les deux premières lignes de cette fiche se lisent ainsi :

- fait_ventes — une ligne de ticket ; clé id_vente ; compte des tickets ; montant additif, quantité additive, taux de remise non additif ; interdit : rien de particulier, c’est le grain le plus fin ; contrôle : `COUNT(*) = COUNT(DISTINCT id_vente)`.
- fait_stock_mensuel — un produit par mois ; clé id_produit et mois ; compte des couples ; mesures semi-additives ; interdit : sommer plusieurs mois ; contrôle : `COUNT(*) = COUNT(DISTINCT id_produit, mois)`.

Cette page est le livrable du mini-projet ci-dessous, et la matière première du livrable P1 du projet de module.

6. Exemple concret — six lignes qui expliquent tout le chapitre

La planche de ce chapitre part d'un extrait minuscule : 6 lignes de ventes, deux produits, deux magasins.

id_vente	id_produit	id_magasin	quantite
1	12	1	40
2	12	1	25
3	7	1	6
4	12	2	18
5	7	2	9
6	31	2	3

Le grain de cet extrait est la ligne de ticket : six lignes, six `id_vente` distincts. Le compte des lignes et le compte des clés coïncident : le grain est prouvé.

Première lecture, correcte. Combien de lignes par magasin ? Magasin 1 : trois lignes (1, 2, 3). Magasin 2 : trois lignes (4, 5, 6). Total : six. Le grain du résultat est « une ligne par magasin », et la mesure est un **compte**.

Deuxième lecture, la bonne clé. Jointure avec un tableau de stock mensuel de deux lignes (produit 12 et produit 7, tous deux en magasin 1 et 2) sur le couple produit et mois : les six lignes restent six, la quantité totale est inchangée. C'est le contrôle négatif : facteur **1,0**.

Troisième lecture, la clé trop large. Jointure avec un historique de coûts logistiques qui porte 44 mois par magasin, sur le seul `id_magasin` : chaque ligne de vente rencontre les 44 lignes du magasin, et six lignes deviennent **264**. Le total de quantité est multiplié par 44 — c'est exactement ce qui se produit sur le socle, où **237 191** ventes deviennent **10 436 404** lignes.

Quatrième lecture, la clé trop fine. Jointure avec un tableau de ruptures qui ne contient qu'un produit par mois : les ventes de l'autre produit disparaissent. Six lignes deviennent trois, et la moitié du chiffre d'affaires s'évapore sans que personne ne se plaigne. Sur le socle, la même opération laisse **64** pour cent du chiffre d'affaires sur le carreau.

Ces quatre lectures, faites sur six lignes, sont les **trois** fautes du §5.6 plus le contrôle qui les détecte. Vous n'avez pas besoin de deux millions de lignes pour apprendre à les voir ; vous avez besoin de six lignes et de la bonne question : **quelle est la clé du grain ?**

7. Démonstration pas à pas — quatre gestes, dans cet ordre

Étape 1 — écrire le grain en une phrase, avant toute chose.

```
fait_ventes : une ligne de ticket vendue
```

Cette phrase fixe la clé (`id_vente`), l'unité de comptage (le ticket), et la question à laquelle la table peut répondre. Elle s'écrit dans le fichier SQL qui crée la table, en commentaire.

Étape 2 — prouver le grain sur les données chargées.

```
SELECT COUNT(*) AS lignes FROM fait_ventes;
SELECT COUNT(*) AS cles FROM (SELECT DISTINCT id_vente FROM fait_ventes);
```

Si les deux nombres diffèrent, la table ne porte pas le grain annoncé : la cause est une jointure trop large dans la requête de chargement, ou un doublon dans la source. On corrige **avant** d'écrire la suite.

Étape 3 — écrire la règle d'additivité de chaque mesure.

Pour `fait_ventes` : `montant_ttc`, `montant_ht`, `quantite`, `poids_kg` sont additifs ; `taux_remise` et `prix_unitaire_ht` sont non additifs (ils se moyennent, ou se recalculent). Pour `fait_stock_mensuel` : tout est semi-additif — la précision s'écrit une fois, dans la fiche.

Étape 4 — écrire le contrôle négatif, et le garder.

```
-- Contrôle négatif : la jointure au grain du stock ne doit rien changer au total
SELECT COUNT(*) AS lignes, ROUND(SUM(montant_ttc)) AS ca
FROM fait_ventes;
SELECT COUNT(*) AS lignes, ROUND(SUM(v.montant_ttc)) AS ca
FROM fait_ventes v
JOIN fait_stock_mensuel s
  ON s.id_produit = v.id_produit AND s.mois = strftime(v.date_vente, '%Y-%m');
```

Attendu : le deuxième total est **égal** au premier (facteur **1,0**) — la jointure est disculpée, et l'on peut passer à la métrique suivante. S'il diffère, la clé n'est pas celle du grain.

8. Erreurs fréquentes

Erreur	Ce qu'on observe	La correction
Joindre sans écrire la clé du grain	10 436 404 lignes au lieu de 237 191	nommer le grain, puis la clé, avant d'écrire la jointure
Croire qu'un JOIN qui filtre est un JOIN qui garde	64 pour cent du chiffre d'affaires envoyés	LEFT JOIN, ou jointure à un grain commun
Joindre deux agrégats internes	143 816 487 FCFA de coûts au lieu de 172 710 030	jointure externe, et COALESCE sur la mesure absente
Sommer des instantanés	2 034 733 unités de stock au lieu de 46 244	dernier instantané, ou moyenne
Sommer des taux	4 248,7 % de marge	recalculer sur deux montants, ou moyenner avec un poids
Additionner deux tables de faits	+ 43,0 % à deux, + 89,0 % à trois	présenter côte à côte, agréger séparément
Valider un grain avec la mauvaise clé	60 valeurs distinctes pour 218 lignes	écrire la clé du grain en toutes lettres
Ne tester que le cas qui marche	aucune erreur, aucun avertissement	écrire aussi le contrôle négatif, et celui qui doit échouer

9. Bonnes pratiques professionnelles

1. **Le grain s'écrit dans la table, pas dans un document à part.** Un commentaire en tête de la requête de création, une ligne dans la fiche de grain, et la phrase est la même dans les deux.
2. **Une table de faits, un grain.** Si deux processus se ressemblent, ils font deux tables dont on assume qu'elles ne se joignent pas sans agrégation préalable.
3. **Le contrôle d'unicité s'exécute à chaque chargement,** pas une fois pour toutes : il est le garde-fou du grain, et il coûte deux requêtes de comptage.

4. **Chaque mesure reçoit sa règle d'additivité**, y compris celles qui ne s'additionnent pas. Écrire « non additive : moyenne pondérée » est plus utile que ne rien écrire.
5. **Le contrôle négatif fait partie du livrable**. Une requête qui ne sert à rien métier est une requête qui prouve quelque chose.
6. **On nomme les colonnes de résultat avec leur grain** : `nb_lignes_ticket`, `stock_fin_de_mois`, `ca_ttc_mois` — le nom du résultat rappelle le grain, et une erreur de lecture devient visible.

10. Exercice guidé

Sujet. Le service logistique publie chaque mois un coût de livraison par magasin : **264** lignes, un magasin et un mois par ligne. Un rapport demande « le coût logistique par franc de chiffre d'affaires ». La requête publiée joint les ventes à la logistique sur `id_magasin` et rend un coût multiplié par **44**. Trouvez la faute, écrivez la requête juste, et prouvez que la nouvelle version respecte le grain des deux tables.

Étape 1 — les grains, en deux phrases. `fait_ventes` : une ligne de ticket (**240 000** lignes, clé `id_vente`). `fait_logistique` : un magasin par mois (**264** lignes, clé `id_magasin` et `mois`).

Étape 2 — la faute. La requête de départ joint sur `id_magasin` seul : la clé de la première table est plus fine que la clé de la seconde, donc chaque vente rencontre les **44** lignes de son magasin. Le coût logistique est compté **44** fois ; le chiffre d'affaires est, lui, multiplié par **44** aussi, ce qui « sauve » le ratio — mais les deux totaux publiés à côté sont faux, et le ratio lui-même perd son sens dès qu'un magasin a un historique incomplet.

Étape 3 — la requête juste. On agrège chaque table à **son propre grain**, puis on joint les deux résultats sur le grain commun (le mois) — par une jointure **externe**, car le dépôt central n'a aucune vente :

```
WITH ca AS (
  SELECT strftime(date_vente, '%Y-%m') AS mois, id_magasin, ROUND(SUM(montant_ttc)) AS ca
  FROM fait_ventes WHERE est_retour = 0 GROUP BY 1, 2
), logi AS (
  SELECT mois, id_magasin, ROUND(SUM(cout_total)) AS cout FROM fait_logistique GROUP BY 1, 2
)
SELECT logi.mois, logi.id_magasin, logi.cout, COALESCE(ca.ca, 0) AS ca,
  ROUND(logi.cout / NULLIF(COALESCE(ca.ca, 0), 0), 4) AS cout_par_franc
FROM logi LEFT JOIN ca ON ca.mois = logi.mois AND ca.id_magasin = logi.id_magasin
ORDER BY logi.mois, logi.id_magasin;
```

Étape 4 — la preuve. Deux contrôles : la somme du chiffre d'affaires de la requête égale le total du modèle (**15 595 154 955** FCFA), et la somme des coûts logistiques égale le total de la table logistique, **172 710 030** FCFA — ni multiplié, ni filtré. Avec une jointure **interne**, ce second contrôle échoue : le total des coûts tombe à **143 816 487** FCFA, et les **28 893 543** FCFA du dépôt central ont disparu — six magasins portent des coûts, cinq seulement portent des ventes, et le sixième a précisément **44** mois de dépenses pour zéro vente.

Barème (15 points) : les deux grains énoncés (3) · la faute expliquée et chiffrée (4) · la requête juste, jointure externe comprise (5) · les deux contrôles exécutés (3).

11. Exercices autonomes

Exercice 3.1. Prouvez le grain de `fait_ruptures` (**2 428** lignes) et celui de `fait_objectifs` (**218** lignes) : écrivez pour chacune la requête d'unicité, puis la même requête avec la **mauvaise** clé

(`id_magasin` et `mois` pour les objectifs), et expliquez en trois lignes pourquoi le second résultat fait croire à des doublons.

Exercice 3.2. Classez ces six mesures du socle en additif, semi-additif ou non additif : `montant_ttc` (ventes), `stock_moyen_unites` (stock), `marge_objectif_pct` (objectifs), `jours_rupture` (ruptures), `colis` (logistique), `couverture_mois` (stock). Pour chacune, écrivez l'agrégation autorisée sur les mois et celle qui est interdite, et donnez la requête qui démontre l'interdiction sur une des mesures semi-additives.

Exercice 3.3. Reproduisez l'explosion en lignes sur un extrait : prenez les **6** lignes de l'extrait du §6, joignez-les à un tableau de coûts de **44** lignes par magasin sur le seul `id_magasin`, et vérifiez que vous obtenez **264** lignes. Puis écrivez la jointure corrigée (clé magasin et mois) et vérifiez que vous retrouvez **6** lignes.

Exercice 3.4. Le filtre silencieux : écrivez la requête qui joint les ventes aux ruptures sur le produit et le mois, mesurez le nombre de lignes et le chiffre d'affaires obtenus, et comparez au total du modèle (**15 595 154 955** FCFA). Expliquez en cinq lignes pourquoi **64** pour cent du chiffre d'affaires disparaît, puis proposez **deux** corrections différentes — l'une qui garde toutes les ventes, l'autre qui assume de ne garder que les produits en rupture, en le déclarant.

Exercice 3.5. Écrivez un contrôle négatif pour la requête « chiffre d'affaires par famille de produits » : proposez une jointure qui doit rendre le total **inchangé** (facteur **1,0**), exécutez-la et commentez le résultat en trois lignes.

12. Correction détaillée

Exercice 3.1. `fait_ruptures` — les deux comptages coïncident : **2 428** lignes, **2 428** clés distinctes sur (`id_produit`, `id_magasin`, `mois`) ; le grain est prouvé. `fait_objectifs` — avec la bonne clé (`id_magasin`, `annee_mois`), **218** lignes et **218** clés : prouvé. Avec la mauvaise clé (`id_magasin`, `mois`), **60** valeurs distinctes pour **218** lignes : cinq magasins fois douze numéros de mois font **60** combinaisons possibles, et la table en occupe **218** parce qu'elle couvre **44** mois. Le contrôle ne révèle donc pas des doublons : il révèle une clé mal choisie.

Exercice 3.2. Additifs : `montant_ttc`, `jours_rupture`, `colis`. Semi-additifs : `stock_moyen_unites`, `couverture_mois`. Non additif : `marge_objectif_pct`. Démonstration du semi-additif :

```
SELECT SUM(stock_moyen_unites) AS somme_des_44_mois
FROM fait_stock_mensuel;    -- 2 034 733 unites
```

à comparer à la lecture correcte, le dernier mois :

```
SELECT SUM(stock_moyen_unites) AS stock_du_dernier_mois
FROM fait_stock_mensuel
WHERE mois = (SELECT MAX(mois) FROM fait_stock_mensuel);    -- 45 882 unites
```

La première requête rend **2 034 733** unités, soit **44** fois la seconde : le nombre est arithmétiquement juste et métier faussement utile.

Exercice 3.3. Sur l'extrait : six lignes de ventes, deux magasins, et un tableau de coûts de **44** lignes par magasin. Jointure sur `id_magasin` seul : 6 fois 44 égale **264** lignes. Jointure sur (`id_magasin`, `mois`) : les six lignes trouvent chacune leur ligne de coût du mois, et restent **6**. Le premier résultat n'est pas « plus riche » : il compte 44 fois la même vente.

Exercice 3.4. La requête :

```
SELECT COUNT(*) AS lignes, ROUND(SUM(v.montant_ttc)) AS ca
FROM fait_ventes v
JOIN fait_ruptures r
  ON r.id_produit = v.id_produit AND r.mois = strftime(v.date_vente, '%Y-%m')
WHERE v.est_retour = 0;
```

Elle rend **85 126** lignes et **5 558 813 747** FCFA, contre **237 191** lignes et **15 595 154 955** FCFA : facteur **0,36**, soit **64** pour cent du chiffre d'affaires perdu. La raison : `fait_ruptures` ne contient que les couples produit, magasin et mois **en rupture** ; une vente dont le produit n'était pas en rupture n'a pas de ligne à rejoindre, et disparaît. Deux corrections : (a) garder toutes les ventes avec un `LEFT JOIN` et traiter les valeurs manquantes ; (b) assumer le périmètre « ventes de produits en rupture » et l'écrire dans le titre du rapport, avec le chiffre d'affaires **du périmètre**, jamais présenté comme le chiffre d'affaires total.

Exercice 3.5. Le contrôle négatif :

```
SELECT ROUND(SUM(f.montant_ttc)) AS ca_avec_jointure
FROM fait_ventes f
JOIN dim_produit p ON p.id_produit = f.id_produit
WHERE f.est_retour = 0;
```

Résultat : **15 595 154 955** FCFA, identique au total sans jointure — facteur **1,0**. La dimension produit est au grain du produit (**154** lignes, clé unique), donc chaque vente rencontre exactement une ligne, et la jointure ne change ni le nombre de lignes ni le total. Ce contrôle est le modèle de tous les autres : il transforme « je crois que la jointure est bonne » en « je l'ai vérifié ».

13. Mini-projet de chapitre

« **La fiche de grain du modèle** » (2 h). Produisez, pour les **7** tables de faits du fil rouge, la fiche de grain complète : une ligne par table, six colonnes comme au §5.8 — table, grain écrit en une phrase, clé du grain, unité de comptage, régime d'additivité des mesures, agrégations interdites — et la requête d'unicité qui prouve le grain. Ajoutez une seconde feuille avec les **trois** contrôles négatifs : celui qui ne change rien (facteur **1,0**), celui qui doit multiplier (facteur **44,0**), celui qui doit filtrer (facteur **0,36**) — avec, pour chacun, la phrase qui explique ce qu'il prouve. Ce livrable alimente directement **P1** (MCD et MLD) et **P3** (revue du modèle) du projet de module.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Grain	ce que porte une ligne, écrit en une phrase, prouvé par l'unicité de sa clé
Sept grains	de la ligne de ticket (240 000) à l'objectif mensuel (218), facteur 1 101
Preuve du grain	<code>COUNT(*)</code> égal au compte des valeurs distinctes de la clé du grain
Bonne clé	sur <code>fait_objectifs</code> , (<code>id_magasin</code> , <code>annee_mois</code>) rend 218 clés ; (<code>id_magasin</code> , <code>mois</code>) en rend 60
Additif	<code>montant_ttc</code> : 44 totaux mensuels se re-somment au total, facteur 1,0
Semi-additif	le stock : 2 034 733 unités sur 44 mois, contre 46 244 pour un mois
Non additif	le taux : 4 248,7 % en somme, 19,5 % en moyenne, et ni l'un ni l'autre
Clé trop large	10 436 404 lignes et 686 186 818 020 FCFA : facteur 44,0
Clé trop fine	85 126 lignes et 5 558 813 747 FCFA : facteur 0,36 , 64 pour cent perdus

Notion	Ce qu'il faut retenir
Mélange de grains	ventes plus commandes : 22 305 610 960 FCFA (+ 43,0 %)
Contrôle négatif	la bonne clé laisse le total intact : facteur 1,0
Filtre par jointure	deux agrégats au même grain, et 28 893 543 FCFA de coûts perdus
Fiche de grain	sept lignes de documentation qui évitent les trois fautes

15. À retenir

1. **Le grain est une phrase, pas une intuition.** « Une ligne de ticket » se dit, s'écrit, et se vérifie par deux comptages.
2. **Le grain se prouve avec la bonne clé.** Un contrôle qui échoue signale parfois une clé mal choisie : **60** valeurs distinctes pour **218** lignes ne sont pas des doublons, c'est un numéro de mois pris pour un mois daté.
3. **Le filtre silencieux prend deux formes** : la clé trop fine, et la jointure interne entre deux ensembles qui ne couvrent pas les mêmes objets — **28 893 543** FCFA du dépôt central disparaissent ainsi d'un total de coûts.
4. **Trois régimes, trois règles.** Le montant s'additionne (**facteur 1,0**), le stock s'additionne seulement entre magasins (**2 034 733** contre **46 244**), le taux ne s'additionne jamais (**4 248,7 %** contre **19,5 %**).
5. **Les trois faux totaux s'exécutent sans erreur** : clé trop large (**44,0**), clé trop fine (**0,36**), grains mélangés (+ **43,0 %**).
6. **Le contrôle négatif fait partie du travail** : il disculpe la jointure et désigne la clé.

À retenir. Un total juste n'est pas un total qui s'exécute : c'est un total dont on peut dire, pour chaque ligne lue, ce qu'elle comptait, et pour chaque colonne sommée, dans quel sens elle s'additionne.

À retenir. Quand un total paraît étrange, la question n'est pas « la requête est-elle juste ? » mais « **quelle est la clé du grain ?** » : dans les trois fautes mesurées du chapitre, la même jointure a rendu **44,0** fois trop, **0,36** fois trop peu, et exactement le bon chiffre — selon la clé employée.

16. Évaluation formative

1. Écrivez le grain de `fait_logistique` en une phrase, sans regarder le chapitre.
2. Comment prouve-t-on qu'un grain est respecté ? Donnez les deux comptages et leur lecture.
3. `fait_objectifs` : pourquoi la clé (`id_magasin`, `mois`) rend-elle **60** valeurs pour **218** lignes ?
4. Citez une mesure additive, une semi-additive et une non additive du socle, avec leur règle.
5. Les **44** instantanés de stock se somment à **2 034 733** unités. Quelle est la lecture correcte sur la période, et pourquoi ?
6. Pourquoi **4 248,7 %** n'est-il pas la marge du réseau, et que faut-il calculer à la place ?
7. Qu'est-ce qu'une explosion en lignes ? Donnez le facteur mesuré sur le socle et sa cause exacte.
8. Pourquoi un `JOIN` qui filtre est-il plus difficile à détecter qu'un `JOIN` qui multiplie ?
9. À quoi sert un contrôle négatif, et que prouve précisément le facteur **1,0** ?
10. Deux tables de faits portent toutes deux une colonne `montant_ttc`. Peut-on les additionner ? Justifiez par la mesure du socle.

Corrigé : 1. « un magasin, un mois, un coût de livraison » (clé `id_magasin` et `mois`, **264** lignes). 2. `COUNT(*)` et le compte des valeurs distinctes de la clé du grain : égaux, la clé est unique, donc aucune jointure n'a multiplié la table. 3. Parce que `mois` y porte le numéro du mois : 5 magasins fois 12 numéros donnent **60** combinaisons, alors que la table couvre **44** mois datés, d'où **218** lignes avec la clé (`id_magasin`, `annee_mois`). 4. Additive : `montant_ttc` (somme partout) ; semi-additive : `stock_moyen_unites` (somme entre magasins, jamais entre mois) ; non additive : `marge_objectif_pct` (moyenne pondérée ou recalcul). 5. Le dernier instantané (**45 882** unités) ou une moyenne, jamais une somme : sommer des photos de stock ne fait pas un stock. 6. Parce qu'un taux ne s'additionne pas : la marge du réseau se recalcule à partir du montant de marge et du chiffre d'affaires, pas en moyennant des taux. 7. C'est la multiplication des lignes d'une table par une clé trop large : **44,0** sur le socle, parce qu'un magasin possède **44** mois d'histoire logistique. 8. Parce qu'un total trop gros attire l'œil, alors qu'un total trop petit ressemble à une bonne nouvelle : **64** pour cent du chiffre d'affaires peuvent disparaître sans que rien ne paraisse anormal. 9. À disculper (ou accuser) la jointure : avec la clé du grain, le total est **identique** (facteur **1,0**), donc la jointure n'est pas la cause du faux total. 10. Non : même nom, grains différents. Sur le fil rouge, additionner ventes et commandes donne **22 305 610 960** FCFA, soit **+ 43,0 %** du chiffre d'affaires réel ; deux tables de faits se présentent côte à côte ou s'agrègent séparément avant d'être jointes.

Module M13.C04 — Le schéma en étoile et les changements lents

Outils : DuckDB 1.5.5 (exécuté) ; pandas (contrôle croisé, exécuté). PostgreSQL et Power BI (cités, non exécutés — règle §1.5).

Les 4 types de changement lent, sur un tarif qui monte de 24 % en 4 ans

Le fait réel : une vente par année, au tarif de son année

vente 2023 tarif 5 415 FCFA	vente 2024 tarif 5 769 FCFA	vente 2025 tarif 6 225 FCFA	vente 2026 tarif 6 816 FCFA
--------------------------------	--------------------------------	--------------------------------	--------------------------------

Type 0, ne change jamais date de creation, nom : une seule ligne, la valeur d'origine, pour toujours

Type 1, on écrase (correction) 140 mouvements : conditions, ville
l'ancienne valeur n'existe plus (67 + 73)

Type 2, on versionne (histoire) 980 mouvements : segment (658), ville (322)
24 892 versions de clients, 900 clients concernés

Type 3, colonne de la valeur d'avant une valeur précédente par attribut :
4 clients ont changé 3 fois de ville

Lire le passe coute 14 ms au lieu de 7 ms, et change le prix moyen des ventes : 9 045 FCFA historisé contre 8 222 FCFA courant.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **reconnaître une étoile** et dire ce qui la distingue d'un schéma en flocon : une table de faits au centre, des dimensions autour, et **une seule jointure** pour atteindre un attribut ;
2. **repérer une dimension conforme** et la mesurer : `dim_magasin` sert 5 des 7 tables de faits du fil rouge, `dim_client` et `dim_produit` en servent 3 chacune — c'est ce qui rend deux rapports comparables ;
3. **distinguer les quatre types de changement lent** (SCD 0, 1, 2, 3) et **choisir** le bon selon ce que l'historique doit permettre : sur le socle, **980** mouvements sont des versions (type 2) et **140** des corrections (type 1) ;
4. **écrire la requête « au moment du fait »** — une jointure par intervalle, et non par égalité de clé — puis **vérifier** qu'elle rend exactement **240 000** lignes pour **240 000** ventes, sans doublon ni perte ;
5. **mesurer ce que l'historique change** : le même chiffre d'affaires par segment vaut **7 838 286 840** FCFA lu sur la dimension courante et **7 735 369 260** FCFA lu sur la dimension historisée — **102 917 580** FCFA d'écart sur un seul segment.

2. Pourquoi cette notion est importante

Le chapitre C03 a fixé le grain : chaque table de faits dit ce qu'elle compte. Il reste à savoir **autour de quoi** elle le compte.

Une table de faits ne contient que des clés et des mesures : `id_client`, `id_produit`, `id_magasin`, un montant, une quantité. Toutes les questions qu'on pose à un rapport — par segment, par région, par famille de produits — passent par une **dimension**. C'est la dimension qui porte le sens métier, et c'est elle qui pose le problème que ce chapitre résout : **les attributs changent**.

Le fil rouge en donne la mesure exacte. Le référentiel clients contient **1 120** mouvements : **980** changements d'état (un client change de ville, ou passe du segment Argent au segment Or) et **140** corrections de saisie. Si la dimension écrase tout — un client, une seule ligne, toujours à jour —, alors les ventes de 2023 sont attribuées à la ville **d'aujourd'hui**, et le chiffre d'affaires par segment devient faux de **102 917 580** FCFA. Personne ne le verra : le rapport s'affiche, les segments se somment au total, et le total est juste. Seule la **répartition** est faussée.

Dans les faits. L'écart mesuré sur le fil rouge est de **102 917 580** FCFA, et il ne porte que sur la répartition : le total des trois segments est identique dans les deux lectures (**15 595 154 955** FCFA). C'est le pire genre de défaut — celui qu'aucun contrôle de total ne détecte. Trois segments, dont deux se déplacent de **34** à **61** millions de francs, et un tableau de bord qui attribue à la mauvaise région l'effort commercial de l'année.

Ce chapitre construit donc la partie du modèle qui **gère le temps qui passe** : les dimensions historisées. C05 complètera l'étoile quand elle ne suffit plus (flocon, table de pont, faits multiples), C06 branchera le calendrier, C07 vérifiera et documentera l'ensemble.

3. Explication simple — l'étoile, la fiche et la photographie

L'étoile. Imaginez un entrepôt de quincaillerie. Au centre, un comptoir où l'on enregistre chaque mouvement : telle date, tel client, tel produit, tel magasin, telle quantité, tel montant. Autour du comptoir, quatre classeurs : le classeur des clients, celui des produits, celui des magasins, celui des vendeurs. Le registre central ne recopie pas les adresses et les désignations — il écrit un **numéro de fiche**. Pour savoir quelle famille de produits a été vendue, on ouvre le classeur des produits à la bonne fiche. Une jointure, et l'on sait. C'est une **étoile**.

La fiche. Chaque fiche porte plusieurs attributs : pour un client, sa ville, son segment, ses conditions de paiement. Ces attributs ne sont pas de même nature. Certains ne changeront **jamais** (la date de création du compte) : c'est le **type 0**. D'autres sont des **erreurs de saisie** qu'on corrige (un « 30 j » noté avec une espace finale) : on écrase l'ancienne valeur, c'est le **type 1**. D'autres encore **changent pour de vrai** (le client déménage, il devient un gros client) : là, on garde l'ancienne fiche **et** la nouvelle, en datant chacune — c'est le **type 2**, celui qui porte l'histoire.

La photographie. Le type 2 revient à photographier la fiche chaque fois qu'elle change, et à inscrire au dos de chaque photo la période pendant laquelle elle était vraie. Le client **2 030** a trois photos : Koudougou du 2020-03-06 au 2023-06-08, Kaya du 2023-06-09 au 2025-08-22, Ouagadougou depuis le 2025-08-23. Quand vous vendez quelque chose à ce client, vous ne lui donnez pas « sa » ville : vous lui donnez la ville de **la photo en vigueur ce jour-là**.

Le **type 3**, enfin, est une variante paresseuse : au lieu de garder toutes les photos, on garde seulement la valeur actuelle et la précédente, dans deux colonnes. Sur le fil rouge, **4** clients ont changé **3** fois de ville : le type 3 perdrait la première, et il aurait fallu une colonne par changement possible.

4. Vocabulaire essentiel

Terme	Définition	Sur le fil rouge
Schéma en étoile	une table de faits au centre, des dimensions autour, une jointure par attribut	7 faits, 5 dimensions
Dimension conforme	dimension partagée par plusieurs tables de faits	<code>dim_magasin</code> , 5 faits
Attribut dimension	de colonne descriptive d'un objet (ville, segment, famille)	<code>segment</code> , <code>ville</code> , <code>famille</code>
Hiérarchie	niveaux d'agrégation d'une dimension	<code>produit</code> , <code>sous-catégorie</code> (12), <code>famille</code> (7)
SCD	Slowly Changing Dimension : changement lent de dimension	les 4 types ci-dessous
SCD type 0	attribut qui ne change jamais	<code>date_creation</code> , <code>nom</code>
SCD type 1	on écrase l'ancienne valeur (correction)	140 mouvements (ville, conditions)
SCD type 2	on ajoute une version , datée et conservée	980 mouvements, 24 892 versions
SCD type 3	on garde la valeur précédente dans une colonne	insuffisant ici : 4 clients à 3 villes
Version courante	la ligne ouverte, sans date de fin	<code>est_courante</code> = 1
Clé de version	identifiant de la version d'un objet	<code>cle_version</code> : 12-4
Jointure intervalle	par jointure sur date BETWEEN <code>date_debut</code> AND <code>date_fin</code>	240 000 lignes, aucune perte

Définition. Un **schéma en étoile** est un modèle où une table de faits, au grain unique, pointe par des clés étrangères vers des tables de dimensions qui décrivent le qui, le quoi et le où. Pour atteindre un attribut, une seule jointure suffit.

Définition. Une **dimension conforme** est une dimension **partagée** par plusieurs tables de faits : les mêmes valeurs, la même clé, la même signification. C'est la condition pour que deux rapports construits sur deux processus différents parlent des mêmes objets — et donc pour qu'ils soient comparables.

Définition. Un **changement lent de dimension** (Slowly Changing Dimension, SCD) est la manière dont un modèle traite la modification d'un attribut de dimension : ne rien faire (type 0), écraser (type 1), versionner (type 2), ou garder la valeur précédente (type 3).

Définition. Une **dimension historisée** (ou dimension de type 2) est une dimension dont chaque ligne porte une **période de validité** (`date_debut`, `date_fin`) et un indicateur de version courante. Chaque objet peut donc y apparaître plusieurs fois, à des périodes différentes.

Définition. La **jointure au moment du fait** est la jointure entre une table de faits et une dimension historisée qui rend l'attribut **tel qu'il était** à la date du fait : elle porte sur un intervalle, pas sur une égalité de clé.

5. Cours approfondi

5.1 L'étoile, et pourquoi pas autre chose

Un schéma en étoile se reconnaît à trois signes : une table de faits au centre, des dimensions tout autour, et **une seule jointure** pour aller d'une mesure à un attribut. Sa force n'est pas es-

thétique, elle est mécanique : moins de jointures, moins d'occasions de se tromper, et des jointures qui restent compréhensibles par un lecteur qui découvre le modèle.

Le fil rouge s'y range exactement : **7** tables de faits, **5** dimensions, **8** clés étrangères déclarées. Les faits portent les mesures et les clés ; les dimensions portent les attributs. Aucune table intermédiaire, aucun attribut recopié dans les faits.

5.2 Les dimensions conformes, mesurées

Une dimension conforme est une dimension **partagée**. Le socle permet de la mesurer, fait par fait :

Dimension	Nombre de faits servis	Lesquels
dim_magasin	5	ventes, commandes, ruptures, logistique, objectifs
dim_client	3	ventes, commandes, encaissements
dim_produit	3	ventes, stock mensuel, ruptures
dim_vendeur	1	ventes
dim_date	0	aucun fait ne pointe dessus par clé

Ce tableau se lit comme une carte des comparaisons possibles. Parce que `dim_magasin` est conforme, on peut mettre côte à côte le chiffre d'affaires, les ruptures, le coût logistique et l'objectif d'un même magasin — quatre processus différents, un seul objet, donc un seul découpage. Parce que `dim_vendeur` n'est servie que par les ventes, aucun rapport ne pourra confronter un vendeur à ses coûts logistiques : la dimension n'a pas été conçue pour cela.

Attention. `dim_date` existe, elle est juste (**1 339** jours, avec trimestres, semaines et jours fériés) et **aucun fait ne pointe dessus par clé** : les ventes portent une date inline. Le trou est déclaré, pas caché ; c'est le chapitre C06 qui le fermera, en branchant le calendrier sur les faits. En attendant, un rapport qui veut agréger par trimestre doit dériver le trimestre de la date — ce que le calendrier aurait évité.

5.3 Attributs et hiérarchies : ce qu'une dimension porte

Une dimension porte des **attributs** (des colonnes descriptives) et souvent une **hiérarchie** (des niveaux qui s'emboîtent). Sur le fil rouge :

- `dim_produit` — **154** produits, **12** sous-catégories, **7** familles : trois niveaux, deux pas de jointure pour aller de la mesure au niveau le plus agrégé ;
- `dim_magasin` — **6** magasins, **4** villes, **4** régions ;
- `dim_client` — **23 913** lignes (les **23 912** clients plus la ligne « client non identifié »), avec type, segment, ville, région, conditions de paiement.

Deux règles de modélisation se déduisent de cette description. D'abord, **un attribut appartient à une dimension, jamais aux faits** : recopier la ville du client dans les ventes reviendrait à la figer à la date de la vente, ce que le §5.5 va exploiter. Ensuite, **une hiérarchie se déclare** : personne ne devine que `famille` est au-dessus de `sous_categorie` — cela s'écrit dans la documentation de la dimension.

5.4 Les quatre types de changement lent

Voici la partie du chapitre qui décide de ce qu'un rapport pourra dire du passé. Les quatre types, sur le socle, sont mesurables.

Type 0 — ne jamais changer. L'attribut est réputé figé : `date_creation` du compte client, `nom` du magasin, `date_embauche` du vendeur. Aucune ligne de mouvement ne les touche, et c'est heureux : si l'un d'eux doit changer, c'est en général le signe qu'il n'était pas le bon attribut.

Type 1 — écraser. La valeur change, l'historique n'est pas conservé. Sur le socle, **140** mouvements : **73** sur la ville et **67** sur les conditions de paiement. Ce sont des **corrections** (« 30 j » au lieu de « 30 j »), pas des changements d'état. Le type 1 a un effet brutal et qu'il faut assumer : le passé est **réécrit**. Une vente de 2023 attribuée à une ville corrigée en 2026 sera publiée avec la ville corrigée, comme si elle y avait toujours été.

Type 2 — versionner. C'est le type qui porte l'histoire : **980** mouvements sur le socle (**658** sur le segment, **322** sur la ville), pour **24 892** versions de clients concernant **900** clients (dont certains ont jusqu'à **3** versions). La ligne ancienne reste, datée et fermée ; la ligne nouvelle s'ouvre. Le prix à payer est triple : la dimension grossit (de **23 912** lignes à **24 893**), chaque objet peut apparaître plusieurs fois — donc **aucune jointure naïve ne doit servir la dimension historisée sans sa clause de date** —, et il faut décider ce qui se passe pour un fait **antérieur à la première version**.

Type 3 — colonne de la valeur précédente. Deux colonnes (`ville`, `ville_precedente`) au lieu d'une historisation complète. C'est économique en lignes et catastrophique en information : le socle compte **4** clients qui ont changé de ville plus de deux fois, avec jusqu'à **3** villes distinctes. Le type 3 en perd une, et il faudrait une colonne par changement possible. Sa seule justification est la comparaison « avant / après » à date fixe, jamais la vraie histoire.

À retenir. Le choix du type de SCD n'est pas technique, il est **métier** : il dit ce qu'un rapport a le droit de demander au passé. Sur le fil rouge, la ville et le segment sont en type 2 parce qu'un directeur régional doit pouvoir lire le chiffre d'affaires **de sa région au moment de la vente** ; les conditions de paiement sont en type 1 parce qu'une correction de saisie n'est pas un changement d'état.

5.5 La requête « au moment du fait »

Une dimension historisée ne se joint pas sur une égalité de clé. Elle se joint sur un **intervalle** :

```
SELECT h.segment, ROUND(SUM(f.montant_ttc)) AS ca
FROM fait_ventes f
JOIN dim_client_scd h
  ON h.id_client = f.id_client
 AND f.date_vente BETWEEN h.date_debut AND COALESCE(h.date_fin, DATE '2100-01-01')
WHERE f.est_retour = 0
GROUP BY 1
ORDER BY 2 DESC;
```

Trois détails décident de la justesse de cette requête.

Le premier est `BETWEEN` : c'est lui qui choisit **la bonne photo**. Le deuxième est le `COALESCE` sur la date de fin : la version courante n'a pas de date de fin (la valeur est nulle), et un `BETWEEN` sur une valeur nulle ne renvoie rien — le client le plus important, celui dont la ville est actuelle, serait purement et simplement exclu du rapport. Le troisième est la **clé de version** : si deux versions se chevauchaient d'un jour, la jointure doublerait les lignes et le chiffre d'affaires avec elles.

Le contrôle est donc double, et il se mesure : la jointure « au moment du fait » rend **240 000** lignes pour **240 000** ventes — aucun doublon, aucune perte.

5.6 Les deux trous, et pourquoi il faut les boucher exprès

La même requête, appliquée à un socle moins soigné, aurait deux trous. Le socle les a traités, et le détail vaut la leçon.

Le client non identifié. Le client 0 apparaît dans **43 161** ventes et n'existe pas dans le référentiel — un défaut classique de caisse, où l'on encaisse sans identifier l'acheteur. Le modèle lui donne une ligne : « client non identifié », **43 161** lignes de ticket, **2 852 612 447** FCFA. Sans cette ligne, une jointure **interne** ferait disparaître **43 161** ventes du rapport, et **2 852 612 447** FCFA avec elles.

Le fait antérieur à la première version. Le référentiel porte une date de **création** du compte ; la première version historisée s'ouvre à cette date. Mais **28 784** ventes ont été enregistrées **avant** cette date de création, pour **6 685** clients — un décalage normal : la fiche a été saisie après la première vente. La première version commence donc à la **plus ancienne** des deux dates, `LEAST(date_creation, premiere_vente)`. Sans ce rattrapage, ces **28 784** ventes ne trouvent aucune version à rejoindre, et disparaissent.

Attention. Une dimension historisée trouée ne lève **aucune erreur**. Une jointure interne perd simplement les lignes qui ne trouvent rien : **43 161** ventes sans client identifié et **28 784** ventes antérieures à la création du compte représentent à elles deux **71 945** lignes de ticket qui s'évaporaient d'un rapport, sans un mot d'avertissement. Le contrôle est le même qu'au chapitre C03 : compter les lignes avant et après.

5.7 Ce que la lecture historisée coûte, et ce qu'elle rapporte

Faut-il historiser ? La question se tranche par la mesure, et le socle donne les deux termes.

Ce qu'elle coûte. La jointure par intervalle est plus chère qu'une égalité de clé : **14** ms contre **7** ms sur le fil rouge, médiane de sept exécutions, pour le même rapport par segment, soit deux fois plus de temps pour la même réponse. Le surcoût est modeste ici et il grandit avec la taille des dimensions — mais il se paie une fois, à la conception, pas à chaque lecture.

Ce qu'elle rapporte. Le même rapport, sur les mêmes ventes, donne deux réponses différentes : **7 735 369 260** FCFA pour le segment Standard et **3 503 366 365** FCFA pour Argent, lus sur la dimension historisée, contre **7 838 286 840** et **3 442 257 458** lus sur la dimension courante. L'écart atteint **102 917 580** FCFA sur un seul segment, et le total, lui, ne bouge pas : les deux lectures somment à **15 595 154 955** FCFA. La question n'est donc pas « quel total est juste ? » — les deux le sont — mais « **quelle répartition voulons-nous publier ?** ». Celle du temps des faits, évidemment : un commercial de Koudougou ne peut pas être crédité de ses ventes dans la ville où son client habite aujourd'hui.

Conseil professionnel. Avant d'historiser un attribut, posez la question qui tranche : « un rapport doit-il pouvoir dire où c'était / combien c'était à une date passée ? » Si la réponse est non, le type 1 suffit ; si la réponse est oui, le type 2 est obligatoire, avec son coût — **24 893** lignes au lieu de **23 912** sur le seul référentiel clients. Historiser par principe alourdit le modèle sans rien apporter ; ne pas historiser un attribut qu'on interrogera dans le temps rend les rapports faux en silence.

Le cas des tarifs. La même logique s'applique aux prix. La grille tarifaire est historisée : **616** versions pour **154** produits, **4** versions par produit, une nouvelle version chaque 1er janvier, **8 783** FCFA en moyenne en 2023 et **10 873** FCFA en 2026 (+ **24** %). Lire le prix **du jour** sur des ventes de 2023 gonfle ou écrase la marge. Mesure sur le fil rouge : le prix moyen des ventes vaut **9 045** FCFA lu sur la dimension historisée et **8 222** FCFA lu sur la dimension courante — **9,1** % d'écart. Un prix moyen de 9 045 FCFA n'est pas « un peu » différent de 8 222 : le second compare des ventes de quatre années au tarif d'aujourd'hui.

5.8 La clé de version, et la ligne qu'on n'oublie pas

Une dimension historisée a besoin d'une clé qui identifie **une version**, pas un objet : `id_client` seul est ambigu dès qu'un client a deux versions. Le socle emploie une **clé de version** construite (`cle_version`, de la forme 12-4 : le produit 12, sa quatrième version). Elle sert à deux choses : joindre sans ambiguïté depuis une table qui référence une version précise, et **compter** les versions d'un objet sans se tromper.

La ligne qu'on n'oublie pas est celle de l'**inconnu** : « client non identifié » dans `dim_client`, porte ouverte aux **43 161** ventes orphelines. Une dimension qui refuse l'inconnu est une dimension qui perd des faits — et un modèle qui perd des faits est un modèle qui publie des totaux faux sans le dire.

6. Exemple concret — deux fiches qui bougent

Un produit, quatre versions de prix. Le produit **12** (ciment) a quatre versions :

Version	Période	Prix de vente HT
1	2023	5 415 FCFA
2	2024	5 769 FCFA
3	2025	6 225 FCFA
4	depuis 2026	6 816 FCFA

C'est + **25,9 %** en quatre ans, soit quatre lignes dans `dim_produit_scd` là où une dimension courante n'en garderait qu'une. La conséquence est directe : une vente de mars 2023 dont on lit le tarif sur la dimension courante est valorisée au prix de 2026. Le chiffre d'affaires, lui, ne bouge pas — il est dans les faits —, mais **toute analyse de prix ou de marge devient fausse**, parce que la comparaison oppose des montants réels à un tarif anachronique.

Un client, trois villes. Le client **2 030** apparaît trois fois :

Version	Période	Ville
1	du 2020-03-06 au 2023-06-08	Koudougou
2	du 2023-06-09 au 2025-08-22	Kaya
3	depuis le 2025-08-23	Ouagadougou

Trois ans de ventes à Koudougou, deux ans à Kaya, le reste à Ouagadougou. Lu sur la dimension courante, ce client est un client d'Ouagadougou, point final : l'effort commercial de 2021 à Kaya et sa contribution à la région du Centre-Ouest disparaissent des rapports régionaux. Lu sur la dimension historisée, chaque vente va à la ville de son époque. C'est exactement l'écart de **102 917 580** FCFA mesuré au §5.7, sauf qu'ici on peut nommer le client et dater chaque changement.

7. Démonstration pas à pas — quatre gestes

Étape 1 — déclarer les attributs et leur type de changement.

Pour chaque attribut d'une dimension, écrire son type : 0 (figé), 1 (écrasé), 2 (historisé). C'est une décision de conception, prise avec le métier, et elle se documente — sinon chaque nouveau rapport invente sa propre règle.

Étape 2 — construire les colonnes de période.

Le socle construit les versions en SQL, à partir de deux sources : le référentiel clients et le fichier des mouvements. Les bornes d'abord — chaque changement de type 2 ouvre une version, et la première borne est la plus ancienne des deux dates (création du compte, première vente) :

```
WITH prem AS (
  SELECT CAST(id_client AS INTEGER) AS id_client,
         MIN(TRY_CAST(date_vente AS DATE)) AS premiere_vente
  FROM ventes GROUP BY 1),
base AS (
  SELECT CAST(c.id_client AS INTEGER) AS id_client,
         COALESCE(TRY_CAST(c.date_creation AS DATE), DATE '2023-01-01') AS date_creation,
         p.premiere_vente
  FROM clients c LEFT JOIN prem p ON p.id_client = CAST(c.id_client AS INTEGER)),
mvt AS (
  SELECT CAST(id_client AS INTEGER) AS id_client, TRY_CAST(date_effet AS DATE) AS date_effet
  FROM read_csv_auto('03_exercices/dossier_M13/mouvements_clients.csv', header = true)
  WHERE CAST(type_scd AS INTEGER) = 2),
bornes AS (
  SELECT id_client, date_effet AS debut FROM mvt
  UNION
  SELECT id_client, LEAST(date_creation, COALESCE(premiere_vente, date_creation)) AS debut
  FROM base),
versions AS (
  SELECT id_client, debut,
         row_number() OVER (PARTITION BY id_client ORDER BY debut) AS version,
         lead(debut) OVER (PARTITION BY id_client ORDER BY debut) AS fin
  FROM bornes)
SELECT COUNT(*) AS versions, COUNT(DISTINCT id_client) AS clients FROM versions;
```

Résultat : **24 892** versions pour **23 912** clients — **900** clients ont donc une histoire. Le lead ouvre la version suivante, et la fermeture de la précédente s'en déduit par `fin - INTERVAL 1 DAY`. Deux colonnes disent tout : `date_debut` (obligatoire) et `date_fin` (NULL pour la version en cours) ; la clé de version (12-4) identifie une version, pas un objet.

Étape 3 — ouvrir et fermer les versions.

Quand un attribut de type 2 change, on **ferme** la version courante la veille du changement et l'on **ouvre** une nouvelle version au jour du changement. La fermeture est la partie qu'on oublie : une dimension dont deux versions restent ouvertes au même instant double les lignes de toute jointure qui la lit.

Étape 4 — écrire la jointure et son contrôle.

```
-- la jointure au moment du fait
SELECT COUNT(*) AS lignes_jointes
FROM fait_ventes f
JOIN dim_client_scd h
  ON h.id_client = f.id_client
  AND f.date_vente BETWEEN h.date_debut AND COALESCE(h.date_fin, DATE '2100-01-01');
```

Attendu : **240 000** lignes pour **240 000** ventes. Plus petit, la dimension est trouée (clients inconnus, faits antérieurs) ; plus grand, deux versions se chevauchent. Ce contrôle s'exécute à chaque chargement, comme celui du grain au chapitre C03.

8. Erreurs fréquentes

Erreur	Ce qu'on observe	La correction
Joindre une dimension historisée sans clause de date	le rapport double les lignes	<code>BETWEEN date_debut AND COALESCE(date_fin, ...)</code>
Oublier le <code>COALESCE</code> sur la date de fin	les clients actuels disparaissent du rapport	traiter le NULL comme « encore vrai »

Erreur	Ce qu'on observe	La correction
Laisser deux versions ouvertes	240 000 lignes deviennent davantage	fermer la version précédente la veille
Écraser un attribut qui change pour de vrai	102 917 580 FCFA d'écart de répartition	type 2 sur les attributs d'analyse
Historiser un attribut qui ne change jamais	dimension inutilement plus grosse	type 0, et on l'écrit
Croire que le total contrôle la répartition	le total est juste, les segments non	comparer chaque segment, pas le total
Refuser la ligne « inconnu »	43 161 ventes perdues sans erreur	une ligne porteuse de l'inconnu

9. Bonnes pratiques professionnelles

1. **Le type de changement se décide avec le métier**, pas avec le développeur : « voulez-vous retrouver les ventes d'un client dans la ville où il était, ou dans celle où il est ? » suffit à trancher.
2. **Une dimension historisée porte toujours trois colonnes de service** : date de début, date de fin, indicateur de version courante.
3. **La table de faits ne se joint jamais deux fois à une dimension historisée** sans agrégation intermédiaire : deux jointures par intervalle sur la même table multiplient les lignes.
4. **Le contrôle « lignes après jointure = lignes avant »** s'exécute à chaque chargement, et son résultat se publie.
5. **La ligne « inconnu » fait partie du modèle** : elle est documentée, elle porte un nom lisible, et elle ne se supprime pas pour faire propre.
6. **On date les corrections de type 1** dans le journal de chargement, même si la donnée n'en garde pas trace : le prochain analyste doit pouvoir savoir que l'histoire a été réécrite.

10. Exercice guidé

Sujet. Le rapport régional publie « chiffre d'affaires par region du client ». Le service commercial conteste : d'après lui, les ventes de l'année 2023 ne sont pas dans les bonnes régions. Vous avez trente minutes pour dire qui a raison, et le prouver.

Étape 1 — écrire les deux requêtes, mot pour mot. La première lit la dimension courante (`dim_client`), la seconde la dimension historisée (`dim_client_scd`) avec la clause d'intervalle. Les deux comptent les mêmes ventes, sur la même période, avec le même filtre de retours.

Étape 2 — comparer les répartitions, pas les totaux. Les deux requêtes donnent le même total (**15 595 154 955** FCFA). La comparaison utile porte sur **chaque** région et **chaque** segment : Standard **7 838 286 840** contre **7 735 369 260**, Argent **3 442 257 458** contre **3 503 366 365**, Or **1 461 998 210** contre **1 503 806 883**.

Étape 3 — nommer la cause. Les écarts ne viennent pas d'une erreur de calcul, mais du **sens** de la lecture : la dimension courante applique à toutes les ventes la ville **d'aujourd'hui**, alors que **900** clients ont changé d'au moins un attribut, et que le référentiel porte **980** changements d'état.

Étape 4 — trancher et l'écrire. Le service commercial a raison : publier dans la ville actuelle revient à créditer un commercial d'une vente qu'il n'a pas faite. La règle se documente : « tout

rapport régional lit la dimension historisée ; la dimension courante ne sert qu'aux listes de contacts. »

Barème (15 points) : les deux requêtes écrites (4) · la comparaison des trois segments, chiffrée (4) · la cause nommée (3) · la règle écrite et justifiée (4).

11. Exercices autonomes

Exercice 4.1. Pour chacun de ces attributs du fil rouge, choisissez un type de changement (0, 1, 2) et justifiez en une ligne : `date_creation` d'un client, `ville` d'un client, `conditions_paiement`, `famille` d'un produit, `prix_vente_ht` d'un produit, `segment` d'un client. Donnez, pour chaque choix de type 2, la question métier qui l'exige.

Exercice 4.2. Écrivez la jointure « au moment du fait » entre les ventes et la dimension clients historisée, puis son contrôle : le nombre de lignes après jointure doit évaluer le nombre de ventes. Exécutez, et donnez les deux nombres obtenus. Expliquez ensuite ce qui se passerait si vous retiriez le `COALESCE` sur `date_fin`.

Exercice 4.3. Le client **2 030** a trois versions. Écrivez la requête qui donne son chiffre d'affaires **par ville et par année**, et vérifiez que la somme par ville correspond bien aux périodes de ses versions. Combien de villes trouve-t-on, et lesquelles ?

Exercice 4.4. Le type 3 : écrivez, sur `dim_client_scd`, la requête qui compte les clients ayant changé de ville **plus de deux fois**. Donnez le nombre obtenu, puis la liste des colonnes qu'il aurait fallu ajouter pour représenter ces clients en type 3 — et concluez en trois lignes.

Exercice 4.5. Un collègue propose de « simplifier » le modèle en supprimant `dim_client_scd` et en recopiant `region` dans les ventes au moment de l'achat. Écrivez ce que cette solution donne de juste (le rapport régional devient exact), et ce qu'elle coûte : citez deux requêtes que le modèle ne permettra plus d'écrire, et un chiffre du socle qui montre le prix de la redondance.

12. Correction détaillée

Exercice 4.1. Type 0 : `date_creation` (une date de création ne change pas) et `famille` d'un produit (elle se corrige si le classement est faux, mais elle n'est pas une histoire). Type 1 : `conditions_paiement` — c'est une donnée contractuelle qui se corrige, et l'historique du contrat n'est pas géré ici. Type 2 : `ville` et `segment` du client (le directeur régional doit lire la ville **de l'époque de la vente**), `prix_vente_ht` (la marge se calcule avec le tarif de la période). Chaque choix de type 2 répond à une question datée : « où ce client habitait-il quand il a acheté ? », « quel prix s'appliquait ce jour-là ? ».

Exercice 4.2. La requête et son contrôle :

```
SELECT COUNT(*) FROM fait_ventes f;
SELECT COUNT(*) FROM fait_ventes f
JOIN dim_client_scd h
  ON h.id_client = f.id_client
 AND f.date_vente BETWEEN h.date_debut AND COALESCE(h.date_fin, DATE '2100-01-01');
```

240 000 puis **240 000** : la jointure est complète. Sans le `COALESCE`, la condition `date_vente BETWEEN date_debut AND NULL` est inconnue pour toutes les versions courantes : les ventes des clients dont la fiche est à jour — donc l'immense majorité — disparaîtraient du rapport.

Exercice 4.3. Le chiffre d'affaires du client **2 030** par ville et par année :

```
SELECT h.ville, MIN(f.date_vente) AS premiere, MAX(f.date_vente) AS derniere,
       COUNT(*) AS ventes, ROUND(SUM(f.montant_ttc)) AS ca
FROM fait_ventes f
JOIN dim_client_scd h
  ON h.id_client = f.id_client
 AND f.date_vente BETWEEN h.date_debut AND COALESCE(h.date_fin, DATE '2100-01-01')
WHERE f.id_client = 2030 AND f.est_retour = 0
GROUP BY 1
ORDER BY 1;
```

Trois villes apparaissent — **Koudougou, Kaya, Ouagadougou** — et les périodes de ventes tombent dans les intervalles de validité des versions. Si une seule ville ressortait, c'est que la jointure aurait lu la dimension courante.

Exercice 4.4. La requête de comptage :

```
SELECT COUNT(*) FROM (
  SELECT id_client FROM dim_client_scd
  GROUP BY id_client HAVING COUNT(DISTINCT ville) > 2
);
```

Résultat : **4** clients (jusqu'à **3** villes distinctes). En type 3, il faudrait les colonnes `ville`, `ville_precedente`, `ville_avant_precedente` — et l'on devrait recommencer à chaque nouveau changement. Le type 3 ne représente pas une histoire ; il représente une comparaison.

Exercice 4.5. La solution « recopier la région dans les ventes » rend le rapport régional exact — c'est son seul mérite. Elle coûte d'abord la **redondance** : la région est recopiée sur les **240 000** lignes de ventes au lieu de **6** lignes de magasins, et la moindre correction devient une mise à jour de masse. Elle coûte ensuite deux requêtes que le modèle ne permettra plus d'écrire : (a) « quelle est la répartition actuelle de mes clients par région ? » — les ventes portent une région figée, la dimension porte la région vivante ; (b) « quels clients ont changé de région cette année ? » — l'information n'existe plus nulle part. Le chiffre qui mesure le prix de la redondance est celui du chapitre C02 : **2 826 394** caractères de noms de clients recopiés dans la table plate, contre **239 599** dans la dimension — un facteur **11,8**.

13. Mini-projet de chapitre

« **La note de choix de la dimension client** » (2 h 30). Produisez, pour la dimension clients du fil rouge : (1) la liste de ses attributs avec, pour chacun, le type de changement retenu (0, 1, 2) et la justification métier ; (2) le tableau des **980** mouvements de type 2 par attribut (**658** segment, **322** ville) et des **140** mouvements de type 1 (**73** ville, **67** conditions) ; (3) la requête qui prouve que la jointure historisée ne perd ni ne duplique aucune ligne (**240 000** pour **240 000**) ; (4) la requête qui chiffre l'écart de répartition entre lecture courante et lecture historisée, segment par segment ; (5) une page de règle : « quel rapport lit quelle dimension ». Ce livrable alimente directement **P2** (SQL et contrôles) et **P4** (note de choix de 3 pages) du projet de module.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Étoile	un fait au centre, des dimensions autour, une jointure par attribut
Dimensions conformes	<code>dim_magasin</code> sert 5 faits, <code>dim_client</code> et <code>dim_produit</code> 3 ; <code>dim_date</code> 0
Hiérarchie	produit, 12 sous-catégories, 7 familles
SCD type 0	<code>date_creation</code> , <code>nom</code> : jamais modifiés

Notion	Ce qu'il faut retenir
SCD type 1	140 corrections, l'ancienne valeur est écrasée
SCD type 2	980 mouvements, 24 892 versions, 900 clients
SCD type 3	insuffisant : 4 clients ont 3 villes
Jointure au moment du fait	BETWEEN et COALESCE : 240 000 lignes pour 240 000 ventes
Les deux trous	client inconnu (43 161 ventes) et faits antérieurs à la création (28 784 ventes)
Coût de l'historique	14 ms contre 7 ms, médiane de sept exécutions
Gain de l'historique	répartition par segment juste à 102 917 580 FCFA près
Tarif historisé	616 versions, 4 par produit, 9,1 % d'écart de prix moyen

15. À retenir

1. **Une dimension est le seul endroit où vit le sens métier.** Le fait porte des clés et des montants ; la ville, le segment et la famille sont dans les dimensions.
2. **Une dimension conforme rend deux rapports comparables** : `dim_magasin` sert **5** faits, et c'est pour cela que l'on peut confronter ventes, ruptures, logistique et objectifs d'un magasin.
3. **Les quatre types de SCD sont une décision métier**, pas un réglage technique : figer, écraser, versionner, ou comparer avant et après.
4. **La jointure au moment du fait impose deux écritures précises** : BETWEEN sur l'intervalle et COALESCE sur la date de fin ; et son contrôle est un comptage — **240 000** lignes pour **240 000** ventes.
5. **Une dimension historisée qui perd un fait ne prévient personne** : **43 161** ventes sans client identifié et **28 784** ventes antérieures à la création du compte, soit **71 945** lignes de ticket qui disparaîtraient d'un rapport en silence.

À retenir. Le total ne contrôle pas la répartition : les deux lectures du chiffre d'affaires par segment somment au même **15 595 154 955** FCFA, et pourtant l'une des deux attribue mal **102 917 580** FCFA. Un rapport juste au total peut être faux dans chaque case.

À retenir. Historiser coûte deux choses et en rapporte une. Cela coûte des lignes (**24 893** au lieu de **23 912**), une clause de date dans **chaque** jointure, et une lecture environ deux fois plus longue sur le fil rouge (**14** ms contre **7** ms, médiane de sept exécutions). Cela rapporte la seule chose qu'un modèle ne peut pas fabriquer après coup : **l'état du monde au moment du fait**.

16. Évaluation formative

1. Qu'est-ce qui distingue un schéma en étoile d'un schéma en flocon ? Donnez le critère pratique.
2. Citez une dimension conforme du fil rouge et le nombre de faits qu'elle sert.
3. Que signifie « dimension historisée » ? Citez ses trois colonnes de service.
4. Décrivez les quatre types de SCD et donnez, pour chacun, un attribut du socle (ou la raison de son absence).
5. Pourquoi le type 3 ne convient-il pas au socle clients ? Donnez le chiffre.
6. Écrivez, en cinq lignes, la jointure « au moment du fait », COALESCE compris.
7. Que se passe-t-il si l'on omet le COALESCE sur `date_fin` ?

8. La jointure historisée rend **240 000** lignes pour **240 000** ventes : que prouve ce contrôle, et que ne prouve-t-il pas ?
9. Pourquoi le total des ventes par segment est-il identique dans les deux lectures, alors que les segments diffèrent de **102 917 580** FCFA ?
10. Le prix moyen des ventes vaut **9 045** FCFA en lecture historisée et **8 222** FCFA en lecture courante : expliquez l'écart en une phrase, et dites laquelle des deux valeurs doit figurer dans un rapport de marge.

Corrigé : 1. Le nombre de jointures nécessaires pour atteindre un attribut : une seule dans l'étoile, plusieurs dans le flocon (la hiérarchie est éclatée en tables). 2. `dim_magasin`, servie par **5** des **7** tables de faits. 3. Une dimension dont chaque objet peut avoir plusieurs lignes datées : `date_debut`, `date_fin`, `est_courante`. 4. Type 0 : `date_creation` ; type 1 : les **140** corrections (ville, conditions) ; type 2 : les **980** changements d'état (segment, ville) ; type 3 : pas retenu, car **4** clients ont changé plus de deux fois de ville. 5. Parce qu'il ne peut garder qu'une valeur précédente : il faudrait une colonne par changement, et le socle compte **4** clients à **3** villes distinctes. 6. `JOIN dim_client_scd h ON h.id_client = f.id_client AND f.date_vente BETWEEN h.date_debut AND COALESCE(h.date_fin, DATE '2100-01-01')`. 7. Les versions courantes ont une date de fin nulle : la condition devient inconnue, et les ventes concernées disparaissent du rapport. 8. Il prouve qu'aucune ligne n'est perdue ni dupliquée par la jointure. Il ne prouve pas que la **bonne** version a été choisie, ni que l'intervalle est correctement borné : pour cela, il faut vérifier les dates, version par version. 9. Parce que la répartition change sans que le total change : les ventes sont les mêmes, seule la ville ou le segment **attribué** diffère. 10. Le tarif ou le prix de référence lu est celui d'aujourd'hui, alors que les ventes s'étalent sur quatre ans d'inflation tarifaire (**+ 24 %** au total) ; c'est la valeur historisée (**9 045** FCFA) qui doit figurer dans un rapport de marge, parce qu'elle correspond au prix réellement pratiqué.

Module M13.C05 — Le schéma en flocon et ses variantes

Outils : DuckDB 1.5.5 (exécuté) ; pandas (contrôle croisé, exécuté). PostgreSQL et Power BI (cités, non exécutés — règle §1.5).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **choisir entre l'étoile et le flocon** en connaissant le prix de chacun : sur le fil rouge, la même question posée par famille de produits demande **1** jointure en étoile et **3** en flocon, pour un résultat **identique** — le flocon ne rend pas le chiffre plus juste, il rend la hiérarchie partageable ;
2. **reconnaître une table de pont** et l'écrire sans doubler les montants : le socle porte un vrai N-M (**240 000** lignes pour **146 161** tickets, **1,64** ligne par ticket), et le pont qui recopie le montant du ticket sur chacune de ses lignes produit **31 298 080 184** FCFA au lieu de **15 595 154 955**, soit un facteur **2,01** ;
3. **ranger les petits drapeaux hétérogènes dans une dimension déchet** : les **5** modes de paiement, les **3** canaux et le drapeau de retour occupent **30** combinaisons sur **30** possibles, qu'une seule colonne à joindre suffit à porter ;
4. **traiter les faits multiples sans les mélanger** : sur le socle, les **2 809** lignes de retour sont des montants **négatifs** (**175 169 798** FCFA, **1,17 %** des lignes) rangés dans la même table que les ventes, avec un drapeau — un choix qui se paie d'une règle de filtrage ;
5. **dénormaliser exprès, et le mesurer** : écrire la famille **7** fois (table des familles) ou **154** fois (dans la dimension produit) est une décision, pas un accident.

2. Pourquoi cette notion est importante

Les chapitres précédents ont construit une étoile propre : un fait au centre, des dimensions autour, une jointure par attribut. Ce chapitre traite les cas où **l'étoile ne suffit plus** — et il n'en invente aucun : les quatre situations sont déjà dans le socle.

La hiérarchie partagée. La famille de produits sert à la fois au rapport de ventes, à celui des ruptures et à celui du stock. Si elle est écrite dans `dim_produit`, chaque rapport la lit à sa manière ; si elle est extraite dans sa propre table, elle devient un objet unique, que l'on peut corriger une fois. C'est **le flocon**, et son prix est visible : **173** lignes au lieu de **154**, et **3** jointures au lieu d'**1**.

Le N-M réel. Un ticket porte **1,64** produit en moyenne, et **45,4 %** des tickets en portent plus d'un. Chaque ticket est donc relié à plusieurs produits, et chaque produit à plusieurs tickets : c'est une relation **plusieurs à plusieurs**, que seule une table de pont (ou une table de faits au grain de la ligne) peut représenter. Le danger n'est pas conceptuel mais arithmétique : un pont qui porte le montant du ticket sur chacune de ses lignes fabrique un chiffre d'affaires **2,01** fois trop grand.

Les drapeaux hétérogènes. Un fait de ventes porte trois petites listes de valeurs : **5** modes de paiement, **3** canaux, et le drapeau de retour. Groupées sur trois colonnes, elles encombrant chaque requête ; rangées dans une petite table, elles deviennent une **dimension déchet** de **30** lignes que l'on joint une fois.

Les faits multiples. Retours, ruptures, coûts logistiques, objectifs : le modèle porte 7 tables de faits, et **aucune** ne se dit « la même chose » qu'une autre. Le cas le plus trompeur est le retour, qui ressemble à une vente en négatif — et qui l'est, mais avec une conséquence : tout rapport qui oublie de le filtrer publie **175 169 798** FCFA d'indicateurs faux.

Dans les faits. Le socle a fait un choix, et ce chapitre le rend visible : les retours vivent **dans** fait_ventes, distingués par un drapeau, et non dans une table séparée. Ce choix n'est ni bon ni mauvais — il déplace la responsabilité du modèle vers **chaque requête**. Un modèle qui fait ce choix peut publier un chiffre d'affaires net juste **ou** un chiffre d'affaires brut juste, mais il ne peut pas empêcher un rapport de faire la moyenne des deux.

3. Explication simple — l'entrepôt, la fiche d'expédition et la caisse à objets perdus

Le flocon, c'est ranger la fiche produit par étagères. Dans l'étoile, la fiche produit porte tout en clair : le nom du produit, sa sous-catégorie **et** sa famille. C'est pratique et redondant : la famille « Matériaux » est écrite **154** fois, une fois par produit. Dans le flocon, on range par niveaux : une fiche par produit, qui renvoie à une fiche de sous-catégorie (au nombre de **12**), qui renvoie elle-même à une fiche de famille (**7**). La famille n'est plus écrite que **7** fois — mais il faut maintenant ouvrir trois tiroirs pour aller du produit à la famille.

La table de pont, c'est la fiche d'expédition. Un camion livre plusieurs commandes ; une commande part dans plusieurs camions. Aucune des deux fiches ne suffit : il faut une troisième fiche, la **feuille de tournée**, qui dit « ce camion a pris ces commandes ». Sur le fil rouge, le ticket joue le rôle de la tournée : il porte plusieurs produits (**1,64** en moyenne, jusqu'à **4**). Le piège est de croire qu'une tournée peut porter le **montant** : un camion qui livre dix commandes ne vaut pas dix fois le montant de chacune. Le socle le mesure : compter le montant du ticket sur chacune de ses lignes fait passer le chiffre d'affaires de **15 595 154 955** à **31 298 080 184** FCFA.

La dimension déchet, c'est la caisse à objets perdus. Trois petites listes de valeurs qui ne méritent pas chacune sa propre dimension : le mode de paiement, le canal de vente, le retour. On les met ensemble dans une même petite table, et chaque ligne de la table devient une combinaison. Le socle en occupe **30** sur **30** — toutes les combinaisons existent, et cette information a de la valeur : une combinaison absente serait une anomalie à expliquer.

Les faits multiples, ce sont les différents compteurs de l'entrepôt. Un compteur de ventes, un compteur de retours, un compteur de ruptures, un compteur d'objectifs : quatre appareils, quatre unités, quatre histoires. Ils se lisent **à côté**, jamais additionnés.

4. Vocabulaire essentiel

Terme	Définition	Sur le fil rouge
Schéma en flocon	hiérarchie éclatée en plusieurs tables reliées entre elles	produit → sous-catégorie → famille
Hiérarchie partagée	niveaux d'agrégation réutilisables par plusieurs rapports	12 sous-catégories, 7 familles
Dénormalisation volontaire	recopier un attribut pour économiser une jointure, en le déclarant	famille dans dim_produit

Terme	Définition	Sur le fil rouge
Relation N-M	plusieurs à plusieurs : chaque objet d'un côté en touche plusieurs de l'autre	tickets et produits
Table de pont	table qui matérialise un lien N-M sans porter de montant	ligne de ticket, tournée
Facteur de répétition	rapport entre le montant dupliqué et le montant réel	2,01 sur le socle
Dimension déchet	table qui rassemble les petits drapeaux peu nombreux	30 combinaisons
Attribut de fait	colonne du fait qui n'est ni clé ni mesure	mode_paielement, canal
Faits multiples	plusieurs tables de faits, non additionnables entre elles	7 faits, dont les retours
Table de faits de retours	fait séparé portant les retours, quand on ne veut pas de drapeau	choix non retenu ici
Agrégat avant jointure	réduire au grain commun avant de joindre	GROUP BY puis JOIN
Filtre de périmètre	condition qui définit ce que le rapport couvre	est_retour = 0

Définition. Un **schéma en flocon** est un modèle en étoile dont une ou plusieurs dimensions sont **normalisées** : une hiérarchie est éclatée en tables successives, reliées par des clés, au prix d'une jointure par niveau. Il s'oppose à l'**étoile**, où chaque dimension porte directement tous ses attributs.

Définition. Une **table de pont** (ou table d'association) est une table qui représente une relation plusieurs à plusieurs entre deux objets — un ticket et ses produits, une campagne et ses produits, une commande et ses tournées. Elle ne porte **aucun montant** : c'est précisément ce qui la distingue d'une table de faits.

Définition. Une **dimension déchet** rassemble en une seule table les attributs de faible cardinalité et sans hiérarchie (drapeaux, statuts, codes courts), pour éviter d'encombrer la table de faits ou de créer une dimension par drapeau.

Définition. Le **facteur de répétition** mesure combien de fois un montant est compté quand on le lit à travers une relation N-M : c'est le quotient du total obtenu par le total réel. Sur le fil rouge, il vaut **2,01** quand le montant du ticket est répété sur chacune de ses lignes.

Définition. Des **faits multiples** sont plusieurs tables de faits qui décrivent des processus différents — ventes, retours, ruptures, objectifs. Elles partagent des dimensions conformes, et ne s'additionnent **jamais** entre elles.

5. Cours approfondi

5.1 L'étoile et le flocon : deux façons de ranger la même information

La différence tient en une phrase : dans l'étoile, la dimension porte tous ses attributs ; dans le flocon, elle renvoie ses niveaux à des tables successives.

Le socle a choisi l'étoile, et le chapitre C02 a montré pourquoi la famille est écrite dans `dim_produit`. Construisons le flocon pour mesurer ce que ce choix coûte et rapporte :

	Étoile	Flocon
Tables pour la hiérarchie produit	<code>dim_produit</code> seul	<code>dim_produit</code> , <code>dim_famille</code> , <code>dim_sous_categorie</code> ,
Lignes au total	154	173 (154 + 12 + 7)
Jointures pour aller du fait à la famille	1	3

	Étoile	Flocon
Temps mesuré sur le fil rouge	3 ms	3 ms
Résultat obtenu	Matériaux 3 535 582 421 FCFA	le même
La famille est écrite	154 fois	7 fois

Deux observations, et elles sont contre-intuitives.

Le flocon ne rend pas plus juste. La mesure le prouve : les deux lectures rendent un résultat **identique**. Choisir le flocon ne protège d'aucune erreur de calcul.

Le flocon ne coûte rien à la lecture, et beaucoup à la discipline. 3 ms contre 3 ms sur ce moteur, médiane de sept exécutions : ces durées changent d'une exécution et d'une machine à l'autre, seul le rapport entre les deux lectures est stable : l'écart est négligeable, comme celui mesuré en C02 pour la normalisation. Le vrai prix est ailleurs : trois jointures au lieu d'une, c'est trois fois plus d'occasions de se tromper de clé, et une requête que le lecteur ne comprend plus d'un coup d'œil.

Attention. Le flocon se justifie quand une **hiérarchie est partagée** par plusieurs dimensions ou plusieurs processus, ou quand un niveau doit être **corrigé une fois pour toutes**. Il ne se justifie pas par le goût de la normalisation : le socle compte 7 familles pour 154 produits, et personne n'a jamais eu besoin de corriger une famille sans toucher ses produits en quatre ans d'histoire. Le jour où trois référentiels métier se mettront à écrire « Matériaux » de trois façons, la question se reposera.

5.2 La table de pont : le N-M et son facteur de répétition

Le socle porte une relation plusieurs à plusieurs, et elle est massive :

Mesure	Valeur
Lignes de ventes	240 000
Tickets distincts	146 161
Lignes par ticket	1,64 en moyenne, jusqu'à 4
Tickets à une seule ligne	79 763
Tickets à plus d'une ligne	66 398, soit 45,4 %

Chaque ticket est relié à plusieurs produits, et chaque produit apparaît dans plusieurs tickets : c'est un N-M, et il est résolu **par la table de faits elle-même**, dont le grain est la ligne de ticket — et non le ticket. C'est le choix le plus robuste : la table de faits est le pont, et elle porte les montants au bon grain.

La faute apparaît quand on fait du **ticket** un objet porteur de montant, puis qu'on le relie à ses produits par un pont : le montant du ticket se retrouve alors sur chacune de ses lignes.

```
-- Ce que fait cette requete : elle recopie le montant du ticket sur TOUTES ses lignes
WITH t AS (
  SELECT id_ticket, SUM(montant_ttc) AS ca_ticket
  FROM fait_ventes WHERE est_retour = 0 GROUP BY id_ticket
)
SELECT ROUND(SUM(t.ca_ticket)) AS ca_duplique
FROM fait_ventes f JOIN t ON t.id_ticket = f.id_ticket
WHERE f.est_retour = 0;
```

Résultat : **31 298 080 184 FCFA** au lieu de **15 595 154 955**, facteur **2,01**. Le facteur n'est pas le nombre moyen de lignes par ticket (**1,64**) : il est **pondéré** par le montant, car les gros tickets

ont tendance à porter plus de lignes. C'est une leçon à retenir : **le facteur de répétition se mesure, il ne se déduit pas** d'un ratio de comptage.

Attention. Le facteur de répétition d'un montant ne se déduit **jamais** d'un ratio de comptage. Le socle compte **1,64** ligne par ticket, et le montant lu à travers un pont est multiplié par **2,01** : l'écart vient de la pondération, car les tickets à plusieurs lignes pèsent plus lourd que les tickets à une ligne. Un contrôle qui se contenterait de comparer les comptages aurait conclu « la duplication est sous contrôle » alors qu'il manquait **15 702 925 229** FCFA.

Réserve honnête et importante : dans cet exemple, la table de pont est `fait_ventes` elle-même, donc le montant dupliqué vient d'une agrégation préalable au grain du ticket. La règle générale s'écrit ainsi : **une table de pont ne porte pas de montant** ; si un montant doit être rattaché à un objet du N-M (une campagne, une tournée), il appartient à une table de faits propre, à un grain déclaré — et si l'on doit lire les deux ensemble, on agrège chaque côté **avant** de joindre.

Le cas classique — une **campagne** qui couvre plusieurs produits, et un produit couvert par plusieurs campagnes — n'est pas instrumenté sur le socle : il est **cité, non mesuré**. Sa structure est la même que celle du ticket, à une différence près qui change tout : le montant de la campagne n'est rattaché à aucun produit en particulier, donc il n'y a **pas** de grain commun possible, et toute lecture de type « chiffre d'affaires par campagne » doit s'accompagner d'une règle de partage écrite (par exemple au prorata du chiffre d'affaires des produits couverts).

5.3 La dimension déchet : trois drapeaux, une table

Le fait de ventes porte trois colonnes de faible cardinalité : `mode_paiement` (5 valeurs), `canal` (3 valeurs) et `est_retour` (2 valeurs). Ensemble, elles occupent **30** combinaisons — et sur le socle, **30** sur **30** sont effectivement occupées.

Trois façons de les traiter :

Solution	Ce qu'on écrit dans les requêtes	Ce que ça coûte
Les laisser dans le fait	<code>GROUP BY mode_paiement, canal, est_retour</code>	trois colonnes à répéter partout
Trois dimensions séparées	trois jointures	trois tables pour 5 , 3 et 2 lignes
Une dimension déchet	une jointure, <code>GROUP BY id_dechet</code>	un identifiant opaque à décoder

Le socle a choisi la première solution — les trois colonnes sont dans le fait — et c'est défendable à cette taille. La dimension déchet devient préférable dès que le nombre de drapeaux augmente : avec **6** drapeaux de deux valeurs, la table de faits porterait **6** colonnes là où une seule clé suffirait.

Conseil professionnel. Construisez la dimension déchet à **partir des données**, pas d'une liste théorique de combinaisons : les **30** combinaisons du socle sont toutes occupées, mais rien ne garantit qu'elles le restent. Une dimension déchet construite sur les valeurs observées garde sa propriété la plus utile : **toute combinaison absente est une anomalie** — et c'est exactement le signal qu'un rapport de qualité attend.

5.4 Les faits multiples : quatre appareils, quatre unités

Le modèle porte 7 tables de faits. Le chapitre C03 en a mesuré les grains ; ce chapitre traite la question du **partage des montants**.

Le cas le plus instructif est le retour. Sur le socle, un retour est une **ligne de vente à montant négatif** : **2 809** lignes, **175 169 798** FCFA, **1,17** % des lignes, distinguées par `est_retour = 1`. Deux modélisations possibles :

Choix	Avantage	Ce qu'il impose
Un drapeau dans <code>fait_ventes</code> (retenu)	un seul grain, un seul chiffre à filtrer	chaque requête doit filtrer <code>est_retour = 0</code>
Une table de faits de retours	impossible de l'oublier	toute lecture nette doit joindre ou soustraire deux tables

Le socle a choisi le drapeau, et le chapitre C02 de M12 en a montré la conséquence chiffrée : le chiffre d'affaires **net** vaut **15 595 154 955 FCFA** et le chiffre d'affaires **toutes lignes** **15 419 985 157 FCFA** — l'écart, **175 169 798 FCFA**, est exactement le montant des retours. Deux chiffres justes, deux définitions, et une seule question à trancher : **laquelle publie-t-on ?**

Les autres faits multiples du modèle — ruptures (**2 428** lignes), logistique (**264**), objectifs (**218**), stock mensuel (**6 776**), commandes et encaissements (**9 000** chacun) — ne se partagent aucun montant : chacun mesure son processus. Ce qui les unit, ce sont les **dimensions conformes** du chapitre C04 : le magasin, le produit, le client et le temps. C'est par elles qu'on les compare, et jamais par une addition.

5.5 Dénormaliser exprès : la seule question qui compte

Le mot « dénormaliser » porte une réputation injuste : il laisserait croire qu'on renonce à la rigueur. C'est faux. Une dénormalisation assumée est une **décision documentée**, avec un gain et un coût.

Décision du socle	Gain mesuré	Coût assumé
Famille écrite dans <code>dim_produit</code>	1 jointure au lieu de 3	154 écritures au lieu de 7
Trois drapeaux laissés dans le fait	aucune jointure pour les lire	3 colonnes à grouper
Retours dans la table de ventes	un seul grain à connaître	un filtre à ne jamais oublier

À retenir. Le choix entre deux variantes de modèle ne se tranche pas par principe : il se tranche par une mesure portant sur quatre chiffres — **combien de lignes, combien de jointures, combien de temps, quel résultat**. Sur le fil rouge, l'étoile et le flocon donnent le même résultat pour **1** et **3** jointures ; ce n'est que si le résultat avait différé que le choix aurait été autre chose qu'une commodité.

La règle qui se dégage de ces trois lignes tient en une phrase : **on dénormalise pour économiser des gestes, jamais pour économiser de la vérité**. Recopier la famille économise deux jointures et ne change aucun chiffre — le résultat mesuré est identique. Recopier un montant dans une table de pont économise une agrégation et **change tous les chiffres** : c'est la frontière.

6. Exemple concret — le ticket qui vaut quatre fois son montant

Prenez un ticket ordinaire du fil rouge, avec **4** lignes (le maximum du socle) : deux sacs de ciment, un pot de peinture, un rouleau, une pince.

Vu au grain de la ligne — le grain de `fait_ventes` — ce ticket est **4** lignes, chacune avec sa quantité et son montant. Le chiffre d'affaires du ticket est la somme de ses 4 lignes, comptée **une fois**.

Vu au grain du ticket, si l'on décide que le ticket est l'objet porteur du montant, on obtient une ligne par ticket, avec le total du ticket. Jusque-là, rien de faux : c'est simplement un autre grain.

Vu à travers un pont, le danger apparaît. Si l'on relie le ticket à ses 4 produits par une table de pont qui porte aussi `ca_ticket`, alors le montant du ticket est lu **4** fois. À l'échelle du socle, cela donne **31 298 080 184** FCFA au lieu de **15 595 154 955** — facteur **2,01**. Et le facteur n'est **pas** 1,64 : il est plus grand, parce que les tickets à plusieurs lignes pèsent plus lourd en montant que les tickets à une seule ligne.

Ce que ce ticket enseigne, en une phrase : **une table de pont sert à relier, pas à mesurer**. Le montant appartient au grain qui le compte ; le pont dit seulement quels objets vont ensemble.

7. Démonstration pas à pas — quatre gestes

Étape 1 — décider étoile ou flocon, et l'écrire.

Posez la question du partage : la hiérarchie est-elle utilisée par plusieurs rapports, ou par un seul ? Sur le fil rouge, la famille de produits sert aux ventes, au stock et aux ruptures : elle est **partagée**. Le socle la garde pourtant dans `dim_produit`, avec 7 valeurs pour **154** lignes : c'est un choix explicite, écrit dans le commentaire de la table.

Étape 2 — mesurer le prix de la variante non retenue.

```
CREATE OR REPLACE TABLE dim_sous_categorie AS
SELECT DISTINCT sous_categorie, famille FROM dim_produit;
CREATE OR REPLACE TABLE dim_famille AS SELECT DISTINCT famille FROM dim_produit;
```

Puis la même question, par les deux chemins, et la comparaison : **1** jointure et **3** ms en étoile, **3** jointures et **3** ms en flocon, résultat **identique**. Une variante se mesure avant d'être écartée, sinon on ne saura pas répondre à celui qui la proposera dans six mois.

Étape 3 — écrire la contrainte du pont.

La règle tient en une ligne, et elle s'écrit dans le modèle : **une table de pont ne porte pas de montant**. Si un montant doit y apparaître, c'est le signe que l'objet du N-M est en réalité un fait — et il lui faut sa propre table, à son propre grain déclaré.

Étape 4 — vérifier le facteur de répétition.

```
-- Controle : le montant doit etre identique avec et sans passage par le pont
SELECT ROUND(SUM(montant_ttc)) FROM fait_ventes WHERE est_retour = 0;
```

Attendu : **15 595 154 955** FCFA, identique à toute autre lecture du même périmètre. Toute valeur supérieure signale une duplication — le facteur mesuré (**2,01** sur le socle) dit de combien.

8. Erreurs fréquentes

Erreur	Ce qu'on observe	La correction
Mettre un montant dans une table de pont	31 298 080 184 FCFA au lieu de 15 595 154 955	le montant reste dans la table de faits
Déduire le facteur de répétition du nombre de lignes	on attend 1,64 , on mesure 2,01	mesurer le facteur, toujours
Floconner sans raison	3 jointures au lieu d' 1 pour un résultat identique	partager la hiérarchie, ou l'écrire dans la dimension
Garder un drapeau sans règle de filtre	175 169 798 FCFA de retours dans le net	écrire la règle, et la contrôler
Créer une dimension par drapeau	trois tables de 5 , 3 et 2 lignes	une dimension déchet, ou les colonnes du fait

Erreur	Ce qu'on observe	La correction
Additionner deux faits multiples	voir chapitre C03 : + 43,0 %	comparer par dimensions conformes

9. Bonnes pratiques professionnelles

1. **Une variante de modèle se mesure avant d'être retenue ou écartée** : lignes, jointures, temps, résultat. Le socle applique cette règle à l'étoile contre le flocon.
2. **Une table de pont ne porte que des clés** — et, si elle doit porter une pondération, elle la déclare comme telle.
3. **Le facteur de répétition se contrôle sur le montant**, jamais sur les comptages : le socle donne **2,01** là où le ratio de lignes vaut **1,64**.
4. **Le drapeau de périmètre s'écrit dans le nom du rapport** : « chiffre d'affaires net, retours exclus » évite les discussions au moment de la publication.
5. **Une dimension déchet se construit sur les combinaisons observées**, pour que toute combinaison absente devienne une anomalie détectable.
6. **Les faits multiples se comparent par les dimensions conformes** du chapitre C04, jamais par addition.

10. Exercice guidé

Sujet. Un rapport publie « le chiffre d'affaires par mode de paiement », et un second publie « le chiffre d'affaires par canal et par mode de paiement ». Les deux doivent sommer au même total. Un stagiaire a écrit la première requête sans filtrer les retours. Vous avez vingt minutes pour trouver l'écart, le chiffrer et corriger.

Étape 1 — écrire les deux requêtes telles quelles. La première groupe sur `mode_paiement` sans condition ; la seconde groupe sur `canal` et `mode_paiement` avec `est_retour = 0`.

Étape 2 — mesurer l'écart. Le montant des retours est de **175 169 798** FCFA, soit **1,17 %** des lignes : le total de la première requête vaut **15 419 985 157** FCFA (toutes lignes), celui de la seconde **15 595 154 955** FCFA (net). Les deux sont **justes** ; elles ne parlent pas du même périmètre.

Étape 3 — nommer la règle. Un fait de ventes porte deux définitions dans une seule table. Le rapport doit porter sa définition dans son titre : « net, retours exclus » ou « toutes lignes, retours inclus ». La pire des réponses est de ne rien écrire et de laisser le lecteur comparer deux chiffres différents en croyant à une erreur de calcul.

Étape 4 — vérifier la couverture des combinaisons. Les **5** modes de paiement et les **3** canaux occupent **30** combinaisons sur **30** possibles : le rapport par canal et mode de paiement affiche donc **30** lignes, sans trou. Si une combinaison manquait, il faudrait la signaler avant de conclure à une absence d'activité.

Barème (15 points) : les deux requêtes écrites (3) · l'écart chiffré et expliqué (4) · la règle de nommage formulée (4) · le contrôle des **30** combinaisons (4).

11. Exercices autonomes

Exercice 5.1. Construisez le flocon de la hiérarchie produit sur le socle : deux tables (`dim_sous_categorie`, `dim_famille`) et la dimension produit allégée. Donnez le nombre de lignes

des trois tables, la requête qui donne le chiffre d'affaires par famille dans chaque modèle, et vérifiez que les deux résultats sont identiques. Concluez en trois lignes sur ce que le flocon a rapporté.

Exercice 5.2. Écrivez la requête qui compte les tickets portant plus d'une ligne, puis celle qui donne la moyenne de lignes par ticket. Comparez ces deux nombres au facteur de répétition du montant (**2,01**) et expliquez, en cinq lignes, pourquoi le facteur est plus grand que la moyenne.

Exercice 5.3. Le N-M des tickets. Écrivez une table de pont `pont_ticket_produit` (un ticket, un produit) et la requête qui donne, pour chaque produit, le nombre de tickets distincts où il apparaît. Puis dites pourquoi il serait faux d'y ajouter une colonne `montant_ticket`, en citant le chiffre que cette colonne ferait apparaître.

Exercice 5.4. La dimension déchet. Construisez `dim_dechet` à partir des combinaisons observées de `mode_paiement`, `canal` et `est_retour`, puis écrivez la requête qui donne le chiffre d'affaires par canal **à travers** cette dimension. Combien de lignes contient la dimension, et pourquoi ce nombre est-il exactement égal à l'espace des combinaisons possibles ?

Exercice 5.5. Les retours. Écrivez les deux requêtes du fil rouge : le chiffre d'affaires **net** et le chiffre d'affaires **toutes lignes**. Donnez les deux totaux et l'écart, puis expliquez en quatre lignes ce qu'un rapport gagne et ce qu'il perd en choisissant le drapeau plutôt qu'une table de retours séparée.

12. Correction détaillée

Exercice 5.1. Les trois tables comptent **154** produits, **12** sous-catégories et **7** familles, soit **173** lignes au total contre **154** en étoile. Les deux requêtes (par famille, avec **1** jointure et avec **3**) rendent un résultat **identique** : Matériaux **3 535 582 421** FCFA en tête. Le flocon n'a donc rien changé au chiffre ; il a partagé la hiérarchie et ramené l'écriture de la famille de **154** à **7** occurrences — au prix de deux jointures supplémentaires, pour un temps de lecture indiscernable (**3** ms dans les deux cas). Il se justifie le jour où plusieurs référentiels doivent partager le même niveau « famille », pas avant.

Exercice 5.2. Le comptage :

```
SELECT COUNT(*) AS tickets_multi FROM (
  SELECT id_ticket FROM fait_ventes GROUP BY id_ticket HAVING COUNT(*) > 1
);
```

66 398 tickets portent plus d'une ligne, soit **45,4 %**, et la moyenne vaut **1,64** ligne par ticket. Le facteur de répétition du montant vaut **2,01**, donc **plus grand** que la moyenne : quand on répète un montant, les tickets lourds — qui portent souvent plus de lignes — comptent davantage. Le facteur est une moyenne **pondérée par le montant**, pas un ratio de comptage ; c'est pourquoi il se mesure.

Exercice 5.3. La table de pont et la requête :

```
CREATE OR REPLACE TABLE pont_ticket_produit AS
SELECT DISTINCT id_ticket, id_produit FROM fait_ventes;

SELECT id_produit, COUNT(DISTINCT id_ticket) AS tickets
FROM pont_ticket_produit GROUP BY id_produit ORDER BY 2 DESC;
```

Ajouter `montant_ticket` à cette table serait faux : le montant du ticket se retrouverait répété autant de fois que le ticket a de produits, et le chiffre d'affaires lu à travers la table passerait de **15 595 154 955** à **31 298 080 184** FCFA — facteur **2,01**. Une table de pont relie des objets ; elle ne mesure pas.

Exercice 5.4. La dimension déchet et sa lecture :

```
CREATE OR REPLACE TABLE dim_dechet AS
SELECT ROW_NUMBER() OVER (ORDER BY mode_paiement, canal, est_retour) AS id_dechet,
       mode_paiement, canal, est_retour
FROM (SELECT DISTINCT mode_paiement, canal, est_retour FROM fait_ventes);

SELECT COUNT(*) FROM dim_dechet;    -- 30

SELECT d.canal, ROUND(SUM(f.montant_ttc)) AS ca
FROM fait_ventes f
JOIN dim_dechet d
  ON d.mode_paiement = f.mode_paiement
 AND d.canal = f.canal
 AND d.est_retour = f.est_retour
WHERE f.est_retour = 0
GROUP BY 1 ORDER BY 2 DESC;
```

La lecture par la dimension donne Magasin **10 295 255 849** FCFA, Livraison **3 732 464 992** FCFA et Téléphone **1 567 434 114** FCFA.

La dimension contient **30** lignes, soit exactement l'espace des combinaisons possibles (**5** modes de paiement × **3** canaux × **2** valeurs de retour). Cette égalité est une information, pas une coïncidence : elle dit qu'aucune combinaison n'est absente — et donc que toute combinaison manquante, plus tard, signalera un changement de pratique ou une anomalie.

La jointure se fait sur les **trois** colonnes : c'est le prix de cette dimension, et il faut le dire — la clé `id_dechet` sert à lire la table, pas à la joindre. Une variante plus économique consiste à calculer cet identifiant **dans** la table de faits, au chargement ; elle se paie d'une colonne dans les faits, et se justifie quand la dimension est jointe par plusieurs rapports.

Exercice 5.5. Les deux requêtes :

```
SELECT ROUND(SUM(montant_ttc)) FROM fait_ventes WHERE est_retour = 0;    -- 15 595 154 955 FCFA
SELECT ROUND(SUM(montant_ttc)) FROM fait_ventes;                        -- 15 419 985 157 FCFA
```

L'écart est de **175 169 798** FCFA, exactement le montant des **2 809** lignes de retour (**1,17 %** des lignes). Le drapeau gagne la simplicité : un seul grain, un seul objet à connaître. Il perd l'impossibilité de l'oubli : toute requête doit filtrer, et rien dans le modèle ne l'impose. La table de retours séparée gagne la sécurité et perd la simplicité : toute lecture nette devient une opération sur deux tables, et il faut décider ce que signifie « jointure » entre une vente et un retour.

13. Mini-projet de chapitre

« **La note de variante : étoile ou flocon pour la hiérarchie produit** » (2 h). Produisez, pour le fil rouge : (1) le tableau comparatif mesuré des deux modèles — lignes, jointures, temps, résultat — avec les requêtes qui l'établissent ; (2) la requête du N-M des tickets et son contrôle de non-duplication ; (3) `dim_dechet` construite sur les combinaisons observées, avec le nombre de combinaisons occupées et l'espace théorique ; (4) la liste des **3** dénormalisations volontaires du socle, chacune avec son gain mesuré et son coût assumé ; (5) une page de règle : « quand passer au flocon, quand garder un drapeau, quand séparer un fait ». Ce livrable alimente **P4** (note de choix, 3 pages) du projet de module.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Flocon	173 lignes au lieu de 154 , 3 jointures au lieu d' 1
Résultat du flocon	identique à l'étoile : Matériaux 3 535 582 421 FCFA
Prix du flocon	Deux jointures de plus à écrire juste, et un temps de lecture indiscernable
Gain du flocon	la famille écrite 7 fois au lieu de 154
N-M du socle	240 000 lignes pour 146 161 tickets (1,64 par ticket)
Tickets multi-lignes	66 398 , soit 45,4 % , jusqu'à 4 lignes
Facteur de répétition	2,01 (31 298 080 184 contre 15 595 154 955 FCFA)
Dimension déchet	30 combinaisons occupées sur 30 possibles
Retours	2 809 lignes, 175 169 798 FCFA, 1,17 % des lignes
Deux périmètres	net 15 595 154 955 · toutes lignes 15 419 985 157 FCFA
Dénormaliser	pour économiser des gestes, jamais de la vérité

15. À retenir

1. **Le flocon ne rend pas le chiffre plus juste** : il rend la hiérarchie partageable et corrigable en un point. Sur le fil rouge, les deux modèles rendent exactement le même résultat.
2. **Une relation N-M se résout par une table de pont**, et cette table ne porte **aucun montant** : y recopier le montant du ticket produit **2,01** fois le chiffre d'affaires.
3. **Le facteur de répétition se mesure** : **2,01** ici, quand le ratio de lignes par ticket vaut **1,64** — parce que la pondération par le montant n'est pas celle du comptage.
4. **Les petits drapeaux se rangent** dans une dimension déchet construite sur les combinaisons observées : **30** occupées sur **30**, et toute absence devient une anomalie lisible.
5. **Un drapeau de périmètre est une règle, pas un détail** : **175 169 798** FCFA séparent le chiffre d'affaires net du chiffre d'affaires toutes lignes, et rien dans le modèle n'oblige une requête à choisir.

À retenir. Une variante de modèle ne se choisit pas au goût : elle se **mesure**. Ce chapitre a mesuré les quatre variantes du socle — flocon, pont, déchet, faits multiples — et chacune a livré le même verdict : ce qui coûte cher n'est jamais la ligne supplémentaire, c'est la règle qu'on oublie d'écrire.

16. Évaluation formative

1. Donnez le critère qui distingue l'étoile du flocon, et le prix du flocon sur le fil rouge.
2. Le flocon change-t-il le résultat d'un rapport ? Justifiez par la mesure.
3. Qu'est-ce qu'une table de pont ? Citez le N-M du socle et ses deux nombres.
4. Quelle règle interdit absolument de mettre un montant dans une table de pont, et pourquoi ?
5. Le facteur de répétition vaut **2,01** alors que la moyenne de lignes par ticket vaut **1,64** : expliquez l'écart.
6. À quoi sert une dimension déchet, et combien de lignes compte celle du socle ?
7. Pourquoi les **30** combinaisons occupées sur **30** sont-elles une information utile ?

8. Quelles sont les deux modélisations possibles d'un retour, et que coûte chacune ?
9. Que valent le chiffre d'affaires net et le chiffre d'affaires toutes lignes, et quel est l'écart ?
10. « On dénormalise pour économiser des gestes, jamais de la vérité. » Illustrez cette phrase par deux décisions du socle, dont une qui change tous les chiffres.

Corrigé : 1. Le nombre de jointures pour atteindre un attribut : **1** en étoile, **3** en flocon ; le flocon coûte **2** jointures, et fait passer la dimension de **154** à **173** lignes au total. 2. Non : les deux lectures rendent un résultat **identique** (Matériaux **3 535 582 421** FCFA). 3. Une table qui matérialise une relation plusieurs à plusieurs : sur le socle, tickets et produits — **240 000** lignes pour **146 161** tickets. 4. Parce que le montant serait répété autant de fois que l'objet a de liens : **31 298 080 184** au lieu de **15 595 154 955** FCFA. 5. Parce que le facteur est pondéré par le montant : les tickets à plusieurs lignes pèsent plus lourd que les tickets à une ligne. 6. À rassembler des drapeaux de faible cardinalité en une seule table : **30** lignes pour le socle. 7. Parce que l'absence d'une combinaison devient alors un signal : si une combinaison disparaît, c'est un changement de pratique ou une anomalie, pas un simple vide. 8. Un drapeau dans la table de ventes (simple, mais chaque requête doit filtrer) ou une table de faits de retours (sûr, mais toute lecture nette porte sur deux tables). 9. Net **15 595 154 955** FCFA, toutes lignes **15 419 985 157** FCFA, écart **175 169 798** FCFA, soit les **2 809** lignes de retour. 10. Dénormalisation qui ne change rien : la famille dans `dim_produit` (**1** jointure au lieu de **3**, résultat identique). Dénormalisation qui change tout : le montant du ticket dans une table de pont (**2,01** fois le chiffre d'affaires).

Module M13.C06 — Le temps et le calendrier

Outils : DuckDB 1.5.5 (exécuté) ; pandas (contrôle croisé, exécuté). PostgreSQL et Power BI (cités, non exécutés — règle §1.5).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **construire et brancher une dimension de temps** : le socle porte un calendrier de **1 339** lignes, du 2023-01-01 au 2026-08-31, et la jointure rend **240 000** lignes sur **240 000** — c'est la fermeture du trou déclaré au chapitre 4, où la dimension de temps n'était branchée sur aucun fait ;
2. **mesurer la complétude du calendrier dans les deux sens** : les jours du calendrier sans vente, et les dates des faits absentes du calendrier. Sur le socle, la première réponse vaut **0** et la seconde vaut **0** pour les ventes, mais **13** dates de livraison tombent hors du calendrier, pour **46** commandes et **32 772 075** FCFA ;
3. **lire le temps par attributs, pas par la date** : les jours fériés (**18**), le dimanche (**829 983 276** FCFA, soit **5,32 %** du chiffre d'affaires), le poids du quatrième trimestre (**29,7 %**) et celui de décembre dans ce trimestre (**33,6 %**) ne se calculent pas sur une colonne de date : ils se lisent dans la dimension ;
4. **comparer deux périodes honnêtement** : sur le même socle, le même chiffre d'affaires se lit **- 28,9 %**, ou **+ 15,4 %**, selon que l'année en cours est comparée entière ou sur les **8** premiers mois — **44,3** points d'écart pour une seule erreur de borne ;
5. **choisir la nature du temps dans une table de faits** : une date de vente, **3** dates dans un fait de commande ou d'encaissement, un mois dans **4** autres faits — et savoir ce que le modèle ne saura jamais dire, faute d'heure (**0** colonne d'heure sur **17**).

2. Pourquoi cette notion est importante

Le temps est la seule dimension qu'un modèle de données peut **remplir d'avance**. Les clients arrivent au fil des ventes, les produits aussi ; les jours, eux, sont connus avant les faits, et même avant l'ouverture du magasin. Un calendrier écrit une fois sert ensuite à tout le monde : il porte le trimestre, le jour de la semaine, les fériés, l'exercice fiscal et les semaines commerciales, autant d'attributs qu'aucune table de faits n'a envie de recalculer.

C'est aussi la dimension la plus souvent **déclarée et jamais branchée**. Le chapitre 4 a mesuré la conformité des dimensions du socle et trouvé un résultat à zéro : la dimension de temps existe, elle est propre, elle porte **1 339** lignes — et aucun fait ne la référence. Ce chapitre ferme ce trou, et il explique pourquoi ce trou existe : un calendrier qui arrive dans un entrepôt avant les faits reste inutile tant que personne ne l'a joint, et rien dans la base ne le signale.

La suite du raisonnement tient en trois pièges, tous mesurés dans ce chapitre. Le premier est une **borne de temps** : comparer une année de **243** jours à une année complète produit un effondrement de **- 28,9 %** qui n'existe pas. Le deuxième est une **date hors calendrier** : un rapport qui joint en interne perd **46** commandes sans afficher la moindre erreur. Le troisième est une **ambiguïté de vocabulaire** : le socle porte deux calendriers concurrents, la semaine nor-

malisée qui plafonne à **52** et la semaine commerciale qui monte à **53**. Trois pièges, aucune faute de calcul, et des chiffres faux quand même.

3. Explication simple — le calendrier du mur et le cahier de caisse

Imaginez la salle de réunion d'un magasin. Sur le mur, un calendrier : une case par jour, avec la date, le jour de la semaine, les jours fériés en rouge. Personne ne le remplit, il est imprimé d'avance.

À côté, le cahier de caisse : une ligne par vente, avec la date du jour en marge. Il ne contient que les jours où l'on a vendu quelque chose.

	Calendrier du mur	Cahier de caisse
Qui le remplit	personne, il est imprimé	la caissière, à chaque vente
Combien de lignes	1 339 jours	240 000 ventes
Que dit-il de janvier	il existe, même si le magasin est fermé	rien, s'il n'y a pas de vente
Que porte-t-il en plus de la date	trimestre, semaine, férié, exercice	date, montant, produit, client
Sert à quoi	comparer, cumuler, compléter	raconter ce qui s'est passé

Le geste de ce chapitre consiste à **poser le cahier à côté du calendrier du mur**. C'est tout : on joint chaque vente à son jour. Et ce geste simple répond à des questions qu'aucune des deux sources ne sait poser seule. Combien de dimanches compte l'année, et combien pèsent-ils ? Quel chiffre d'affaires a été fait un jour férié ? Le mois de décembre pèse-t-il plus que les trois autres mois du trimestre ? Le cahier de caisse ne le dira jamais ; le calendrier du mur le dit si on le lui demande.

L'inverse est vrai aussi. Le calendrier du mur sait quel jour on n'a **pas** vendu : **1 339** cases contre **1 339** jours travaillés sur le socle, donc aucune case vide. Sur un vrai magasin, cette comparaison est le premier rapport qu'on ouvre, parce qu'une case vide au calendrier est soit une fermeture légitime, soit une panne de caisse.

4. Vocabulaire essentiel

Définition. Une **dimension de temps** (ou **calendrier**) est une table dont la ligne représente un jour, et dont les colonnes sont les lectures de ce jour : année, trimestre, mois, semaine, jour de la semaine, jour férié, exercice. Elle ne décrit aucun client, aucun produit : elle décrit le temps lui-même, et se remplit par génération, jamais par les faits.

Définition. La **clé de date** est l'identifiant qui relie un fait à son jour. Elle prend deux formes : un entier lisible, qui colle l'année, le mois et le jour dans cet ordre, ou la date native du moteur. Le socle utilise la date native et écrit la comparaison `d.date = f.date_vente` ; les deux formes se valent, à condition de choisir une fois pour tout le modèle.

Définition. La **semaine normalisée** (dite ISO) est la semaine qui commence le lundi et dont la première contient au moins quatre jours de janvier. Elle a deux propriétés qui étonnent : elle peut valoir **52** pour le 1er janvier d'une année, et une année peut compter **53** semaines.

Définition. Un **exercice fiscal décalé** est une période comptable qui ne coïncide pas avec l'année civile. Le socle en porte un : l'exercice 2022 court du 2023-01-01 au 2023-06-30, et **725** jours du calendrier appartiennent à une année fiscale différente de leur année civile.

Définition. Une **dimension à rôles** est une même dimension référencée plusieurs fois par un fait, chaque référence jouant un rôle différent. Un fait de commande porte trois dates : commandée, promise, livrée. Les trois pointent vers le même calendrier, et le rôle se lit dans le nom de l'alias, jamais dans la table.

Définition. Une **période partielle** est une période encore en cours : au 2026-08-31, l'année 2026 compte **243** jours sur **365**. Comparer une période partielle à une période complète n'a aucun sens ; une comparaison n'est recevable que si les deux bornes de fin sont identiques.

5. Cours approfondi

5.1 Pourquoi une dimension plutôt qu'une colonne de date

Une table de faits porte déjà la date : elle n'a besoin de rien d'autre pour être juste. Ce qu'elle ne sait pas faire, c'est répondre aux questions qui portent sur le **contexte** de cette date.

Prenons le fait de ventes du socle : **17** colonnes, dont une date. Pour obtenir le chiffre d'affaires du premier trimestre, il faut écrire un test sur le mois ; pour les fériés, il faut une liste de dates tenue à la main ; pour la semaine commerciale, une formule ; pour l'exercice fiscal, une seconde formule. Chaque question devient du code, et chaque code peut se tromper différemment.

La dimension de temps remplace ces formules par une jointure. Le socle en porte **15** colonnes : la date, l'année, le trimestre, le mois et son libellé, la semaine normalisée, le jour de la semaine et son libellé, le dimanche, le férié et son libellé, l'année-mois, la semaine commerciale, l'année fiscale et la période fiscale. Une seule jointure ouvre les quinze.

Le gain n'est pas la vitesse. Le gain est que **la définition vit à un seul endroit**. Si le calendrier décide que la semaine commence le lundi, tous les rapports la suivent ; s'il faut passer à la semaine du dimanche au samedi, une seule table change.

D'où la règle, et elle est absolue : **une seule table de dates par modèle**. Pas une par fait, pas une par service, pas une par usage. Le socle l'applique : sa table de ventes, ses **3** dates de commande et ses **3** dates d'encaissement pointent toutes vers la même dimension de **1 339** lignes, et c'est ce qui rend le chiffre d'affaires, le délai de livraison et le délai d'encaissement comparables sur le même axe de temps. Deux calendriers dans un même modèle, ce sont deux façons de découper l'année, donc deux rapports qui ne se recoupent pas.

5.2 Ce que le calendrier du socle contient

Mesure	Valeur	Lecture
Lignes du calendrier	1 339	3 ans et 8 mois de jours
Première date	2023-01-01	un dimanche, jour férié
Dernière date	2026-08-31	l'année en cours est ouverte
Découpage annuel	2023 : 365 jours — 2024 : 366 jours — 2025 : 365 jours — 2026 : 243 jours	une année bissextile dans la période
Jours fériés	18 jours, 5 libellés distincts	tous travaillés par le magasin
Dimanches	192	une seule valeur du jour de la semaine
Semaine maximum normalisée,	52	aucune année de la période ne compte 53 semaines
Semaine maximum commerciale,	53	8 jours portent cette valeur

Les **5** libellés de fériés du socle sont ceux du pays où opère Sahel Distribution : Nouvel An, Fête du Travail, Anniversaire de la Révolution, Fête nationale et Noël. Tous portent une date fixe, et la répétition se lit dans le calendrier : le 01-01, le 05-01 et le 08-05 reviennent **4** fois chacun sur la période, le 12-11 et le 12-25 seulement **3** fois, parce que le calendrier s'arrête le 2026-08-31 et que leurs deux dates tombent après la borne. Le compte des jours fériés se vérifie ainsi de tête : $4 + 4 + 4 + 3 + 3 = \mathbf{18}$. Ce petit calcul dit quelque chose d'important sur les compteurs : **18** n'est pas le nombre de jours fériés du pays, c'est le nombre de jours fériés compris entre la première et la dernière date du calendrier. Un compteur dépend toujours de sa borne, et la borne ici se lit dans la dimension, pas dans le fait.

Le 29 février 2024 mérite un mot : ce jour n'existe qu'une année sur quatre, le calendrier le porte une fois, et le fait de ventes y réalise **195** lignes pour **13 044 279** FCFA. Un modèle qui connaît les années bissextiles n'a rien à faire pour ce jour ; un modèle qui compte les jours en divisant par **365** se trompe une année sur quatre.

La dernière ligne du tableau vaut un avertissement. Une année civile fait **365** ou **366** jours, mais une année **commerciale** faite de semaines de sept jours fait 364 ou 371 jours : les deux calendriers ne se superposent jamais exactement, et le socle en porte la trace en portant les deux.

5.3 Le branchement : fermer le trou déclaré au chapitre 4

Le chapitre 4 avait mesuré la conformité des dimensions du socle et rendu un zéro pour la dimension de temps : aucun fait ne la référençait. La fermeture est une jointure, et elle se contrôle dans les deux sens.

Contrôle	Résultat
Lignes du fait jointes au calendrier	240 000 sur 240 000
Dates de vente absentes du calendrier	0
Jours du calendrier sans aucune vente	0
Rapport des volumes	1 ligne de calendrier pour 179,2 lignes de faits

Le rapport des volumes explique une propriété du calendrier qu'il faut connaître avant de le prolonger : c'est la seule table du modèle dont le volume ne suit pas l'activité. Doubler les ventes ne change rien au calendrier ; traverser une année de plus lui ajoute **365** lignes, et une année bissextile **366**. C'est une table qu'on remplit par décennie, pas par chargement.

Le fait que la seconde et la troisième ligne valent toutes deux zéro mérite qu'on s'y arrête. Cela signifie que le magasin du socle n'a **jamais** fermé : un calendrier complet et des faits complets sont deux affirmations différentes, et leur égalité est un résultat, pas une hypothèse de départ.

Dans les faits. Le socle est un cas limite, et il faut le dire avant d'en tirer une règle. Ses **1 339** jours portent tous au moins une vente, y compris les **18** jours fériés et les **192** dimanches. Un commerce réel ferme : jours de fête religieuse, inventaire annuel, travaux, panne de caisse. Sur un tel modèle, le contrôle « jours du calendrier sans vente » ne rendrait pas **0** mais une dizaine de journées par an, et chacune d'elles demanderait une explication avant d'être acceptée. C'est précisément parce que le socle répond **0** que le contrôle des dates hors calendrier, lui, devient la question intéressante : quand tout est complet d'un côté, le manque se cache de l'autre, et il se cachait dans les livraisons de septembre et d'octobre 2026.

Attention. Une date d'un fait peut **sortir du calendrier**, et une jointure interne la supprime alors en silence. Le calendrier s'arrête au 2026-08-31 ; les dates de livraison du fait de commandes vont jusqu'au 2026-10-14. Sur le socle, **13** dates de livraison tombent hors du calendrier, pour **46** commandes et **32 772 075** FCFA — sur les **6 491 975 325** FCFA de commandes livrées, soit **0,5** %. La jointure interne rend **8 635** lignes au lieu de **8 681**, et le montant manquant n'apparaît nulle part : aucun message, aucune ligne en erreur, un rapport simplement incomplet. La correction tient en **122** lignes ajoutées au calendrier, jusqu'au 2026-12-31, et elle ramène les **8 681** commandes, leur montant, sans toucher aux ventes (**240 000** lignes). Un calendrier se prolonge **avant** que les faits n'y arrivent.

5.4 Les attributs qu'aucun fait ne sait dire

Le socle offre une occasion rare : un magasin qui travaille tous les jours, y compris les jours fériés. Le tableau suivant ne se calcule pas sur le fait de ventes ; il se calcule en joignant le calendrier.

Lecture	Valeur	Part du chiffre d'affaires
Lundi	2 663 495 054 FCFA	17,1 %
Dimanche	829 983 276 FCFA	5,32 %
Week-end (samedi et dimanche)	—	21,0 %
Jours fériés (18 jours, tous travaillés)	234 388 370 FCFA	1,5 %
Quatrième trimestre 2025	—	29,7 %
Décembre, dans ce quatrième trimestre	—	33,6 %

Le dimanche est le jour le plus faible de la semaine : **3,21** fois moins de chiffre d'affaires que le lundi, et **4 322 830** FCFA en moyenne par dimanche contre **11 646 867** FCFA en moyenne par jour. Les deux journées les plus faibles du socle sont d'ailleurs deux dimanches, dont le 2023-06-18 avec **1 313 588** FCFA. La journée la plus forte, le 2026-03-17, est un mardi, à **26 933 751** FCFA — soit **20,5** fois la plus faible, et **31** journées dépassent les 20 millions de FCFA.

Aucune de ces phrases ne demande une jointure supplémentaire : tous ces attributs sont dans la dimension, écrits une fois, et lus autant de fois qu'on veut. C'est le rendement réel d'une dimension de temps.

5.5 Les deux calendriers du socle

La semaine normalisée et la semaine commerciale ne s'accordent pas, et le socle le montre.

Semaine normalisée		Semaine commerciale
Commence le	lundi	lundi (fixé par l'entreprise)

Semaine normalisée			Semaine commerciale		
Première semaine de l'année	celle qui contient au moins quatre jours de janvier		la première semaine complète de l'exercice		
Maximum sur la période	52		53		
1er janvier 2023	semaine 52		semaine 1		
Jours portant la valeur 53	aucun		8		

La première ligne du tableau est la source d'erreur la plus fréquente des rapports hebdomadaires : le 1er janvier 2023 appartient à la semaine **52**, celle de décembre 2022, parce que la semaine normalisée n'appartient pas à l'année où elle commence. Un rapport qui groupe les ventes par année et par semaine normalisée range donc ce jour dans une semaine qui n'existe pas dans l'année des ventes, et deux rapports annuels successifs se recouvrent ou laissent un trou.

À retenir. Le temps ne se lit pas dans une colonne de date, il se lit dans des **attributs** préparés une fois pour toutes. Et quand deux calendriers coexistent, comme ici la semaine normalisée et la semaine commerciale, ce n'est pas une incohérence à corriger : c'est un choix à écrire, parce qu'un rapport hebdomadaire qui ne dit pas quel calendrier il suit n'est pas vérifiable.

5.6 L'exercice fiscal, et un nom qui promet plus que la mesure

L'exercice fiscal du socle ne suit pas l'année civile : l'exercice 2022 court du 2023-01-01 au 2023-06-30, ce qui place **725** jours du calendrier dans une année fiscale différente de leur année civile. Deux chiffres d'affaires annuels coexistent donc pour la même activité : celui de l'année civile et celui de l'exercice, et un rapport qui n'annonce pas lequel il utilise ne peut pas être comparé à un autre.

Le calendrier porte deux colonnes pour cela : `annee_fiscale` et `periode_fiscale`. La première tient sa promesse. La seconde, non : elle ne prend que **2** valeurs sur les **1 339** lignes du calendrier, alors que son nom laisse attendre un découpage en douze périodes. C'est un défaut de nom, pas un défaut de calcul, et il coûte cher : un auteur de rapport qui écrit « par période fiscale » en attendant douze lignes en reçoit deux, et croit à une erreur de jointure.

La leçon dépasse ce cas. Un attribut dont le **libellé promet plus que la mesure** se détecte par un contrôle simple : compter les valeurs distinctes de chaque colonne du calendrier avant de s'en servir. Un contrôle de dix lignes de SQL évite des heures de doute.

5.7 Le mois partiel : deux vérités contradictoires sur la même table

Le calendrier s'arrête au 2026-08-31. L'année 2026 compte donc **243** jours sur **365**, et c'est la lecture la plus utile du chapitre, parce qu'elle ne demande aucun outil nouveau.

Comparaison	Chiffre d'affaires	Écart
2026 entier contre 2025 entier	3 360 553 372 FCFA contre 4 724 700 286 FCFA	– 28,9 %
Janvier à août 2026 contre janvier à août 2025	3 360 553 372 FCFA contre 2 913 055 394 FCFA	+ 15,4 %

Les deux lignes portent sur la même table, la même jointure et le même filtre de retours. La première annonce un effondrement, la seconde une croissance, et il y a **44,3** points d'écart entre les deux. La règle est donc simple : une comparaison entre deux périodes n'est recevable que si **la borne de fin est la même** des deux côtés.

Attention. Un cumul n'échappe pas au piège du mois partiel, il le déplace. Le cumul des douze derniers mois vaut **5 172 198 264** FCFA au 2026-08-31, contre **4 452 117 890** FCFA pour les douze mois précédents, soit **+ 16,2 %**. Ce chiffre est comparable parce que les deux fenêtres font exactement douze mois, quelle que soit l'année civile dans laquelle elles tombent : le cumul glissant est la seule lecture annuelle qui reste juste quand l'année n'est pas finie.

Conseil professionnel. Remplissez le calendrier **par horizon**, jamais par chargement : deux ans d'avance coûtent moins de huit cents lignes et suppriment toute la classe d'erreurs décrite ici. Puis posez trois contrôles fixes, exécutés à chaque chargement. Le premier compte les jours du calendrier sans aucun fait rattaché, et les compare aux fermetures connues de l'entreprise. Le deuxième compte les dates des faits absentes du calendrier, et **échoue** au lieu de continuer : c'est ce contrôle qui aurait signalé les **13** dates de livraison hors calendrier du socle. Le troisième vérifie que la première et la dernière date du calendrier encadrent bien celles des faits, avec une marge d'au moins un exercice. Ces trois contrôles tiennent en une requête chacun et remplacent une confiance qui, elle, ne se mesure pas.

5.8 Le temps dans les six autres faits

Le socle porte **7** tables de faits et **3** manières d'y écrire le temps.

Nature du temps	Faits	Colonnes	Pourquoi
Une date native	fait de ventes	date_vente	la vente a lieu à un instant unique
Trois dates	commandes, encaissements	commandée, facturée, promise, échue, livrée, encaissée	/ le processus a trois étapes observables
Un mois	logistique, ruptures, mensuel, objectifs	stock mois (ou année et mois)	la mesure est mensuelle par nature

Les deux premiers cas se lisent avec la même dimension, jointe une fois ou trois fois. Le troisième cache une ambiguïté qu'il faut voir : le mot **mois** recouvre **deux natures**. Dans trois faits, c'est un texte de la forme `2023-01`, qui porte l'année et le mois. Dans le fait des objectifs, c'est l'entier **12** au maximum, sans l'année — l'année vit dans une colonne voisine. Un rapport qui joint ces faits au calendrier doit donc écrire deux fois la même intention, une fois pour chaque nature du mot. Rien ne le signale : la colonne porte le même nom.

Le fait de commandes montre ce que trois dates permettent : le délai moyen entre commande et livraison vaut **6,66** jours, **1 646** commandes sur **8 681** livrées dépassent la date promise, soit **19,0 %**, et le retard moyen de ces commandes vaut **10,29** jours. Ces trois mesures ne sont pas additives : le délai moyen d'un trimestre n'est pas la somme des délais moyens des trois mois. Une moyenne de délais se recalcule toujours au grain supérieur, et c'est le lien direct avec le chapitre 3 sur l'additivité.

Reste ce que le modèle ne saura jamais dire. Le fait de ventes porte **17** colonnes, et **0** d'entre elles contient une heure. L'heure de pointe, la répartition par créneau, l'affluence du samedi après midi : hors de portée. Ce n'est pas une erreur de modélisation, c'est un choix de grain qu'il faut nommer, parce qu'aucune analyse ne peut le contourner après coup.

6. Exemple concret — les 46 commandes que la jointure interne effaçait

Voici le cas, dans l'ordre où il se produit en atelier.

Une analyste veut publier le chiffre d'affaires livré par semaine. Elle écrit une jointure ordinaire entre le fait de commandes et le calendrier, filtre les commandes livrées, groupe par semaine, et obtient un tableau propre. Personne ne proteste : les semaines se suivent, les totaux se cumulent, la somme des semaines égale le total général — puisqu'il a été calculé dans la même requête.

Le contrôle arrive le lendemain, quand le service logistique annonce **6 491 975 325** FCFA de commandes livrées pour la période et que le rapport en affiche **6 459 203 250** FCFA. L'écart vaut **32 772 075** FCFA, et rien dans le rapport ne permet de le localiser.

Voici les trois nombres qui racontent tout.

Ligne du rapport	Valeur
Commandes livrées dans le fait	8 681
Commandes rendues par la jointure au calendrier	8 635
Dates de livraison hors calendrier (13 dates, du 2026-09-01 au 2026-10-14)	46 commandes

Le calendrier s'arrête au 2026-08-31. Les commandes passées en août 2026 sont livrées en septembre et parfois en octobre, et la jointure ne trouve pas ces jours : elle les élimine. Une jointure interne ne perd pas des lignes, elle perd des lignes **silencieusement** — c'est la différence entre un modèle incomplet et un modèle faux.

La correction ne touche ni le fait, ni la requête : elle touche le calendrier, prolongé de **122** jours jusqu'au 2026-12-31. Le calendrier passe de **1 339** à **1 461** lignes, les **8 681** commandes reviennent, le montant aussi, et les ventes ne bougent pas. Le coût de la correction tient en **122** lignes d'une table qu'aucune activité n'alimente, pour récupérer **32 772 075** FCFA de commandes qui existaient depuis le début.

7. Démonstration pas à pas — cinq gestes

Les cinq gestes se font dans l'ordre, et chacun produit un contrôle qui reste au modèle.

Geste 1 — vérifier la complétude du calendrier avant de s'en servir.

```
SELECT COUNT(*) AS jours,
       MIN(date) AS premier,
       MAX(date) AS dernier
FROM dim_date;
```

1 339 jours, du 2023-01-01 au 2026-08-31. Un calendrier se contrôle avant sa jointure, pas après : le même contrôle appliqué aux faits ne dirait rien de ses trous.

Geste 2 — joindre le fait au calendrier, et compter les deux côtés.

```
SELECT COUNT(*) AS jointes
FROM fait_ventes f
JOIN dim_date d ON d.date = f.date_vente;
```

240 000 lignes pour **240 000** lignes du fait : aucune perte. C'est le contrôle de recette du branchement, et il se pose une fois pour toutes.

Geste 3 — chercher ce que la jointure a laissé de côté.

```
SELECT COUNT(*) AS dates_hors_calendrier
FROM (SELECT DISTINCT date_livraison FROM fait_commandes
      WHERE date_livraison IS NOT NULL) v
LEFT JOIN dim_date d ON d.date = v.date_livraison
WHERE d.date IS NULL;
```

13 dates de livraison sortent du calendrier. Le geste est un **left join suivi d'un test de nul**, jamais une jointure interne : la jointure interne aurait rendu **8 635** lignes sans rien signaler, le left join rend les **8 681** lignes et désigne les fautives.

Geste 4 — prolonger le calendrier au lieu de corriger la requête.

```
SELECT COUNT(*) AS jours_prolonges
FROM (SELECT date FROM dim_date
      UNION ALL
      SELECT (DATE '2026-08-31' + INTERVAL (i) DAY)::DATE
      FROM generate_series(1, 122) AS t(i)) c;
```

Dans le script de chargement, les **14** autres colonnes se calculent de la même façon, à partir de la date engendrée : `EXTRACT(year FROM ...)`, `STRFTIME(..., '%Y-%m')` pour l'année-mois, et ainsi de suite. Le geste 4 ne touche ni la structure du calendrier ni son grain : il ajoute des jours.

1 461 lignes. Après prolongation, la jointure sur la date de livraison rend **8 681** lignes, retrouve les **6 491 975 325** FCFA, et il ne reste **0** date orpheline ni côté livraison ni côté date promise. Le geste 4 rend le geste 5 possible.

Geste 5 — borner la comparaison avant de publier.

```
SELECT d.annee, SUM(f.montant_ttc) AS ca
FROM fait_ventes f
JOIN dim_date d ON d.date = f.date_vente
WHERE f.est_retour = 0 AND d.mois <= 8
GROUP BY 1 ORDER BY 1;
```

Les années 2023 à 2026 rendent respectivement **2 127 742 355**, **2 527 257 254**, **2 913 055 394** et **3 360 553 372** FCFA. La comparaison des deux dernières donne **+ 15,4 %**, et c'est la seule lecture annuelle recevable au 2026-08-31. Comparer les années entières aurait donné **- 28,9 %**.

8. Erreurs fréquentes

Ce sont les six pièges de dates que le modèle du projet doit avoir fermés, dans l'ordre où ils se rencontrent en atelier.

1. **Comparer une période partielle à une période complète.** Le cas le plus fréquent, et le plus coûteux : **44,3** points d'écart entre les deux lectures du même chiffre d'affaires.
2. **Joindre un fait au calendrier en interne.** Les dates hors calendrier disparaissent sans message : **46** commandes et **32 772 075** FCFA sur le socle.
3. **Remplir le calendrier avec les dates des faits.** Un calendrier construit sur les ventes ne contient que les jours vendus, donc il ne peut pas montrer les fermetures — exactement ce qu'on lui demande. Le socle a la chance d'avoir **0** jour sans vente ; un vrai magasin en a.
4. **Confondre semaine normalisée et semaine commerciale.** L'une plafonne à **52** et l'autre à **53** sur la même période ; un rapport qui ne dit pas laquelle il suit n'est pas vérifiable.
5. **Joindre trois fois la même dimension sans alias dans la même requête.** Les trois dates d'une commande pointent vers la même table : sans alias, la requête est illisible ou refusée.

6. **Confondre l'année civile et l'exercice fiscal.** 725 jours du socle appartiennent à une autre année fiscale que leur année civile, et `periode_fiscale` promet douze périodes pour en rendre 2 : ni la borne du rapport, ni le libellé d'une colonne, ne disent ce que l'auteur croyait avoir demandé.

9. Bonnes pratiques professionnelles

1. **Remplir le calendrier par horizon**, avec au moins un exercice d'avance, et le prolonger par un script rejouable : 122 lignes ont suffi à récupérer 32 772 075 FCFA.
2. **Poser trois contrôles à chaque chargement** : jours sans fait, dates de fait hors calendrier, encadrement des bornes. Les deux premiers se comptent, le troisième se compare.
3. **Nommer le calendrier dans chaque rapport.** Une semaine sans mention de son calendrier, un trimestre sans mention de son exercice, sont des chiffres non vérifiables.
4. **Comparer à bornes égales**, et écrire la borne dans le titre du rapport : « janvier à août », jamais « 2026 contre 2025 » quand l'année n'est pas finie.
5. **Recalculer au grain supérieur toute mesure non additive.** Les délais moyens du socle (6,66 jours de commande à livraison, 10,29 jours de retard moyen) ne s'additionnent pas.
6. **Déclarer ce que le grain ne porte pas.** 0 colonne d'heure sur 17 : l'analyse par créneau horaire est impossible, et le dire évite qu'on la demande.

10. Exercice guidé

Énoncé. Le service commercial veut un rapport du chiffre d'affaires par jour de la semaine et par trimestre pour l'exercice 2025, avec la part des jours fériés. Construisez-le et rendez les quatre nombres demandés.

Étape 1. Joindre le fait de ventes au calendrier et filtrer l'année.

```
SELECT d.libelle_jour, d.trimestre, SUM(f.montant_ttc) AS ca
FROM fait_ventes f JOIN dim_date d ON d.date = f.date_vente
WHERE f.est_retour = 0 AND d.annee = 2025
GROUP BY 1, 2 ORDER BY 2, 1;
```

Étape 2. Le jour le plus fort de la semaine rend 2 663 495 054 FCFA, le plus faible 829 983 276 FCFA : le rapport vaut 3,21 pour un.

Étape 3. Le trimestre le plus lourd rend 1 405 274 549 FCFA, soit 29,7 % de l'année.

Étape 4. Les jours fériés de l'année pèsent 234 388 370 FCFA sur l'ensemble de la période du socle, soit 1,5 %.

Attendu. Quatre nombres : 2 663 495 054 · 829 983 276 · 29,7 % · 1,5 %. Le rapport doit nommer son calendrier, son exercice et sa borne de fin.

11. Exercices autonomes

Exercice 6.1. Le calendrier du socle porte 1 339 lignes. Combien en faudrait-il pour couvrir dix ans à partir du 2023-01-01, années bissextiles comprises ? Quel volume de faits cela représenterait-il, au rapport mesuré de 179,2 lignes de faits pour une ligne de calendrier ?

Exercice 6.2. Un rapport annonce une baisse de **– 28,9 %** du chiffre d'affaires 2026. Écrivez la requête qui produit ce chiffre, puis celle qui produit la lecture comparable. Donnez les deux valeurs et l'écart en points.

Exercice 6.3. Écrivez la requête qui compte les dates de livraison absentes du calendrier, puis mesurez le montant perdu par une jointure interne. Proposez une correction qui ne modifie ni le fait ni la requête.

Exercice 6.4. Le fait de commandes porte trois dates. Écrivez la requête qui donne le délai moyen de livraison et la part des commandes en retard sur la date promise. Expliquez pourquoi ces deux mesures ne s'additionnent pas d'un mois sur l'autre.

Exercice 6.5. Le 1er janvier 2023 appartient à la semaine normalisée **52**. Écrivez la requête qui le prouve, puis expliquez en trois phrases pourquoi un rapport annuel par semaine normalisée peut recouvrir deux années ou en laisser une de côté.

12. Correction détaillée

Exercice 6.1. Dix années civiles, du 2023-01-01 au 2032-12-31, comptent **3 653** jours : sept années de **365** jours et trois années bissextiles de **366** (2024, 2028 et 2032), soit **2 555** plus **1 098**. Au rapport mesuré de **179,2** lignes de faits pour une ligne de calendrier, ces **3 653** lignes de calendrier porteraient environ **654 618** lignes de faits, et c'est beaucoup pour dix ans. La conclusion utile n'est pas le nombre, c'est le rapport : le calendrier grandit de **365** ou **366** lignes par an, quand les faits grandissent de dizaines de milliers.

Exercice 6.2. La première requête groupe par année sur le fait entier et compare deux années dont l'une est partielle : **3 360 553 372** FCFA contre **4 724 700 286** FCFA, soit **– 28,9 %**. La seconde ajoute `AND d.mois <= 8` et borne les deux années aux huit premiers mois : **3 360 553 372** FCFA contre **2 913 055 394** FCFA, soit **+ 15,4 %**. L'écart entre les deux lectures vaut **44,3** points, et il tient à une clause de trois mots.

Exercice 6.3. Le comptage se fait par `LEFT JOIN` et test de valeur nulle sur le calendrier : **13** dates de livraison hors calendrier, **46** commandes, **32 772 075** FCFA. La jointure interne rend **8 635** lignes au lieu de **8 681**. La correction consiste à prolonger le calendrier de **122** jours, jusqu'au 2026-12-31 : il passe à **1 461** lignes, et les commandes livrées reviennent toutes, avec leur montant de **6 491 975 325** FCFA.

Exercice 6.4. Le délai moyen vaut **6,66** jours, la part des commandes livrées après la date promise vaut **19,0 %** — soit **1 646** commandes sur **8 681** — et le retard moyen de ces commandes vaut **10,29** jours. Ces trois nombres sont des moyennes : le délai moyen d'un trimestre n'est pas la moyenne des délais moyens des mois qui le composent, parce que les trois mois n'ont pas le même nombre de commandes. Une moyenne se recalcule toujours à partir des lignes, jamais à partir d'autres moyennes.

Exercice 6.5. La preuve tient en une requête : `SELECT date, semaine_iso FROM dim_date WHERE date = DATE '2023-01-01'` rend **52**. Un rapport annuel groupé par semaine normalisée range ce jour dans la semaine **52**, celle de décembre 2022 : l'année 2023 compte donc **52** semaines et le rapport de 2022 en compte **52** également, avec un jour à cheval. La même année, la semaine commerciale du socle monte jusqu'à **53**, sur **8** jours : deux rapports annuels bâtis sur deux calendriers différents ne se recoupent pas.

13. Mini-projet de chapitre

Objet. Doter le modèle de Sahel Distribution d'un calendrier qui ne produit plus de pertes.

Livrables.

1. Un script rejouable qui génère le calendrier du 2023-01-01 au 2027-12-31, avec les **15** colonnes du socle, les jours fériés à une liste déclarée en clair, et un identifiant de ligne.
2. Trois contrôles de complétude, chacun en une requête : jours du calendrier sans fait, dates de fait absentes du calendrier, encadrement des bornes par un exercice de marge.
3. Un rapport trimestriel du chiffre d'affaires livré, avec mention du calendrier utilisé et des bornes de comparaison.
4. Une note de trois paragraphes sur ce que le modèle ne saura jamais dire faute d'heure, et sur ce qu'il faudrait changer pour le dire.

Barème indicatif. Génération et colonnes : **3** points. Contrôles : **4** points. Rapport borné : **2** points. Note sur le grain : **1** point. Total : **10** points.

Critère de réussite. Le contrôle des dates de fait absentes du calendrier rend **0**, et le rapport affiche un chiffre d'affaires livré égal à la somme des lignes du fait, sans écart.

14. Résumé du chapitre

Notion	Mesure du socle	Ce qu'elle enseigne
Dimension de temps	1 339 lignes, du 2023-01-01 au 2026-08-31	une table qui se remplit par génération, pas par chargement
Branchement	240 000 lignes sur 240 000	le trou déclaré au chapitre 4 se ferme par une jointure
Complétude	0 jour sans vente, 0 date de vente hors calendrier	deux contrôles différents, deux réponses distinctes
Date hors calendrier	13 dates, 46 commandes, 32 772 075 FCFA	une jointure interne perd en silence
Correction	122 lignes ajoutées, 8 681 commandes retrouvées	prolonger le calendrier, pas la requête
Attributs	dimanche 5,32 %, fériés 1,5 %, quatrième trimestre 29,7 %	ce qu'aucun fait ne sait dire
Deux calendriers	semaine normalisée 52 , semaine commerciale 53	une ambiguïté à écrire, pas à corriger
Exercice fiscal	725 jours décalés, 2 valeurs de période	un libellé n'est pas une spécification
Période partielle	- 28,9 % contre + 15,4 %	44,3 points pour une borne mal choisie
Cumul glissant	5 172 198 264 FCFA, + 16,2 %	la seule lecture annuelle juste en année ouverte
Grain du temps	3 dates, 4 mois, 0 heure sur 17 colonnes	ce que le modèle ne dira jamais

Instrument. Toutes les valeurs de ce chapitre sont reproductibles : `python3 tools/modele_M13.py` exécute la section **15** de l'instrument, qui contrôle le branchement du calendrier dans les deux sens, prolonge le calendrier en mémoire, compare les deux lectures d'une année ouverte et rend les quatre textes de synthèse.

15. À retenir

Trois phrases portent ce chapitre.

1. **Une dimension de temps se remplit d'avance et se joint sans perte.** Le socle en porte **1 339** lignes et la jointure rend **240 000** lignes sur **240 000** : le travail est fait une fois, lu autant de fois qu'on veut.
2. **Une borne de fin, une jointure interne et un libellé ambigu suffisent à rendre un rapport faux.** Le même chiffre d'affaires s'écrit **- 28,9 %** ou **+ 15,4 %** selon la borne, **46** commandes disparaissent dans une jointure interne, et `periode_fiscale` promet douze périodes pour en rendre 2.
3. **Le contrôle se fait dans les deux sens.** Les jours du calendrier sans fait disent les fermetures et les pannes ; les dates des faits hors calendrier disent les horizons trop courts. Le socle répond **0** à la première question et **13** à la seconde, ce qui est exactement l'inverse de ce que le bon sens suggérerait.

À retenir. Le temps est la seule dimension que le modèle connaît avant les faits, et c'est pour cela qu'elle pardonne moins que les autres. Un client manquant se rattrape par une ligne inconnue ; un jour manquant ne se rattrape pas, parce qu'il n'existe nulle part dans les faits. Prolonger un calendrier coûte **122** lignes ; ne pas le prolonger coûte **32 772 075** FCFA qu'aucun message d'erreur ne signalera.

16. Évaluation formative

1. Pourquoi une dimension de temps existe-t-elle alors que la table de faits porte déjà une date ?
2. Combien de lignes porte le calendrier du socle, et sur quelle période ?
3. Quels sont les deux contrôles de complétude d'un calendrier, et que répond le socle à chacun ?
4. Qu'est-ce qu'une date hors calendrier, et pourquoi une jointure interne est-elle dangereuse ?
5. Combien de lignes faut-il ajouter au calendrier du socle pour le prolonger jusqu'à la fin de 2026, et qu'est-ce que cela récupère ?
6. Le dimanche pèse **5,32 %** du chiffre d'affaires et les jours fériés **1,5 %** : pourquoi ces deux nombres ne se calculent-ils pas sans le calendrier ?
7. Quelles sont les deux semaines du socle, et de combien diffèrent leurs maximums ?
8. Que fait un rapport annuel groupé par semaine normalisée du 1er janvier d'une année ?
9. Le chiffre d'affaires 2026 vaut **3 360 553 372** FCFA contre **4 724 700 286** FCFA en 2025 : cette comparaison est-elle recevable, et que faut-il écrire à la place ?
10. Que vaut, au 2026-08-31, le cumul des douze derniers mois, et pourquoi cette lecture échappe-t-elle au piège du mois partiel ?

Corrigé. 1. Parce que la date seule ne porte pas le contexte : trimestre, jour de la semaine, férié, exercice, semaine. 2. **1 339** lignes, du 2023-01-01 au 2026-08-31. 3. Les jours sans fait, qui valent **0**, et les dates de fait hors calendrier, qui valent **0** pour les ventes mais **13** pour les livraisons de commandes. 4. Une date d'un fait absente de la dimension : la jointure interne la supprime sans erreur — **46** commandes et **32 772 075** FCFA sur le socle. 5. **122** lignes, jusqu'au 2026-12-31, ce qui ramène les **8 681** commandes livrées et leurs **6 491 975 325** FCFA. 6. Parce que ni le jour de la semaine ni le statut de férié ne figurent dans le fait : ils se lisent dans la dimension, jointure faite. 7. La semaine normalisée, maximum **52**, et la semaine commerciale, maximum **53** sur **8** jours. 8. Il range ce jour dans la semaine **52**, celle de décembre de l'année précédente, et décale de fait la première semaine du rapport. 9. Non : l'année 2026 est ouverte

et compte **243** jours sur **365**. Il faut borner les deux années aux huit premiers mois, ce qui donne **+ 15,4 %** au lieu de **– 28,9 %**. 10. Il vaut **5 172 198 264** FCFA, contre **4 452 117 890** FCFA pour la fenêtre précédente, soit **+ 16,2 %** : les deux fenêtres comptent douze mois pleins, donc la comparaison est juste quelle que soit la position de l'année civile.

Module M13.C07 — La qualité du modèle, sa documentation et sa revue

Outils : DuckDB 1.5.5 (exécuté) ; pandas (contrôle croisé, exécuté). PostgreSQL et Power BI (cités, non exécutés — règle §1.5).

La grille de revue en 15 points, et les 5 défauts qu'elle attrape

sur le modèle du fil rouge, les 15 points passent ; sur le modèle fautif, 5 points échouent

- | | |
|--|---|
| ■ 1. Une cle primaire identifie la table | ■ 9. Les cle etrangeres sont declarees
0 declaree en tout et pour tout |
| ■ 2. L'unicite de la cle est prouvee | ■ 10. Les libelles viennent du referentiel
16 libelles pour 7 familles |
| ■ 3. Le grain du fait egale celui de la source | ■ 11. L'historique des changements est present |
| ■ 4. Aucune cle etrangere orpheline | ■ 12. Une ligne inconnue accueille les cle absentes |
| ■ 5. La recette egale la source de verite | ■ 13. Le calendrier est complet des deux cotes |
| ■ 6. Aucun attribut de reference recopie
2 826 394 caracteres recopies au lieu de 239 599 | ■ 14. Les mesures non additives sont isolees |
| ■ 7. Une seule table porte le temps
1 339 jours d'un cote, 44 mois de l'autre | ■ 15. La documentation nomme tables et mesures |
| ■ 8. Chaque mesure est au grain de son fait
un objectif pose sur la vente : + 42,9 % | |

Le modele du fil rouge : 14 tables sur 14, 7 faits sur 7, 12 cle etrangeres verifiees,
0 orphelin et la recette au franc pres : 15 595 154 955 FCFA des deux cotes.

Le modele fautif echoue sur les points 6 a 10 : cinq defauts que l'oeil ne voit pas, et cinq totaux faux.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

- exécuter les quatre contrôles qui décident qu'un modèle est livrable** : l'unicité des clés (**14** tables sur **14**), l'égalité du grain (**7** faits sur **7**), l'absence d'orphelin (**12** clés étrangères vérifiées, **0** orphelin) et la recette (**15 595 154 955** FCFA des deux côtés, le modèle et la source) ;
- documenter un modèle sans l'alourdir** : le script du socle compte **315** lignes dont **99** de commentaire, et il ne déclare **aucune** clé primaire, **aucune** clé étrangère et **aucun** commentaire de colonne — la garantie est déplacée des contraintes vers les contrôles, et cela se dit ;
- passer la grille de revue en 15 points** sur un modèle qu'on n'a pas écrit, et transformer chaque point en une preuve : une requête, un compte ou un nom ;
- chiffrer les cinq défauts d'un modèle fautif** : un nom de client recopié sur **2 826 394** caractères au lieu de **239 599**, deux colonnes pour dire le temps, un objectif posé sur la ligne de vente qui ajoute **42,9 %** au chiffre d'affaires, **0** clé déclarée, et **16** libellés de catégorie pour **7** familles ;

5. **refuser de livrer** : trois modèles fautifs du socle s'exécutent sans erreur et rendent **22 305 610 960 FCFA (+ 43,0 %)**, **29 474 916 530 FCFA (+ 89,0 %)** et **686 186 818 020 FCFA (44 fois le chiffre juste)**.

2. Pourquoi cette notion est importante

Un modèle de données ne se juge pas à son élégance. Il se juge à ce qu'il **prouve**, et une preuve se répète. Les six chapitres précédents ont construit un modèle : entités, normalisation, grain, étoile, historisation, calendrier. Ce chapitre répond à la seule question que se pose un responsable avant de s'appuyer dessus : **comment savez-vous que c'est juste ?**

La réponse tient en quatre contrôles qui ne coûtent rien et qui prennent l'essentiel : l'unicité des clés, l'égalité du grain, l'absence d'orphelin, la recette. Sur le socle, les quatre passent, et ils passent **parce qu'ils sont écrits** : un modèle juste dont personne ne peut le démontrer ne vaut pas mieux, en pratique, qu'un modèle faux.

Le reste du chapitre est consacré à ce qu'aucun contrôle automatique ne voit. Les contrôles comparent des comptes et des totaux ; ils ne savent pas qu'un nom de client est recopié, que la même réalité porte deux colonnes de temps, qu'un objectif a été posé au mauvais grain, ou que la catégorie du fichier source n'est pas celle du référentiel. Ces cinq-là se voient à la **lecture**, guidée par une grille de 15 points, et le chapitre les chiffre un par un.

3. Explication simple — le contrôle technique et la fiche de visite

Avant de reprendre une voiture à un garagiste, deux choses se passent, et elles ne se remplacent pas.

Le **contrôle technique** : des appareils mesurent des choses comparables à des valeurs de référence. Freinage, émissions, éclairage. Ils rendent un verdict binaire, et ils ne savent absolument pas dire si la voiture est agréable à conduire, si le siège est à la bonne hauteur ou si le coffre convient à la famille.

La **fiche de visite** : un mécanicien suit une liste écrite, point par point, et répond par oui ou par non, en montrant ce qu'il a vu. État des pneus, jeu de la direction, propreté du filtre. La liste est la même pour toutes les voitures, ce qui la rend comparable d'une visite à l'autre.

Un modèle suit exactement ce parcours. Les **quatre contrôles automatiques** sont le contrôle technique : ils comparent le modèle à sa source, à son grain déclaré et à son référentiel, et ils rendent **14** verdicts d'unicité, **7** verdicts de grain, **12** verdicts de clé étrangère et un verdict de recette. La **grille de revue en 15 points** est la fiche de visite : elle porte sur ce qu'aucun appareil ne mesure, et elle exige une réponse **documentée** à chaque point.

Un modèle qui passe les quatre contrôles et rate la grille est un modèle complet et faux. Un modèle qui passe la grille et rate un contrôle ne peut pas être livré du tout : la recette se vérifie d'abord.

4. Vocabulaire essentiel

Définition. Un **contrôle de recette** est une requête qui compare le modèle à une référence extérieure et rend un verdict binaire. Les quatre du socle comparent : les valeurs distinctes d'une clé au nombre de lignes (unicité), les lignes d'un fait aux lignes de sa source (grain), les clés étrangères au référentiel (orphelins), et le total du modèle au total de la source (recette).

Définition. L'**unicité d'une clé** est la propriété qui fait d'une colonne une clé : le nombre de valeurs distinctes égale le nombre de lignes. Elle se **prouve** en une comparaison, jamais en la déclarant. Sur le socle, **14** tables sur **14** la vérifient — la table des clients en porte **23 913** lignes et **23 913** valeurs distinctes.

Définition. Un **orphelin** est une clé étrangère qui ne trouve pas sa ligne dans le référentiel : une vente dont le produit n'existe pas. Un orphelin ne produit pas d'erreur : il produit une ligne qui disparaît d'un rapport groupé. Le socle en compte **0** sur **12** clés vérifiées, et la seule clé absente du référentiel — le client non identifié — reçoit une ligne déclarée.

Définition. La **recette d'un modèle** est l'égalité entre le total d'une mesure lue dans le modèle et le même total lu dans la source. C'est le contrôle le plus fort et le plus simple : **15 595 154 955** FCFA des deux côtés. Aucune autre vérification ne remplace celle-là, parce que c'est la seule qui porte sur le chiffre que les rapports publieront.

Définition. Un **dictionnaire de données** est la liste écrite des tables et des colonnes du modèle, avec pour chacune son type, sa définition métier, son grain et ses règles. Le socle compte **127** colonnes, la plus large des tables en portant **17** : c'est le volume à documenter, et il tient en quelques pages.

Définition. Une **revue par grille** est une relecture conduite avec une liste fermée de points, chaque point devant être justifié par une preuve. La grille du socle compte **15** points ; elle transforme « le modèle me paraît bon » en quinze réponses vérifiables par un tiers.

5. Cours approfondi

5.1 Les quatre contrôles, et ce qu'ils ont en commun

Contrôle	Question posée	Résultat sur le socle
Unicité	le nombre de valeurs distinctes égale-t-il le nombre de lignes ?	14 tables sur 14
Grain	le fait porte-t-il exactement les lignes de sa source ?	7 faits sur 7
Orphelins	chaque clé étrangère trouve-t-elle sa ligne ?	12 clés vérifiées, 0 orphelin
Recette	le total du modèle égale-t-il celui de la source ?	15 595 154 955 FCFA des deux côtés

Ces quatre contrôles ont une propriété commune qui explique leur puissance : **aucun ne regarde le modèle seul**. L'unicité compare la clé à elle-même mais aussi au grain déclaré ; le grain compare le fait à sa source ; les orphelins comparent le fait à ses dimensions ; la recette compare le modèle entier à la source. Un contrôle qui ne compare rien à rien ne vaut rien : compter les lignes d'une table et annoncer « la table contient 240 000 lignes » n'est pas un contrôle, c'est une description.

5.2 Ce qu'aucun contrôle automatique ne voit

Les quatre contrôles passent sur le socle, et ils passeraient tout aussi bien sur un modèle parfaitement faux. Voici pourquoi, en trois exemples.

Un fait dont les montants seraient ceux d'une autre période garde le même nombre de lignes et le même total : le contrôle de recette ne verrait rien. Un fait dont les catégories seraient celles du fichier source, avec ses majuscules et ses espaces, garde son grain : le contrôle de grain ne verrait rien. Un objectif posé sur la ligne de vente au lieu de la table des objectifs garde toutes ses clés valides : le contrôle d'orphelins ne verrait rien.

Ce sont exactement les trois défauts du modèle fautif de la planche, et ils sont mesurés. La conclusion est nette : **l'automatique couvre la forme, la relecture couvre le sens**. Livrer un modèle avec les quatre contrôles et sans relecture revient à livrer un modèle dont on sait seulement qu'il ne s'est pas trompé de volume.

5.3 La documentation : ce que le script dit de lui-même

Élément	Mesure
Lignes du script de construction	315
Lignes de commentaire	99
Clés primaires déclarées	0
Clés étrangères déclarées	0
Commentaires de colonne	0
Tables du modèle	14
Colonnes du modèle	127

Le script commente presque une ligne sur trois, et il ne contraint rien. Ce n'est pas un oubli : le socle est construit par des requêtes qui fabriquent les tables à partir de leurs sources, et les moteurs de cette famille n'imposent pas de contrainte sur une table fabriquée. Le choix se défend, à une condition : **ce que la base n'impose pas, le contrôle doit le prouver à chaque chargement**.

Attention. Une garantie déplacée n'est plus une garantie tant qu'elle n'est pas exécutée. Sur le socle, l'unicité des clés n'est protégée par aucune déclaration : elle est **mesurée** quatorze fois par l'instrument. Le jour où quelqu'un charge une table sans lancer l'instrument, plus rien n'empêche un doublon d'entrer. Une clé primaire déclarée se défend toute seule, la nuit et sans témoin ; un contrôle ne se défend que si quelqu'un le lance. Écrire les contrôles ne suffit pas : il faut les brancher sur le chargement, et faire échouer le chargement quand un contrôle échoue.

La documentation d'un modèle se juge donc sur deux pièces, et non une : le script commenté, qui explique **comment** le modèle est construit, et le dictionnaire de données, qui explique **ce que** chaque colonne veut dire. Le socle a la première, partiellement ; la seconde reste à écrire, et c'est l'un des livrables du projet.

5.4 Le dictionnaire de données, colonne par colonne

Le socle compte **127** colonnes. Les trois tables les plus larges sont le fait de ventes avec **17** colonnes, le calendrier avec **15** et la dimension produit avec **11**. Un dictionnaire de données utile porte quatre informations par colonne, et pas une de moins.

Information	Exemple, sur le fait de ventes	Pourquoi elle est indispensable
Nom et type	montant_ttc, décimal	deux lecteurs, deux usages
Définition métier	montant de la ligne, taxes comprises, remise déduite, hors retours	lève l'ambiguïté que le nom laisse

Information	Exemple, sur le fait de ventes	Pourquoi elle est indispensable
Grain	une ligne de ticket	dit ce qu'on a le droit d'additionner
Règles	positif pour une vente, négatif pour un retour	évite le contresens le plus coûteux

La quatrième ligne du tableau vaut **175 169 798** FCFA sur le socle : c'est le montant des **2 809** lignes de retour, rangées dans le même fait avec un drapeau et des montants négatifs. Un lecteur qui ne connaît pas la règle additionne tout et publie un chiffre d'affaires trop faible de ce montant. Le dictionnaire est le seul endroit où cette règle se lit sans exécuter une requête.

5.5 La grille de revue en 15 points

La planche de ce chapitre porte la grille complète. Les points s'ordonnent en quatre familles, et chacune répond à une question différente.

Famille	Points	Question posée
Structure	1 à 5	les clés, le grain, les liens et les totaux tiennent-ils ?
Sémantique	6 à 10	le modèle dit-il la vérité des données, ou seulement leur forme ?
Histoire	11 à 13	le temps qui passe et le temps du calendrier sont-ils maîtrisés ?
Usage	14 et 15	un lecteur qui ne connaît pas le modèle peut-il s'en servir ?

Sur le modèle du fil rouge, les **15** points passent. Sur le modèle fautif du chapitre, les points **6 à 10** échouent, et ce sont exactement les cinq défauts du paragraphe suivant. La grille n'a de valeur que si elle est appliquée **par quelqu'un qui n'a pas écrit le modèle** : un auteur relit ses intentions, un relecteur lit ce qui est écrit.

Dans les faits. La grille se remplit en une réunion, et elle produit deux choses : une liste de corrections, et une liste de justifications. La seconde est la plus utile à long terme. Un modèle livré avec ses quinze réponses écrites se reprend six mois plus tard sans que personne n'ait à deviner pourquoi les choses sont ainsi. C'est la différence entre un modèle qu'on possède et un modèle qu'on subit.

5.6 Les cinq défauts du modèle fautif, et ce que chacun coûte

Point de la grille	Défaut	Mesure sur le socle	La bonne pratique
6	attributs du client recopiés dans les faits	2 826 394 caractères recopiés au lieu de 239 599 , soit 11,8 fois	une dimension client, une clé étrangère
7	deux colonnes pour dire le temps	1 339 dates d'un côté, 44 mois de l'autre	une seule table de dates
8	objectif posé sur la ligne de vente	6 685 260 000 FCFA ajoutés au chiffre d'affaires, soit 42,9 %	un fait d'objectifs à son grain de 218 lignes
9	aucune clé déclarée	0 clé primaire et 0 clé étrangère	des contrôles exécutés à chaque chargement
10	libellés repris tels quels du fichier	16 libellés pour 7 familles réelles	la correspondance vit dans le référentiel

Aucun de ces cinq défauts n'empêche une requête de s'exécuter. C'est ce qui les rend dangereux : ils se manifestent dans les chiffres, jamais dans les messages d'erreur. Le défaut **8** est le plus coûteux, parce qu'il ne se contente pas de fausser une ligne de rapport : il fausse le total, ce que le chapitre 3 a nommé l'erreur de grain.

Le défaut **6** mérite d'être lu deux fois, parce que son coût n'est pas seulement une question de volume. Un nom recopié **11,8** fois crée autant d'endroits où la même information peut diverger : une correction d'adresse faite sur une seule ligne produit deux villes pour un client, et donc deux lignes dans un rapport par ville. Le chapitre 4 a mesuré cette conséquence sur le socle : **4** clients portent plus de deux villes dans l'historique du fait, et c'est le signe que l'information de référence a été recopiée trop longtemps.

5.7 La revue coûte une heure, l'erreur coûte un trimestre

Il reste à répondre à l'objection habituelle : pourquoi relire un modèle qui s'exécute ? Parce que trois modèles fautifs du socle s'exécutent parfaitement, et rendent des chiffres qu'aucun gestionnaire n'accepterait.

Modèle fautif	Ce qu'il fait	Chiffre d'affaires annoncé	Écart
Deux tableaux additionnés	ventes plus commandes, sans vérifier qu'ils parlent du même fait	22 305 610 960 FCFA	+ 43,0 %
Trois tableaux additionnés	on y ajoute les encaissements, troisième comptage du même argent	29 474 916 530 FCFA	+ 89,0 %
Jointure sur la mauvaise clé	ventes et logistique reliées par le seul magasin	686 186 818 020 FCFA	44 fois le chiffre juste

Ces trois requêtes ont été exécutées, et elles ont rendu ces trois nombres sans un avertissement. C'est l'argument le plus fort en faveur de la revue : **le danger n'est pas le modèle qui plante, c'est le modèle qui répond**. Une requête cassée se répare en dix minutes ; un chiffre faux publié en réunion se répare en un trimestre de crédibilité.

Attention. Un total juste ne prouve rien à lui seul. Sur le socle, les deux lectures d'un même rapport par segment somment au même **15 595 154 955** FCFA, et l'une des deux se trompe de **102 917 580** FCFA dans la répartition. La recette valide le **total** ; elle ne valide pas les **cases**. Un contrôle de recette sans contrôle de répartition laisse passer l'erreur la plus difficile à voir de tout le module.

Conseil professionnel. Automatisez les quatre contrôles dans cet ordre, et faites **échouer le chargement** quand l'un d'eux échoue : la recette d'abord, parce qu'un écart de total est le signe d'une erreur de jointure ou de filtre, qui rend tout le reste inutile ; le grain ensuite, qui attrape les multiplications ; les orphelins après, qui attrapent les lignes muettes ; l'unicité en dernier, parce qu'un doublon se voit déjà dans les trois précédents. Puis faites relire le modèle par quelqu'un d'autre avec la grille des 15 points, et gardez ses réponses. Une revue de modèle tient en une heure ; le temps qu'elle coûte est le meilleur investissement du projet.

À retenir. La qualité d'un modèle se compose de deux choses qui ne se remplacent pas : ce qu'il **prouve**, par quatre contrôles exécutés à chaque chargement, et ce qu'il **explique**, par un dictionnaire de données et une revue en **15** points. Le socle prouve beaucoup (**14** tables sur **14**, **7** faits sur **7**, **0** orphelin, la recette au franc près) et documente encore peu ; cette asymétrie est ordinaire, et c'est celle qu'un chef de projet doit savoir lire avant de signer.

6. Exemple concret — cinq défauts, cinq totaux faux

Reprenons le modèle du stagiaire, celui de la fiche de revue. Il tient en deux tables : une table de ventes à **13** colonnes et une table de magasins. Il s'ouvre, il répond aux questions simples, et il porte les cinq défauts du paragraphe 5.6.

Le premier défaut copie le nom, la ville et le segment du client sur chaque ligne de vente. Cela coûte **2 826 394** caractères de noms recopiés, contre **239 599** dans une dimension dédiée. Le rapport par ville qui en sort est vrai le jour du chargement, et faux le lendemain si une adresse change.

Le deuxième défaut garde deux colonnes de temps : une date et un mois écrit à la main. Les deux valent **1 339** jours et **44** mois, et rien ne garantit qu’elles racontent la même histoire. Un mois corrigé dans une colonne et pas dans l’autre produit un rapport hebdomadaire qui ne colle plus au rapport mensuel.

Le troisième défaut pose l’objectif du magasin sur chaque ligne de vente. L’objectif vaut **6 685 260 000** FCFA pour l’ensemble du socle ; recopié sur **240 000** lignes, il s’additionne avec elles et gonfle le total de **42,9 %**. C’est le défaut le plus coûteux des cinq, et il ne se voit pas : le rapport s’exécute, les lignes sont valides, le total est faux.

Le quatrième défaut ne déclare aucune clé : **0** clé primaire, **0** clé étrangère. Rien n’empêche d’écrire deux fois le même identifiant de vente, ni de vendre un produit qui n’existe pas.

Le cinquième défaut recopie la catégorie telle qu’elle arrive du fichier source : **16** libellés pour **7** familles réelles. Un tableau par catégorie affiche donc seize lignes dont neuf sont des doublons déguisés, avec des majuscules et des espaces en trop. La somme reste juste, la répartition est fautive — exactement le type d’erreur que le contrôle de recette laisse passer.

Corriger ces cinq défauts demande une journée. Les avoir laissés passer coûte un trimestre de décisions prises sur des chiffres faux.

7. Démonstration pas à pas — les quatre contrôles en SQL

Les quatre contrôles s’écrivent en quatre requêtes, et chacun rend un verdict binaire.

Contrôle 1 — l’unicité de la clé.

```
SELECT COUNT(*) AS lignes,
       COUNT(DISTINCT id_client) AS valeurs
FROM dim_client;
```

23 913 lignes et **23 913** valeurs : la clé est unique, le grain est prouvé. La même requête répétée sur les **14** tables du modèle donne **14** fois l’égalité.

Contrôle 2 — le grain du fait face à sa source.

```
SELECT COUNT(*) AS lignes_du_fait FROM fait_ventes;
```

240 000 lignes, à comparer aux **240 000** lignes de la source. L’égalité se lit dans le rapport d’exécution, ligne par ligne, pour les **7** faits du modèle.

Contrôle 3 — les orphelins.

```
SELECT COUNT(*) AS orphelins
FROM fait_ventes f
LEFT JOIN dim_client c ON c.id_client = f.id_client
WHERE c.id_client IS NULL;
```

0 orphelin sur la clé client, **0** sur la clé produit. Le **left join** est ici obligatoire : une jointure interne compterait **0** orphelin en supprimant les lignes fautives, ce qui est la démonstration par l’absurde du contrôle.

Contrôle 4 — la recette.

```
SELECT ROUND(SUM(montant_ttc)) AS recette
FROM fait_ventes
WHERE est_retour = 0;
```

15 595 154 955 FCFA, identiques au total de la source. Ce contrôle-là se pose en dernier dans la lecture, et en premier dans les priorités : si le total est faux, aucun autre verdict n'a d'importance.

La lecture de la grille. Les quinze points se remplissent dans l'ordre de la planche, et chacun se justifie par une preuve : une requête pour les points structurels, une mesure pour les cinq défauts, un nom de table pour les points d'usage. Un point sans preuve se note « non revu », jamais « conforme ».

8. Erreurs fréquentes

1. **Se contenter du contrôle de recette.** Un total juste peut cacher une répartition fausse de **102 917 580** FCFA, comme le montre le rapport par segment du chapitre 4.
2. **Confondre décrire et contrôler.** Compter les lignes d'une table n'est pas un contrôle : un contrôle compare à une référence extérieure.
3. **Croire qu'une contrainte déclarée est une contrainte appliquée.** Une clé primaire dans un schéma ne dit rien du chargement ; et **0** déclaration ne veut pas dire **0** garantie, si le contrôle tourne à chaque chargement.
4. **Livrer sans dictionnaire de données.** Les **127** colonnes du socle portent des règles qui ne se devinent pas : **2 809** lignes de retour à montants négatifs, un mois stocké tantôt en texte tantôt en entier, une période fiscale qui n'a que **2** valeurs.
5. **Faire relire son modèle par son auteur.** Une relecture faite par celui qui a écrit le modèle valide ses intentions, pas son résultat.
6. **Ne pas dater la revue.** Un modèle juste en janvier peut devenir faux en juillet, quand les données changent : une revue sans date n'est pas une preuve.

9. Bonnes pratiques professionnelles

1. **Brancher les quatre contrôles sur le chargement**, dans l'ordre recette, grain, orphelins, unicité, et faire échouer le chargement plutôt que publier.
2. **Écrire un dictionnaire de données** pour les **127** colonnes du modèle, avec type, définition métier, grain et règles, et le tenir à côté du script.
3. **Relire par grille**, avec les **15** points, par une personne qui n'a pas écrit le modèle, et conserver les réponses avec leur date.
4. **Prouver chaque point par une requête ou un nom**, jamais par une opinion : « le grain est bon » n'est pas une réponse, « **7** faits sur **7** portent les lignes de leur source » en est une.
5. **Déclarer les exceptions.** Le client non identifié, avec ses **43 161** ventes et **2 852 612 447** FCFA, est une exception assumée : une exception écrite ne se confond pas avec un défaut.
6. **Mesurer le coût de chaque défaut avant de le corriger**, pour arbitrer : les cinq défauts du modèle fautif ne coûtent pas la même chose, entre **0** clé déclarée et **42,9 %** de chiffre d'affaires en trop.

10. Exercice guidé

Énoncé. Vous recevez le modèle fautif et la grille en **15** points. Remplissez les points **6** à **10**, chiffrez chaque défaut, et proposez la correction de deux d'entre eux.

Étape 1. Point **6**, les attributs recopiés. Comptez les caractères de noms de clients dans le fait et dans la dimension : **2 826 394** contre **239 599**, soit **11,8** fois plus.

Étape 2. Point **7**, les deux temps. Comptez les valeurs distinctes de chaque colonne de temps : **1 339** dates et **44** mois. Deux colonnes, deux vérités possibles.

Étape 3. Point **8**, l'objectif au mauvais grain. Additionnez l'objectif avec les ventes : **6 685 260 000** FCFA de trop, soit **42,9 %** du chiffre d'affaires.

Étape 4. Points **9** et **10**. Comptez les clés déclarées, puis les libellés : **0** clé de toute nature, et **16** libellés pour **7** familles.

Attendu. Cinq réponses chiffrées, et deux corrections : une dimension client avec une clé étrangère pour le point **6** ; une table d'objectifs au grain magasin et mois pour le point **8**, ce qui ramène le total à **15 595 154 955** FCFA.

11. Exercices autonomes

Exercice 7.1. Écrivez les quatre contrôles de recette du socle, et donnez pour chacun la référence extérieure à laquelle il compare. Quels sont ceux qui ne pourraient pas passer sur un modèle faux ?

Exercice 7.2. Le contrôle d'unicité donne **23 913** lignes et **23 913** valeurs distinctes sur la dimension client. Écrivez la même requête sur le fait de ventes : la clé de vente est-elle unique, et que se passerait-il si elle ne l'était pas ?

Exercice 7.3. Un modèle publie **22 305 610 960** FCFA de chiffre d'affaires quand la source en donne **15 595 154 955**. Écrivez la requête fautive probable, calculez l'écart en pourcentage, et expliquez quel contrôle l'aurait arrêtée.

Exercice 7.4. Choisissez trois points de la grille des **15** et écrivez, pour chacun, la preuve exacte que vous demanderiez à un stagiaire : la requête, ou le nom de la table.

Exercice 7.5. Le socle déclare **0** clé primaire et **0** clé étrangère, et il compte **0** orphelin sur **12** clés vérifiées. Expliquez en trois phrases comment ces deux affirmations peuvent être vraies ensemble, et ce qui manque pour que la garantie soit permanente.

12. Correction détaillée

Exercice 7.1. L'unicité compare la clé au grain déclaré ; le grain compare le fait à sa source ; les orphelins comparent les clés étrangères au référentiel ; la recette compare le total du modèle à celui de la source. Aucun ne compare le modèle à lui-même. Les quatre peuvent passer sur un modèle faux : un fait dont les dates seraient celles d'une autre période garde son nombre de lignes, son grain et ses clés valides, et son total serait identique — c'est pourquoi la relecture existe.

Exercice 7.2. `SELECT COUNT(*) AS lignes, COUNT(DISTINCT id_vente) AS valeurs FROM fait_ventes;` rend **240 000** et **240 000** : la clé de vente est unique. Si elle ne l'était pas, chaque montant porté par un doublon serait compté deux fois dans tout rapport, et le contrôle de recette échouerait sur la différence exacte des montants dupliqués.

Exercice 7.3. La requête fautive additionne deux tables qui portent toutes les deux un montant : `SELECT (SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour) + (SELECT SUM(montant_ttc) FROM commande)`. Elle rend **22 305 610 960** FCFA, soit **+ 43,0 %** au-dessus des **15 595 154 955** FCFA de la source. Le contrôle de recette l'aurait arrêtée immédiatement, puisque son unique rôle est de comparer le total au chiffre de référence.

Exercice 7.4. Point **2**, l'unicité : `SELECT COUNT(*), COUNT(DISTINCT id_client) FROM dim_client` doit rendre deux fois le même nombre, **23 913**. Point **4**, les orphelins : un **left join** suivi d'un test de valeur nulle, qui doit rendre **0**. Point **13**, la complétude du calendrier : deux comptages, les jours du calendrier sans fait et les dates de fait sans jour correspondant — **0** et **0** pour les ventes, **13** pour les dates de livraison.

Exercice 7.5. Les deux affirmations sont vraies ensemble parce que le socle n'est pas construit par des contraintes mais par des requêtes, et que l'unicité est vérifiée **après** construction, par comptage, sur les **14** tables. Ce qui manque est la permanence : la garantie n'existe qu'au moment où l'instrument s'exécute. Pour la rendre permanente, il faut brancher le contrôle sur le chargement et refuser la publication quand il échoue.

13. Mini-projet de chapitre

Objet. Passer la grille de revue en **15** points sur votre propre modèle, et publier le dictionnaire de données.

Livrables.

1. La grille des **15** points remplie, chaque réponse accompagnée de sa preuve : une requête, un compte ou un nom. Les points non revus sont écrits « non revu », jamais « conforme ».
2. Le dictionnaire des **127** colonnes du modèle, au format tableau, avec type, définition métier, grain et règles.
3. Les quatre contrôles de recette en un seul script, exécuté et joint au rendu avec sa sortie.
4. Une note d'une page sur les trois défauts les plus coûteux trouvés, chacun chiffré, avec la correction proposée et son coût estimé.

Barème indicatif. Grille remplie et prouvée : **4** points. Dictionnaire : **3** points. Contrôles exécutés : **2** points. Note sur les défauts : **1** point. Total : **10** points.

Critère de réussite. Les quatre contrôles passent, la grille est signée et datée, et chaque point portant sur la sémantique est justifié par une mesure, pas par une intention.

14. Résumé du chapitre

Notion	Mesure du socle	Ce qu'elle enseigne
Unicité	14 tables sur 14	le grain se prouve en comparant deux comptages
Grain	7 faits sur 7	un fait porte exactement les lignes de sa source
Orphelins	12 clés vérifiées, 0 orphelin	une clé muette disparaît d'un rapport groupé
Recette	15 595 154 955 FCFA des deux côtés	la seule vérification qui porte sur le chiffre publié
Documentation	315 lignes dont 99 de commentaire	le script explique la construction, pas le sens
Contraintes	0 clé déclarée, 0 commentaire de colonne	la garantie est déplacée vers le contrôle, et doit être exécutée
Dictionnaire	127 colonnes, 17 pour le fait principal	c'est là que vivent les règles, dont celle des retours

Notion	Mesure du socle	Ce qu'elle enseigne
Grille de revue	15 points, 5 échouent sur le modèle fautif	l'automatique couvre la forme, la relecture couvre le sens
Défaut le plus coûteux	42,9 % de chiffre d'affaires en trop	un objectif au mauvais grain fausse le total, pas une case
Le danger	+ 43,0 %, + 89,0 % et 44 fois	trois requêtes fautives s'exécutent sans un avertissement
Répartition	102 917 580 FCFA d'écart pour un total juste	la recette valide le total, jamais les cases

Instrument. Les valeurs de ce chapitre sont reproductibles : `python3 tools/modele_M13.py` exécute la section **16** de l'instrument, qui compte les tables, les faits, les clés étrangères et les colonnes, mesure ce que le script déclare, chiffre les cinq défauts du modèle fautif et rend les quatre textes de synthèse de la revue.

Le module en une ligne. Sept chapitres ont transformé un fichier plat de **240 000** lignes en un modèle de **14** tables dont la recette est prouvée : entités et cardinalités, formes normales, grain et additivité, étoile et historisation, flocon et variantes, temps et calendrier, et cette revue finale.

15. À retenir

1. **Quatre contrôles suffisent à décider qu'un modèle est livrable** : unicité (**14** sur **14**), grain (**7** sur **7**), orphelins (**0** sur **12** clés) et recette (**15 595 154 955** FCFA des deux côtés). Ils ont tous la même force : ils comparent le modèle à une référence extérieure.
2. **Ce qu'un contrôle ne voit pas décide de ce qu'un modèle vaut.** Un nom recopié **11,8** fois, deux colonnes de temps, un objectif posé sur la ligne de vente (**+ 42,9** %), **0** clé déclarée, **16** libellés pour **7** familles : cinq défauts que seule la relecture attrape.
3. **Le danger n'est pas le modèle qui plante, c'est le modèle qui répond.** **+ 43,0** %, **+ 89,0** %, **44** fois : trois modèles fautifs du socle s'exécutent sans erreur.

À retenir. Un modèle de données se livre avec deux choses : la preuve qu'il est juste et la mémoire de ses choix. La preuve tient en quatre requêtes exécutées à chaque chargement ; la mémoire tient dans un dictionnaire de données et dans les **15** réponses d'une grille de revue, signées et datées. Le modèle du fil rouge a les preuves : **14** tables sur **14**, **7** faits sur **7**, **0** orphelin, la recette au franc près. Il lui manque la mémoire, et la produire est l'objet du projet.

16. Évaluation formative

1. Citez les quatre contrôles de recette et la référence extérieure de chacun.
2. Pourquoi un contrôle qui ne compare rien à une référence extérieure n'est-il pas un contrôle ?
3. Le socle déclare **0** clé primaire. L'unicité est-elle pour autant non vérifiée ? Justifiez.
4. Que manque-t-il pour que la garantie d'unicité soit permanente plutôt que ponctuelle ?
5. Combien de colonnes compte le modèle, et quelle est la table la plus large ?
6. Quelles sont les quatre familles de la grille des **15** points, et que demande chacune ?
7. Citez les cinq défauts du modèle fautif et la mesure qui prouve chacun.

8. Le défaut des attributs recopiés coûte **2 826 394** caractères au lieu de **239 599** : quelle conséquence cela a-t-il au-delà du volume ?
9. Le contrôle de recette passe sur un modèle dont le rapport par segment est faux de **102 917 580** FCFA. Comment est-ce possible, et quel contrôle faut-il ajouter ?
10. Trois modèles fautifs rendent **22 305 610 960**, **29 474 916 530** et **686 186 818 020** FCFA. Donnez l'écart de chacun par rapport au chiffre juste, et la leçon commune.

Corrigé. 1. Unicité (le grain déclaré de la table), grain (la source du fait), orphelins (le référentiel), recette (le total de la source). 2. Parce qu'il se contente de décrire : compter les lignes d'une table ne prouve aucune de ses propriétés. 3. Non : l'unicité est mesurée sur les **14** tables, et la dimension client rend **23 913** lignes pour **23 913** valeurs distinctes. 4. Le branchement du contrôle sur le chargement, avec échec du chargement en cas de verdict négatif. 5. **127** colonnes, la plus large étant le fait de ventes avec **17** colonnes. 6. Structure (clés, grain, liens, totaux), sémantique (le modèle dit-il la vérité des données), histoire (historisations et calendrier), usage (un tiers peut-il s'en servir). 7. Attributs recopiés (**2 826 394** caractères recopiés contre **239 599** dans la dimension, soit **8,5 %** du volume utile), deux temps (**1 339** dates et **44** mois), objectif sur la vente (**6 685 260 000** FCFA, **42,9 %**), **0** clé déclarée, **16** libellés pour **7** familles. 8. La divergence : la même information vit à des dizaines de milliers d'endroits, et une correction faite à un seul endroit crée deux versions de la vérité. 9. Parce que le total est juste et que les cases ne le sont pas : une jointure qui déplace des montants entre segments ne change pas la somme. Il faut ajouter un contrôle de répartition, segment par segment. 10. **+ 43,0 %**, **+ 89,0 %** et **44** fois. La leçon : une requête qui s'exécute n'est pas une requête juste, et un chiffre faux ne prévient jamais.

Projet M13.P — « Le modèle de Sahel Distribution »

Module : M13 — Modélisation des données · 4 livrables · 20 points · seuil de réussite 13 · environ 10 h de travail.

Le mandat. Sahel Distribution publie ses chiffres depuis un fichier plat de **240 000** lignes qui porte tout : la vente, le client, le produit, le magasin et le vendeur dans la même ligne. Trois rapports de la même semaine affichent trois chiffres d'affaires différents (**22 305 610 960**, **29 474 916 530** et **15 595 154 955** FCFA), et personne ne sait lequel est le bon. La direction vous mandate pour **construire le modèle** qui remplace ce fichier. Sa consigne, textuelle : « un modèle, pas un fichier ; une table par sujet ; des clés déclarées ; et la preuve que le total est le même qu'avant. »

Le modèle attendu compte 7 dimensions et 7 faits, **127** colonnes, **1 339** jours de calendrier, et sa recette doit tomber au franc près sur **15 595 154 955** FCFA.

1. Énoncé

Le matériel fourni

Pièce	Contenu
ventes_propres.csv	le fichier plat : 240 000 lignes, 11 colonnes, 26,8 Mo
brut/clients.csv brut/produits.csv brut/magasins.csv brut/vendeurs.csv	les quatre référentiels, tels qu'ils sortent du système de caisse (2 prix manquants, des libellés écrits de plusieurs façons)
brut/objectifs_de_ca.csv	218 lignes d'objectifs, au grain magasin et mois, séparateur point-virgule
dim_date.csv	1 339 jours de calendrier, du 2023-01-01 au 2026-08-31
mouvements_clients.csv	1 120 mouvements de clients, à ranger en changements lents
tarifs_produits.csv	616 versions de tarifs, à ranger de la même façon
socle_m13.sql et connexion.py	la base de travail et son ouverture, identiques à ceux des chapitres
ATTENDU.json	les mesures de référence, pour vérifier vos calculs

Les huit totaux de contrôle

Aucun livrable ne peut passer un total faux. Ces huit valeurs sont votre recette ; elles se recalculent sur le modèle que vous construisez.

Contrôle	Valeur attendue
chiffre d'affaires net	15 595 154 955 FCFA
chiffre d'affaires toutes lignes	15 419 985 157 FCFA
lignes de vente	240 000 , dont 237 191 hors retours et 2 809 retours
dimensions	clients 23 913 · produits 154 · magasins 6 · vendeurs 22 · jours 1 339
faits	ventes 240 000 · commandes 9 000 · encaissements 9 000 · objectifs 218 · logistique 264 · ruptures 2 428 · stock mensuel 6 776
enjeu de la recette	le modèle doit rendre le même total que le fichier plat

Contrôle	Valeur attendue
contrôles de recette	4 (unicité, grain, orphelins, total), tous au vert
versions historisées	24 893 versions de clients et 616 versions de produits

2. Les quatre livrables

P1 — Le modèle conceptuel et le modèle logique (6 points)

Attendu	Détail
Modèle conceptuel	les 5 entités du fil rouge (client, produit, magasin, vendeur, temps) avec leurs attributs et leurs cardinalités écrites, y compris le plusieurs à plusieurs entre tickets et produits
Modèle logique	les 7 dimensions et les 7 faits, chaque table portant sa clé et son grain écrit sous le nom de la table
Cardinalités justifiées	pour chaque relation : un à plusieurs, plusieurs à plusieurs, ou aucun, avec l'exemple du socle qui le prouve
Grain des faits	écrit en une phrase par fait, sans le mot « détail » : par exemple « une ligne de ticket »

Ce qui est éliminatoire. Un fait dont le grain n'est pas écrit, ou une relation plusieurs à plusieurs dessinée sans passer par une table.

P2 — Le script de création exécutable (6 points)

Attendu	Détail
Script rejouable	CREATE TABLE puis les chargements, exécutés deux fois de suite sans erreur et sans doublon
Clés	clés primaires et étrangères déclarées, ou bien contrôlées explicitement si votre moteur ne les impose pas
Contraintes d'intégrité	ce qui doit être refusé est refusé : un identifiant en double, une clé sans référentiel
Les 4 contrôles	unicité des clés, égalité des grains, absence d'orphelins, recette — exécutés et rendus avec leur sortie
Preuve	le rapport d'exécution joint : les 4 contrôles au vert, et le total du modèle affiché

Le test que nous appliquerons. Votre script sera relancé sur une base vide, puis les **4** contrôles seront exécutés sans que vous soyez là. Un modèle qui ne se reconstruit pas ne se corrige pas.

P3 — La revue du modèle en 15 points (5 points)

Attendu	Détail
La grille remplie	les 15 points du chapitre C07, chacun avec sa preuve : une requête, un compte ou un nom
Les défauts trouvés	la liste des défauts restants dans votre propre modèle, chiffrés, et la décision prise pour chacun (corrigé, assumé, reporté)
Les exceptions déclarées	ce que le modèle ne sait pas dire, écrit noir sur blanc : l'absence d'heure dans le fait de ventes, le client non identifié, les jours fériés travaillés
La signature	la revue est datée, et vous dites quel point vous n'avez pas pu vérifier

Rappel de méthode. Les points **6** à **10** de la grille ne se voient pas à l'exécution : ils demandent de compter, de comparer et de nommer. Un point sans preuve s'écrit « non revu », jamais « conforme ».

P4 — La note de choix (3 pages, 3 points)

Attendu	Détail
Étoile ou flocon	la variante retenue pour la hiérarchie produit, et la mesure qui tranche
Type de changement lent	le type retenu pour les clients et pour les tarifs, et ce que le type 3 aurait coûté
Dénormalisations assumées	ce que vous avez recopié exprès, combien de fois, et ce que vous y gagnez
Refus argumentés	ce que vous avez refusé de faire, et pourquoi : deux calendriers, un objectif sur la ligne de vente, un montant dans une table de pont
Le goût du socle	une phrase sur ce que vous changeriez si la direction acceptait un jour de casser le modèle

Budget. Trois pages, pas quatre. Une note de choix qui dépasse son format est une note qui n'a pas choisi.

3. Le barème

Livrable	Points
P1 — modèle conceptuel et modèle logique	6
P2 — script exécutable et 4 contrôles	6
P3 — revue en 15 points	5
P4 — note de choix, 3 pages	3
Total	20

Le seuil de réussite est fixé à **13** points sur **20**. Les **4** points d'écart entre le barème et le seuil sont votre marge : vous pouvez perdre un livrable entier et réussir le projet, à condition que le modèle se reconstruise et que la recette tombe juste.

4. Ce qui fait rejeter un livrable

1. **Le modèle ne se reconstruit pas** : le script exécuté deux fois donne deux fois plus de lignes.
2. **La recette ne tombe pas juste** : le total du modèle diffère de celui du fichier plat, quel que soit l'écart.
3. **Un fait n'a pas de grain écrit**, ou son grain est faux.
4. **Une table additionne deux grains** : un objectif posé sur la ligne de vente, un montant recopié dans une table de pont.
5. **Une clé étrangère n'a pas de référentiel** et aucune ligne inconnue ne la rattrape.
6. **La revue est signée « conforme » sans preuve** : chaque point de la grille demande un chiffre ou un nom.
7. **Une historisation dont les jointures oublient la date** : lire l'attribut d'aujourd'hui sur une vente d'avant-hier.
8. **La note de choix décrit le modèle au lieu de le justifier** : nous voulons les décisions, pas le sommaire.

5. Déroulé conseillé (4 séances, environ 10 h)

Séance	Durée	Travail	Livrable visé
1	2 h	lecture du fichier plat, relevé des 8 totaux de contrôle, entités, cardinalités	P1, modèle conceptuel
2	3 h	modèle logique, grains écrits, clés, chargements des 7 dimensions	P1 et P2
3	3 h	les 7 faits, les 4 contrôles, la recette, la reconstruction	P2
4	2 h	la grille des 15 points sur son propre modèle, puis la note de choix	P3 et P4

6. Comment vous corriger vous-même

1. **Les huit totaux** du §1 tombent juste, sur votre modèle et non sur le fichier plat.
2. **Chaque grain est écrit** en une phrase, sous le nom de chaque table de faits.
3. **Les 4 contrôles sont exécutés** et vous pouvez montrer leur sortie, pas seulement leur code.
4. **Chaque point de la grille porte une preuve** : un point sans preuve se note « non revu ».
5. **Les défauts restants sont chiffrés** et la décision est écrite, y compris la décision de ne pas corriger.
6. **La note de choix tient en 3 pages** et chaque décision porte une mesure ou un refus argumenté.
7. **Le modèle se reconstruit à l'identique** deux fois de suite, sans intervention de votre part.

Le test final. Donnez votre modèle et vos **4** contrôles à quelqu'un qui ne connaît pas le dossier, en lui demandant de refaire la recette. S'il y arrive sans vous poser de question, le projet est réussi : vous avez livré un modèle possédable, et non un fichier explicable.

Évaluation M13 — « Modélisation des données »

7 chapitres · 30 h · niveau 3 vers niveau 4 · 75 points au total · durée conseillée : 3 h.
Barème d'ensemble : quiz 15 questions (20 points, seuil 11 sur 20) ; deux exercices de normalisation (20 points) ; étude de cas (30 points, seuil 18) ; auto-test de contrôle du modèle (5 points) — soit 75 points, seuil de réussite 48.

Ce que l'évaluation vérifie. Que vous savez **relier** des entités, **normaliser** un tableau sans le mutiler, **choisir** un grain, **dater** ce qu'un modèle doit conserver, et **prouver** qu'un modèle est juste. Les requêtes sont courtes ; les décisions et les preuves sont l'essentiel.

A · Questions de récupération (non notées)

1. Quelles sont les **4** anomalies que la normalisation évite ?
2. Qu'est-ce que le **grain** d'une table, et quelle requête le prouve ?
3. Quelles mesures sont additives, semi-additives, non additives ? Donnez un exemple du socle pour chacune.
4. Qu'est-ce qu'un changement lent de type 2, et que faut-il ajouter à **chaque** jointure ?
5. Pourquoi un modèle porte-t-il **une seule** table de dates ?

B · Quiz (15 questions · 20 points)

Les questions 1 à 10 valent 1 point ; les questions 11 à 15 valent 2 points. Seuil : 11 points sur 20.

Bloc 1 — Définir (Q1 à Q5)

Q1. Une relation plusieurs à plusieurs entre deux entités se résout par : a) une colonne supplémentaire dans l'une des deux tables ; b) une troisième table qui porte les deux clés ; c) la suppression de l'une des deux tables.

Q2. La troisième forme normale interdit : a) les colonnes calculées ; b) les attributs qui ne dépendent pas de la clé ; c) les tables de plus de dix colonnes.

Q3. Le grain d'une table de faits est : a) le nombre de lignes qu'elle contient ; b) ce que représente **une** ligne, écrit en une phrase ; c) la liste de ses clés étrangères.

Q4. Une mesure semi-additive est additive : a) dans le temps mais pas dans l'espace ; b) dans l'espace mais pas dans le temps ; c) ni dans l'espace ni dans le temps.

Q5. Le type 3 d'un changement lent garde : a) toutes les versions horodatées ; b) une colonne qui porte la valeur précédente ; c) aucune trace, la valeur est écrasée.

Bloc 2 — Mesurer (Q6 à Q10)

Q6. Le socle compte **240 000** lignes de vente pour **146 161** tickets. La moyenne de lignes par ticket vaut environ : a) 0,61 ; b) 1,64 ; c) 2,40.

Q7. Un modèle affiche **31 298 080 184** FCFA là où la recette vaut **15 595 154 955** FCFA. Le facteur vaut : a) 2,01 ; b) 1,43 ; c) 0,50.

Q8. Le chiffre d'affaires du premier trimestre, lu sur un fait de ventes, est une mesure : a) non additive ; b) semi-additive ; c) additive.

Q9. Le socle compte **18** jours fériés travaillés. Où cette information se lit-elle ? a) dans le fait de ventes ; b) dans la dimension de temps ; c) dans la dimension des magasins.

Q10. Un rapport par catégorie affiche **16** lignes là où le référentiel compte **7** familles. La cause la plus probable est : a) une jointure manquante dans la requête ; b) des libellés repris tels quels du fichier source ; c) un filtre de période mal borné.

Bloc 3 — Modéliser (Q11 à Q15, 2 points chacune)

Q11. Un rapport par segment donne **15 595 154 955** FCFA au total, mais deux segments sont faux de **102 917 580** FCFA. Quelle conclusion ? a) le rapport est globalement bon ; b) un contrôle de total ne valide pas la répartition ; c) la clé de segment est dupliquée.

Q12. Pourquoi une table de pont ne doit-elle porter **aucun** montant ? a) parce qu'elle n'a pas de clé ; b) parce que son montant serait répété autant de fois que l'objet a de liens ; c) parce que le moteur refuse les sommes sur une table de pont.

Q13. Le calendrier du socle s'arrête au 2026-08-31 et **46** commandes sont livrées en septembre. La correction juste est : a) filtrer la période du rapport ; b) prolonger le calendrier ; c) joindre en interne sur la date de livraison.

Q14. Le chiffre d'affaires 2026 est comparé à celui de 2025 : **– 28,9 %**. Que faut-il faire ? a) publier le chiffre tel quel, il est mesuré ; b) borner les deux années au même mois de fin ; c) remplacer la comparaison par un cumul depuis janvier.

Q15. Un modèle déclare **0** clé primaire et **0** clé étrangère. Que peut-on en conclure ? a) le modèle est faux ; b) l'unicité et les liens ne sont pas garantis par la base, donc ils doivent être contrôlés à l'exécution ; c) le modèle ne peut pas être chargé.

C · Deux exercices de normalisation (20 points)

N1 — Normaliser une table plate en 3FN (10 points)

Voici un extrait de commandes tel qu'il arrive d'un fichier bureautique.

id_commande	date_commande	id_client	nom_client	ville_client	id_produit	designation	categorie	quantite
1041	2026-03-02	208	Diallo	Kaya	12	Panneau bois	Panels	4
1041	2026-03-02	208	Diallo	Kaya	31	Colle forte	Quincaillerie	2
1042	2026-03-05	208	Diallo	Kaya	12	Panneau bois	Panels	6
1043	2026-03-05	415	Sawadogo	Koudougou	7	Tuyau PVC	Plomberie	12
1044	2026-03-09	415	Sawadogo	Koudougou	12	Panneau bois	Panels	3
1045	2026-03-09	901	Nikiema	Ouagadougou	31	Colle forte	Quincaillerie	5
1046	2026-03-12	901	Nikiema	Ouagadougou	7	Tuyau PVC	Plomberie	8
1047	2026-03-12	901	Nikiema	Ouagadougou	12	Panneau bois	Panels	1
1048	2026-03-15	208	Diallo	Kaya	7	Tuyau PVC	Plomberie	20
1049	2026-03-15	415	Sawadogo	Koudougou	31	Colle forte	Quincaillerie	4
1050	2026-03-18	901	Nikiema	Ouagadougou	7	Tuyau PVC	Plomberie	15

id_commande	date_commande	id_client	nom_client	ville_client	id_produit	designation	categorie	quantite
1051	2026-03-18	208	Diallo	Kaya	12	Panneau bois	Panels	9

Travail demandé.

1. Écrivez le ou les défauts de cette table, dans l'ordre des formes normales : ce qui viole la première, la deuxième, la troisième.
2. Proposez le modèle normalisé : nommez chaque table, écrivez sa clé et son grain.
3. Nommez les **4** anomalies que cette organisation évite, et donnez pour chacune un exemple **pris dans ce tableau**.
4. Le prix unitaire du produit 12 est le même sur les cinq lignes : dites où il doit vivre, et ce qu'un changement de tarif implique dans les deux organisations.

Barème. Défauts identifiés : **3** points. Modèle proposé, clés et grains : **4** points. Quatre anomalies nommées et illustrées : **2** points. Emplacement du tarif et conséquence : **1** point.

N2 — Dénormaliser volontairement, et le chiffrer (10 points)

Un rapport doit afficher le chiffre d'affaires par famille de produits. Dans le modèle en étoile du socle, la famille vit dans une table de famille séparée : la requête coûte **2** jointures de plus qu'une lecture directe.

Travail demandé.

1. Écrivez les deux variantes : celle qui garde la table de familles séparée, et celle qui recopie la famille dans la dimension produit.
2. Chiffrez le coût de la dénormalisation : combien de fois la famille est-elle écrite dans chaque variante, sur les **154** produits et les **7** familles ?
3. Vérifiez que les deux variantes rendent le **même** résultat, et donnez la valeur de la famille Matériaux lue dans le modèle du socle.
4. Rédigez en cinq lignes la justification que vous donneriez à un responsable : ce que la dénormalisation fait gagner, ce qu'elle fait perdre, et ce qu'elle **ne change pas**.

Barème. Deux variantes écrites : **3** points. Chiffrage des occurrences : **3** points. Égalité des résultats vérifiée et valeur donnée : **2** points. Justification écrite : **2** points.

D · Étude de cas — « trois tableaux reliés donnent un chiffre d'affaires faux de 40 % » (30 points · seuil 18)**La commande**

Un distributeur de matériaux vous appelle. Son contrôle de gestion a additionné trois extraits — les ventes, les commandes fournisseurs et les encaissements — dans un même rapport, en les reliant par le magasin. Le rapport annonce **22 305 610 960** FCFA de chiffre d'affaires, là où la comptabilité en attend **15 595 154 955** FCFA. Le directeur financier a arrêté de publier le rapport, et votre commande tient en trois phrases : « dites-nous ce qui s'est passé, corrigez-le, et prouvez-moi que le nouveau chiffre est le bon ».

Vous disposez du socle de Sahel Distribution, qui reproduit exactement ce cas : un fait de ventes, un fait de commandes, un fait d'encaissements, et des rapports fautifs qui s'exécutent sans erreur.

Le résultat attendu (à retrouver, pas à recopier)

Étape	Ce que vous devez produire
Diagnostic	ce que fait chaque modèle fautif, et pourquoi il s'exécute sans erreur
Mesure	le chiffre annoncé par chacun, l'écart en pourcentage, et le facteur de la jointure la plus violente
Correction	la requête juste, et le chiffre de référence du socle
Preuve	les 4 contrôles de recette exécutés, dont le total
Rédaction	une note de 10 lignes pour le directeur financier, sans jargon, avec le chiffre juste

Barème de l'étude de cas

Élément	Points
Diagnostic des trois modèles fautifs, dont la cause de chacun	10
Mesure : trois chiffres, trois écarts, le facteur de la jointure	8
Correction : la requête juste et le chiffre de référence	6
Preuve : les 4 contrôles exécutés et leur sortie	4
Note au directeur financier	2
Total	30

E · Auto-test de contrôle du modèle (5 points)

Écrivez les **12** requêtes suivantes, exécutez-les sur le socle, et rendez leur sortie. Chaque requête juste vaut **0,4** point, l'ensemble étant arrondi à **5** points sur **12**.

N°	Contrôle demandé
1	le nombre de lignes et le nombre de valeurs distinctes de la clé des clients
2	la même paire de comptages sur la clé des ventes
3	le nombre de lignes du fait de ventes, comparé à sa source
4	le nombre d'orphelins de la clé produit, par jointure externe
5	le nombre d'orphelins de la clé client, la ligne inconnue mise à part
6	le chiffre d'affaires net, et le chiffre d'affaires toutes lignes
7	le total des objectifs, et sa part dans le chiffre d'affaires net
8	le nombre de jours du calendrier sans aucune vente
9	le nombre de dates de livraison absentes du calendrier
10	le nombre de versions de la dimension client historisée
11	le nombre de ventes rattachées au client non identifié
12	le nombre de caractères de noms de clients copiés dans la table plate, comparé au même texte rangé dans la dimension

Correction détaillée

A · Réponses de récupération

1. Insertion, mise à jour, suppression, redondance.

2. Ce que représente une ligne, écrit en une phrase ; il se prouve en comparant le nombre de valeurs distinctes de la clé au nombre de lignes.
3. Additive : le chiffre d'affaires (**15 595 154 955** FCFA). Semi-additive : le stock, additif dans l'espace et pas dans le temps (**6 776** lignes mensuelles). Non additive : une moyenne, comme le délai de livraison moyen (**6,66** jours).
4. Une nouvelle version à chaque changement, avec une période de validité ; chaque jointure doit porter la clause de date qui lit la version en vigueur **au moment du fait**.
5. Pour que toutes les lectures du temps soient comparables, et parce que la définition du trimestre, de la semaine ou de l'exercice doit vivre à un seul endroit.

B · Corrigé du quiz

Q	Réponse	Le point à retenir
1	b	une relation plusieurs à plusieurs exige une table intermédiaire, jamais une colonne de plus
2	b	la troisième forme normale traque la dépendance transitive, pas la taille des tables
3	b	le grain se décrit avant de se compter
4	b	le stock monte et descend dans l'espace, pas dans le temps
5	b	le type 3 garde une valeur d'avant, ce qui suffit rarement
6	b	240 000 ÷ 146 161 = 1,64 ligne par ticket
7	a	31 298 080 184 ÷ 15 595 154 955 = 2,01 : le montant répété par la table de pont
8	c	un trimestre est la somme de ses mois, donc additif
9	b	le statut de férié vit dans la dimension de temps, jamais dans le fait
10	b	16 libellés pour 7 familles : le référentiel n'a pas été appliqué
11	b	un total juste autorise une répartition fautive de 102 917 580 FCFA
12	b	le pont relie, il ne porte pas de montant : 2,01 fois le chiffre d'affaires sinon
13	b	la date hors calendrier se corrige dans le calendrier, pas dans la requête
14	b	- 28,9 % contre + 15,4 % : 44,3 points d'écart pour une borne
15	b	sans contrainte déclarée, la garantie vit dans le contrôle, et un contrôle s'exécute

C · Corrigé des exercices de normalisation

N1. La table viole la première forme normale par ses groupes qui se répètent dans la colonne désignation, la deuxième par la dépendance du nom et de la ville du client à l'identifiant de commande autant qu'à celui du client, et la troisième par la catégorie qui dépend de la désignation et non de la ligne de commande. Le modèle attendu compte quatre tables : `commande` (clé identifiant de commande, **11** lignes distinctes), `ligne_commande` (clé commande et produit, grain : une ligne de commande, **12** lignes), `client` (clé identifiant de client, **3** lignes) et `produit` (clé identifiant de produit, **3** lignes). Les quatre anomalies se lisent dans le tableau : l'insertion d'un nouveau client exige une commande (anomalie d'insertion) ; changer la ville de Diallo demande de modifier **5** lignes (mise à jour) ; supprimer les quatre commandes de Nikiema efface le client (suppression) ; et « Kaya » est écrit **5** fois (redondance). Le prix unitaire appartient à la table des produits : dans la table plate, un changement de tarif oblige à réécrire toutes les lignes du produit, ce qui a produit exactement les **616** versions de tarifs du socle.

N2. La variante en flocon joint la table des familles pour atteindre le libellé ; la variante dénormalisée le lit directement dans la dimension produit. La famille est écrite **7** fois dans la table des familles et **154** fois dans la dimension produit, soit une par produit. Les deux variantes rendent le **même** résultat : la famille Matériaux vaut **3 535 582 421** FCFA dans le modèle du socle. La justification attendue : la dénormalisation fait gagner deux jointures et rend la famille lisible

sans détour ; elle fait perdre un point de mise à jour unique, puisqu'un renommage de famille touche **154** lignes au lieu de **7** ; elle ne change **rien** au chiffre d'affaires, ni au grain, ni aux contrôles.

D · Corrigé de l'étude de cas

Diagnostic. Trois causes, toutes silencieuses. La première additionne deux tables qui portent toutes les deux un montant, sans vérifier qu'elles parlent du même fait. La deuxième y ajoute les encaissements, troisième comptage du même argent. La troisième joint les ventes à la logistique sur le seul magasin, si bien que chaque ligne de vente rencontre les **44** mois d'historique du magasin et se répète. Aucune de ces trois requêtes ne produit d'erreur : elles sont syntaxiquement justes.

Mesure. Sur le socle, les trois modèles rendent **22 305 610 960** FCFA (+ **43,0 %**), **29 474 916 530** FCFA (+ **89,0 %**) et **686 186 818 020** FCFA, soit **44** fois le chiffre juste. Le facteur de la jointure sur le magasin vaut **44**, ce qui s'entend : un magasin a **44** mois d'historique dans le fait de logistique. L'écart annoncé par le contrôle de gestion, environ **40 %**, correspond au premier de ces trois cas.

Correction. La requête juste lit un seul fait, au bon grain, avec son filtre de retours : `SELECT SUM(montant_ttc) FROM fait_ventes WHERE est_retour = 0`, et rend **15 595 154 955** FCFA. Le chiffre d'affaires toutes lignes vaut **15 419 985 157** FCFA : l'écart de **175 169 798** FCFA est exactement le montant des **2 809** retours, qu'il ne faut pas oublier de filtrer.

Preuve. Les **4** contrôles du chapitre C07 : unicité sur les **14** tables, grain sur les **7** faits, **12** clés étrangères sans orphelin, et la recette au franc près. Un modèle de **14** tables dont le total tombe juste est un modèle qu'on peut publier.

Note au directeur financier, dix lignes. Les trois extraits ne pouvaient pas s'additionner : les ventes, les commandes et les encaissements décrivent trois moments du même argent. Le rapport comptait donc une partie de l'activité deux fois et l'autre trois fois, et la jointure par magasin multipliait chaque ligne par les **44** mois d'historique. Le chiffre juste est **15 595 154 955** FCFA pour l'exercice, hors retours ; toutes lignes comprises, il vaut **15 419 985 157** FCFA, la différence étant les **2 809** retours de l'année. Nous avons ajouté **4** contrôles qui s'exécutent à chaque chargement et refusent la publication en cas d'écart : le total, le grain, les liens et l'unicité des clés. Aucun rapport ne sortira désormais sans eux.

E · Corrigé de l'auto-test

N°	Réponse
1	23 913 lignes, 23 913 valeurs distinctes
2	240 000 lignes, 240 000 valeurs distinctes : la clé de vente est unique
3	240 000 lignes des deux côtés
4	0 orphelin
5	0 orphelin, hors les 43 161 ventes de la ligne déclarée
6	15 595 154 955 FCFA hors retours, 15 419 985 157 FCFA toutes lignes
7	6 685 260 000 FCFA, soit 42,9 % du chiffre d'affaires net
8	0 jour
9	13 dates, pour 46 commandes et 32 772 075 FCFA
10	24 893 versions, pour 900 clients ayant une histoire
11	43 161 ventes, 2 852 612 447 FCFA
12	2 826 394 caractères recopiés, contre 239 599 dans la dimension

Rappel du barème d'ensemble : quiz **20** pts (seuil **11**) + normalisation **20** pts + étude de cas **30** pts (seuil **18**) + auto-test **5** pts = **75** points, **seuil de réussite 48**.

Pour aller plus loin. Le projet **M13.P** « Le modèle de Sahel Distribution » (**20** points, seuil **13**) demande les quatre livrables du module : modèle conceptuel et logique, script exécutable avec ses **4** contrôles, revue en **15** points, et note de choix de **3** pages. C'est la **4^e** pièce du dossier de portfolio, après les rapports SQL de M11 et la carte de **10** indicateurs de M12.